

M7 PRACTICA

MÓDULO 7 — Práctica: construcción paso a paso de la API de becas

Esta práctica construye un único proyecto Spring Boot completo para gestionar becas, alumnos, solicitudes, documentos, usuarios y roles. El proyecto se prepara desde cero y después se desarrolla de forma incremental a lo largo de los puntos 7.1-7.7.

La secuencia debe seguirse en orden: cada paso deja preparado el estado que necesita el siguiente. Los comandos, ficheros, clases y verificaciones indicados forman parte del resultado final del módulo.

Convención de terminal. Los bloques marcados como `bash` están escritos para Git Bash, WSL, Linux o macOS. En Windows con `cmd.exe`, usa `mvnw.cmd` en lugar de `./mvnw`; para los ejemplos `curl` multilínea utiliza Git Bash/WSL o ejecuta el comando equivalente en una sola línea. No mezcles sintaxis Bash (`\`, `$(...)`, variables `$TOKEN`) con CMD.



PREPARACIÓN DEL PROYECTO M7

Antes de comenzar el punto 7.1, prepara la infraestructura mínima que utilizará el proyecto completo.



PREP-01 — Crear un proyecto Spring Boot nuevo

Crea una carpeta independiente `M7/proyecto` y genera un proyecto Maven con Java 17 y paquete base `es.mecd.demo.miproyecto`. Incluye:

- Spring Web.
- Spring Validation.
- Spring Data JPA.
- Spring Security.
- H2.
- Springdoc OpenAPI.
- JWT (`jjwt-api`, `jjwt-impl`, `jjwt-jackson`).
- Spring Boot Test y Spring Security Test.

El `pom.xml` final añadirá JaCoCo 0.8.11 en 7.7.2. Crea también `MiProyectoApplication` y los perfiles `application.properties`, `application-dev.properties` y `application-test.properties`.

Resultado esperado de PREP-01: el proyecto arranca vacío y no depende de ningún repositorio de M1-M6.

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-02 — Crear el soporte mínimo de Alumno

Crea dentro del proyecto el soporte mínimo del recurso Alumno:

```
alumno/  
├─ Curso.java  
├─ CursoRepository.java  
├─ Alumno.java  
├─ AlumnoRepository.java  
├─ AlumnoRequestDTO.java  
├─ AlumnoResponseDTO.java  
├─ AlumnoService.java  
└─ AlumnoController.java
```

Alumno contiene `id`, `nombre`, `apellidos`, `dni`, `fechaNacimiento` y una relación `@ManyToOne` con `Curso`. El controlador implementa los cinco endpoints de alumnos definidos en 7.1: listar, consultar, crear, actualizar y eliminar. La ruta anidada `/alumnos/{id}/solicitudes` se añadirá más adelante en el reto 7.5.12.

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-03 — Crear el contrato común de errores

Crea ahora el contrato común de errores dentro de `common/`:

```
common/  
├─ dto/error/ErrorResponse.java  
├─ dto/error/ValidationError.java  
└─ exception/  
    ├─ AplicacionException.java  
    ├─ NegocioException.java  
    ├─ RecursoNoEncontradoException.java  
    ├─ RecursoDuplicadoException.java  
    ├─ OperacionNoPermitidaException.java  
    ├─ ValidacionNegocioException.java  
    ├─ ErrorResponseFactory.java  
    ├─ BusinessExceptionHandler.java  
    ├─ ValidationExceptionHandler.java  
    └─ GenericExceptionHandler.java
```

Todas las capas de M7 reutilizarán estas excepciones; no se crean manejadores distintos por recurso.

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-04 — Crear Usuario y Rol

El diseño M7 necesita `Usuario` → `Rol` `@ManyToMany`. Crea `Usuario`, `Rol`, `UsuarioRepository` y `RolRepository`. En este punto también crea los DTO de registro/login/refresh, pero **todavía no** `UsuarioService` ni `UsuarioController`: esos dos se incorporan en los pasos 7.5.5 y 7.5.6.

Roles iniciales de preparación: `USER` y `ADMIN`. Los roles de dominio `CIUDADANO` y `GESTOR` se incorporan en 7.6.1 y `CONSULTOR` en 7.6.12.

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-05 — Crear autenticación JWT dentro de M7

Crea la infraestructura de autenticación JWT del proyecto:

```
auth/
├── JwtConfig.java
├── JwtService.java
├── JwtAuthenticationFilter.java
├── UsuarioDetailsService.java
├── AuthService.java
├── AuthController.java
├── JwtAuthenticationEntryPoint.java
├── JwtAccessDeniedHandler.java
├── SecurityErrorWriter.java
├── CredencialesInvalidasException.java
└── TokenInvalidoException.java
```

`AuthController` expone exactamente los tres endpoints diseñados en 7.1: `/registro`, `/login` y `/refresh`. La respuesta del login usa `access_token` y `refresh_token`, porque los escenarios 7.6–7.7 extraen el token con `jq -r '.access_token'`.

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-06 — Crear una seguridad base y CORS

Crea un `SecurityConfig` inicial stateless con CSRF desactivado, CORS, el filtro JWT y los manejadores 401/403. La allowlist de autenticación se limita desde el principio a `POST /api/v1/auth/registro`, `POST /api/v1/auth/login` y `POST /api/v1/auth/refresh`; Swagger/H2 se permiten para el entorno docente y el resto exige autenticación. **En 7.6.2 se completa esta configuración con la matriz de permisos del dominio.**

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-07 — Crear datos mínimos de desarrollo

Crea un `CommandLineRunner` con `@Profile("dev")` que deje disponibles:

- un curso y un alumno con ID generado para poder crear solicitudes;
- un usuario `admin` con `ADMIN`;
- un usuario `user` con `USER`.

No cargues todavía las becas del ejercicio: se añaden en 7.2.8. No añadas todavía `CIUDADANO`, `GESTOR` ni `CONSULTOR`: pertenecen a 7.6.

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

PREP-08 — Verificar el punto de partida autónomo

Ejecuta:

```
./mvnw test
./mvnw spring-boot:run
```

Comprueba que el contexto arranca, H2 está disponible en desarrollo y `/api/v1/auth/login` responde. A partir de aquí comienza el punto 7.1.

◆ Punto 7.1 — Diseño de una API REST completa

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Definir el alcance y los recursos de una API REST realista.
2. Modelar el dominio con entidades, relaciones y constraints.
3. Diseñar los endpoints con URLs, métodos y códigos de estado coherentes.
4. Identificar los DTOs necesarios y su propósito.
5. Definir las reglas de negocio y los roles de seguridad.
6. Documentar el diseño en un formato reutilizable.
7. Justificar cada decisión de diseño.

VERIFICACIÓN DEL DISEÑO

Código reproducible. Los ficheros exactos de este PREP están en el **Anexo A — Código exacto del PREP autónomo** al final de esta guía. El anexo forma parte de la práctica y fija el estado compilable que debe existir antes de comenzar 7.1.

Paso 7.1.01 — Crear el documento de diseño

Vamos a crear un documento de diseño en el proyecto. Crea el archivo `docs/disenho-api.md` con el siguiente contenido:

```
# Diseño de la API de Gestión de Becas

## Recursos

- Beca: tipo de beca (Beca General, Ayuda de Libros...).
- Alumno: alumno que solicita la beca.
- SolicitudBeca: solicitud de beca de un alumno.
- Documento: documento adjunto a una solicitud.
- Usuario: usuario del sistema.

## Relaciones

- Solicitud → Alumno (@ManyToOne)
- Solicitud → Beca (@ManyToOne)
- Solicitud → Documento (@OneToMany)
- Usuario → Rol (@ManyToMany)

## Nota sobre visibilidad
En un sistema real, un ciudadano solo vería sus propias solicitudes.
En este curso simplificamos y permitimos que cualquier usuario
autenticado vea todas las solicitudes.
```

Endpoints

```

| Método | URL | Actor | Código |
|---|---|---|---|
| POST | /api/v1/auth/registro | Público | 201 |
| POST | /api/v1/auth/login | Público | 200 |
| POST | /api/v1/auth/refresh | Público | 200 |
| GET | /api/v1/becas | Público | 200 |
| GET | /api/v1/becas/{id} | Público | 200 |
| POST | /api/v1/becas | ADMIN | 201 |
| PUT | /api/v1/becas/{id} | ADMIN | 200 |
| DELETE | /api/v1/becas/{id} | ADMIN | 204 |
| GET | /api/v1/alumnos | Autenticado | 200 |
| POST | /api/v1/alumnos | CIUDADANO | 201 |
| PUT | /api/v1/alumnos/{id} | CIUDADANO | 200 |
| DELETE | /api/v1/alumnos/{id} | ADMIN | 204 |
| GET | /api/v1/solicitudes | Autenticado | 200 |
| GET | /api/v1/solicitudes/{id} | Autenticado | 200 |
| POST | /api/v1/solicitudes | CIUDADANO | 201 |
| PATCH | /api/v1/solicitudes/{id}/estado | GESTOR | 200 |
| DELETE | /api/v1/solicitudes/{id} | CIUDADANO | 204 |
| GET | /api/v1/solicitudes/{id}/documentos | Autenticado | 200 |
| POST | /api/v1/solicitudes/{id}/documentos | CIUDADANO | 201 |
| DELETE | /api/v1/solicitudes/{id}/documentos/{docId} | CIUDADANO | 204 |
| GET | /api/v1/usuarios | ADMIN | 200 |
| GET | /api/v1/usuarios/{id} | ADMIN | 200 |
| PATCH | /api/v1/usuarios/{id}/roles | ADMIN | 200 |

```

Roles

- ****CIUDADANO:**** crear solicitudes, consultar solicitudes y añadir documentos. Esta edición no modela ownership Usuario → Alumno.
- ****GESTOR:**** ver todas, cambiar estados, añadir observaciones.
- ****ADMIN:**** administración completa.
- ****CONSULTOR:**** se incorpora en 7.6.12 para consultas GET sin permisos de modificación.

Reglas de negocio

1. Un alumno no puede tener dos solicitudes activas para la misma beca en el mismo año.
2. Solo se puede modificar una solicitud si está en BORRADOR o ENVIADA.
3. Solo se puede cambiar el estado si está en ENVIADA o EN_REVISION.
4. Una solicitud APROBADA no se puede modificar ni eliminar.
5. El importeConcedido es obligatorio si el estado es APROBADA.
6. El importeConcedido no puede superar el importeMaximo de la beca.
7. Al eliminar una solicitud, se eliminan sus documentos (cascade).
8. Una beca no se puede eliminar si tiene solicitudes.
9. El año de la beca debe ser el actual o el siguiente.
10. Un documento solo se puede añadir si la solicitud está en BORRADOR o ENVIADA.
11. Un documento no puede superar 10 MB.
12. Los tipos de documento permitidos son PDF, JPG y PNG.

Paso 7.1.02 — Revisar el diseño con una checklist

Vamos a revisar que el diseño cubre todos los aspectos:

Aspecto	Cubierto
Recursos identificados	Beca, Alumno, SolicitudBeca, Documento, Usuario
Relaciones definidas	@ManyToOne, @OneToMany, @ManyToMany
Campos con constraints	Sí
Endpoints con método, URL, actor y código	Tabla completa
DTOs de entrada y salida	Definidos por recurso
Reglas de negocio	12 reglas
Roles y permisos	CIUDADANO, GESTOR, ADMIN; CONSULTOR se incorpora en 7.6.12
Estrategia de seguridad	JWT, stateless, CORS
Estrategia de errores	Estructura estándar
Estrategia de testing	Tests de integración

Paso 7.1.03 — Identificar dependencias entre puntos

El diseño debe dejar claro qué se implementa en cada punto del Módulo 7:

Punto	Qué se implementa	Depende de
7.2	Entidades	Nada
7.3	Repositorios	7.2
7.4	Servicios	7.3
7.5	Controladores	7.4
7.6	Seguridad	7.5
7.7	Tests de integración y cierre	7.6

Paso 7.1.04 — Crear el paquete base del proyecto

Vamos a crear la estructura de paquetes del proyecto:

```

es.mecd.demo.miproyecto
├── auth/                ← Creado en PREP-04/PREP-05
├── beca/
│   ├── Beca.java
│   ├── BecaRepository.java
│   ├── BecaService.java
│   ├── BecaController.java
│   ├── BecaRequestDTO.java
│   └── BecaResponseDTO.java
├── alumno/              ← Creado en PREP-02
│   ├── solicitud/
│   │   ├── SolicitudBeca.java
│   │   ├── EstadoSolicitud.java
│   │   ├── SolicitudBecaRepository.java
│   │   ├── SolicitudService.java
│   │   ├── SolicitudController.java
│   │   ├── SolicitudRequestDTO.java
│   │   ├── SolicitudResponseDTO.java
│   │   └── CambioEstadoRequestDTO.java
│   ├── documento/
│   │   ├── Documento.java
│   │   ├── DocumentoRepository.java
│   │   ├── DocumentoService.java
│   │   ├── DocumentoController.java
│   │   ├── DocumentoRequestDTO.java
│   │   └── DocumentoResponseDTO.java
│   └── common/
│       ├── exception/    ← Creado en PREP-03
│       ├── config/       ← Creado en PREP-06/PREP-07
│       └── controller/   ← Soporte común del proyecto

```



ERRORES COMUNES

Error	Causa	Solución
No documentar el diseño	Se empieza a codificar sin plan	Crear el documento de diseño
Recursos mal definidos	Se mezclan conceptos	Separar Beca, Solicitud y Documento
Relaciones mal modeladas	Se usa @OneToMany sin mappedBy	Añadirlo
Faltan endpoints	No se cubren todos los casos de uso	Revisar la tabla de endpoints
Faltan reglas de negocio	Se codifica sin definirlas	Documentarlas antes
Roles mal definidos	Se usa un único rol para todo	Definir CIUDADANO, GESTOR y ADMIN
Errores sin contrato común	No se define la estructura de error	Usar el contrato común del proyecto



Resumen del punto

- Alcance: API REST de gestión de becas.
- Actores: ciudadano, gestor, administrador.
- Recursos: Beca, Alumno, SolicitudBeca, Documento, Usuario.
- Relaciones: @ManyToOne, @OneToMany, @ManyToMany.
- Endpoints: 24 endpoints base con método, URL, actor y código; los retos 7.5.12 y 7.7.13 elevan el proyecto final a 26 endpoints.
- DTOs: entrada con validaciones, salida con resúmenes.
- Reglas de negocio: 12 reglas en el servicio.
- Seguridad: JWT con roles CIUDADANO, GESTOR, ADMIN y CONSULTOR en el proyecto final.
- Errores: estructura estándar.



Conclusión y enlace al siguiente punto

En este punto 7.1 hemos diseñado la API completa antes de escribir código. Hemos definido:

- Los recursos y sus relaciones.
- Los endpoints con sus actores y códigos de estado.
- Los DTOs de entrada y salida.
- Las reglas de negocio.
- La estrategia de seguridad y errores.

La idea clave: el diseño es la mitad del trabajo. Si el diseño está bien, la implementación es mecánica. Si el diseño está mal, cada punto del proyecto se convierte en una batalla.

En el siguiente punto, 7.2 – Implementación de entidades, crearemos las entidades JPA (Beca, SolicitudBeca, Documento) con sus relaciones, constraints y anotaciones. Veremos cómo se mapean a la base de datos y verificaremos el esquema generado.



Fin del Punto 7.1.

◆ Punto 7.2 — Implementación de entidades



Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Crear entidades JPA para el dominio de becas.
2. Modelar relaciones @ManyToOne y @OneToMany con mappedBy y @JoinColumn.
3. Aplicar constraints (nullable, unique, length, precision) según las reglas del dominio.
4. Configurar cascade y orphanRemoval según la naturaleza de cada relación.
5. Verificar el esquema generado por Hibernate.
6. Diagnosticar y resolver los errores más comunes al definir entidades con relaciones.

✂ IMPLEMENTACIÓN

🔧 Paso 7.2.01 — Crear el enum EstadoSolicitud

Crea EstadoSolicitud en el paquete solicitud:

```
package es.mecd.demo.miproyecto.solicitud;

public enum EstadoSolicitud {
    BORRADOR,
    ENVIADA,
    EN_REVISION,
    SOLICITUD_DOCUMENTACION,
    APROBADA,
    DENEGADA,
    CANCELADA
}
```

Cada valor representa un estado del ciclo de vida de una solicitud. El ciclo habitual avanza desde BORRADOR a ENVIADA y EN_REVISION, con resolución en APROBADA o DENEGADA y la posibilidad de solicitar documentación adicional. CANCELADA es terminal. La entidad guardará el enum como STRING, no como ordinal.

🔧 Paso 7.2.02 — Crear la entidad Beca

Crea la clase Beca en el paquete beca:

```
package es.mecd.demo.miproyecto.beca;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "becas")
public class Beca {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String nombre;

    @Column(length = 500)
    private String descripcion;

    @Column(name = "importe_maximo", nullable = false, precision = 10, scale = 2)
    private BigDecimal importeMaximo;

    @Column(nullable = false)
    private Integer anio;

    @Column(nullable = false)
    private Boolean activa = true;

    public Beca() {
    }

    public Beca(String nombre, String descripcion, BigDecimal importeMaximo, Integer anio) {
        this.nombre = nombre;
        this.descripcion = descripcion;
        this.importeMaximo = importeMaximo;
        this.anio = anio;
        this.activa = true;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
```



```

public BigDecimal getImporteMaximo() { return importeMaximo; }
public void setImporteMaximo(BigDecimal importeMaximo) { this.importeMaximo = importeMaximo; }

public Integer getAnio() { return anio; }
public void setAnio(Integer anio) { this.anio = anio; }

public Boolean getActiva() { return activa; }
public void setActiva(Boolean activa) { this.activa = activa; }
}

```

Explicación línea por línea

Línea 1: `package es.mecd.demo.miproyecto.beca;` → Paquete por funcionalidad.

Líneas 3-5: imports de JPA y BigDecimal.

Línea 7: `@Entity` → marca la clase como entidad JPA.

Línea 8: `@Table(name = "becas")` → nombre de la tabla en la base de datos. En plural, como es habitual.

Línea 10: `@Id` → marca el campo como clave primaria.

Línea 11: `@GeneratedValue(strategy = GenerationType.IDENTITY)` → el ID lo genera la base de datos con autoincrement.

Línea 12: `private Long id;` → tipo Long, no long, para distinguir "sin ID" de "ID 0".

Línea 14: `@Column(nullable = false, unique = true, length = 100)` → el nombre es obligatorio, único y de máximo 100 caracteres. La restricción unique la aplica la base de datos.

Línea 15: `private String nombre;` → el campo.

Línea 17: `@Column(length = 500)` → la descripción puede ser nula, pero si se pone, máximo 500 caracteres. Línea 18: `private String descripcion;` → el campo.

Línea 20: `@Column(name = "importe_maximo", nullable = false, precision = 10, scale = 2)` → nombre de la columna en snake_case (convención), obligatorio, con 10 dígitos totales y 2 decimales.

- `precision = 10` significa que el número puede tener hasta 10 dígitos en total.
- `scale = 2` significa que 2 de esos dígitos son decimales. Así, el máximo valor sería 99999999.99.

Línea 21: `private BigDecimal importeMaximo;` → BigDecimal, no Double. Los importes no admiten errores de precisión.

Línea 23: `@Column(nullable = false)` → el año es obligatorio.

Línea 24: `private Integer anio;` → Integer, no int. Y el nombre es anio (sin ñ) porque algunos motores de base de datos tienen problemas con caracteres no ASCII en nombres de columnas.

Línea 26: `@Column(nullable = false)` → el campo activa es obligatorio.

Línea 27: `private Boolean activa = true;` → por defecto, una beca nueva está activa.

Líneas 29-32: constructor sin argumentos. Obligatorio para JPA.

Líneas 34-40: constructor con argumentos. Comodidad para crear becas en el código y en los tests.

Líneas 42-56: getters y setters. JPA los usa para acceder a los campos. No hay equals/hashCode porque en este proyecto las becas no se comparan entre sí en colecciones.



Paso 7.2.03 — Crear la entidad SolicitudBeca

Crea la clase SolicitudBeca:

```

package es.mecd.demo.miproyecto.solicitud;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.documento.Documento;
import jakarta.persistence.*;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "solicitudes")
public class SolicitudBeca {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "alumno_id", nullable = false)
    private Alumno alumno;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "beca_id", nullable = false)
    private Beca beca;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private EstadoSolicitud estado = EstadoSolicitud.BORRADOR;

    @Column(name = "fecha_solicitud", nullable = false)
    private LocalDateTime fechaSolicitud;

    @Column(name = "fecha_resolucion")
    private LocalDateTime fechaResolucion;

    @Column(name = "importe_concedido", precision = 10, scale = 2)
    private BigDecimal importeConcedido;

    @Column(length = 1000)
    private String observaciones;

    @OneToMany(mappedBy = "solicitud", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Documento> documentos = new ArrayList<>();

    public SolicitudBeca() {
    }

    public void addDocumento(Documento documento) {
        documentos.add(documento);
        documento.setSolicitud(this);
    }

    public void removeDocumento(Documento documento) {
        documentos.remove(documento);
        documento.setSolicitud(null);
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Alumno getAlumno() { return alumno; }
    public void setAlumno(Alumno alumno) { this.alumno = alumno; }

    public Beca getBeca() { return beca; }
    public void setBeca(Beca beca) { this.beca = beca; }

    public EstadoSolicitud getEstado() { return estado; }
    public void setEstado(EstadoSolicitud estado) { this.estado = estado; }

    public LocalDateTime getFechaSolicitud() { return fechaSolicitud; }
    public void setFechaSolicitud(LocalDateTime fechaSolicitud) { this.fechaSolicitud = fechaSolicitud; }

    public LocalDateTime getFechaResolucion() { return fechaResolucion; }
    public void setFechaResolucion(LocalDateTime fechaResolucion) { this.fechaResolucion =
        fechaResolucion; }

    public BigDecimal getImporteConcedido() { return importeConcedido; }

```

```

    public void setImporteConcedido(BigDecimal importeConcedido) { this.importeConcedido =
importeConcedido; }

    public String getObservaciones() { return observaciones; }
    public void setObservaciones(String observaciones) { this.observaciones = observaciones; }

    public List<Documento> getDocumentos() { return documentos; }
    public void setDocumentos(List<Documento> documentos) { this.documentos = documentos; }
}

```

Explicación línea por línea

Líneas 1-12: paquete e imports. Fíjate en que importamos Alumno, Beca y Documento, aunque todavía no exista el último (lo crearemos en el paso 4). Por eso el IDE te marcará Documento como error hasta que lo crees.

Línea 14: @Entity → marca la clase como entidad.

Línea 15: @Table(name = "solicitudes") → la tabla se llama solicitudes, en plural. Línea 17-18: @Id y @GeneratedValue → igual que en Beca.

Línea 20: @ManyToOne(fetch = FetchType.LAZY) → relación muchos-a-uno hacia Alumno. LAZY para no cargar el alumno si no se necesita.

Línea 21: @JoinColumn(name = "alumno_id", nullable = false) → la columna de clave foránea en la tabla solicitudes se llama alumno_id y es obligatoria.

Línea 22: private Alumno alumno; → el campo de la relación.

Línea 24-26: relación @ManyToOne hacia Beca, con columna beca_id obligatoria.

Línea 28: @Enumerated(EnumType.STRING) → el enum se guarda como string en la base de datos. Sin esta anotación, JPA usaría ORDINAL por defecto, lo cual es peligroso.

Línea 29: @Column(nullable = false, length = 30) → el estado es obligatorio y ocupa hasta 30 caracteres.

Línea 30: private EstadoSolicitud estado = EstadoSolicitud.BORRADOR; → por defecto, una solicitud nueva está en BORRADOR. Es una decisión de negocio: el ciudadano puede preparar la solicitud sin enviarla.

Línea 32: @Column(name = "fecha_solicitud", nullable = false) → la fecha de solicitud es obligatoria. Se asigna en el servicio al crear la solicitud.

Línea 33: private LocalDateTime fechaSolicitud; → LocalDateTime, no LocalDate, porque interesa la hora exacta de la creación.

Línea 35: @Column(name = "fecha_resolucion") → la fecha de resolución es opcional. Solo se rellena cuando la solicitud pasa a APROBADA o DENEGADA.

Línea 36: private LocalDateTime fechaResolucion; → el campo. Línea 38: @Column(name = "importe_concedido", precision = 10, scale = 2) → el importe concedido es opcional. Solo se rellena al aprobar.

Línea 39: private BigDecimal importeConcedido; → BigDecimal.

Línea 41: @Column(length = 1000) → observaciones opcionales, hasta 1000 caracteres.

Línea 42: private String observaciones; → el campo.

Líneas 44-45: @OneToMany(mappedBy = "solicitud", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY) → relación uno-a-muchos hacia Documento.

- mappedBy = "solicitud" → el lado propietario es el campo solicitud de Documento.
- cascade = CascadeType.ALL → las operaciones se propagan: guardar la solicitud guarda los documentos, eliminar la solicitud elimina los documentos.

- orphanRemoval = true → si un documento se quita de la lista, se elimina de la base de datos.
- fetch = FetchType.LAZY → los documentos no se cargan hasta que se accede a ellos.

Línea 46: private List documentos = new ArrayList<>(); → la lista de documentos, inicializada para evitar NullPointerException.

Líneas 48-49: constructor sin argumentos. Obligatorio.

Líneas 51-54: método helper addDocumento. Añade el documento a la lista y establece la solicitud en el documento. Así ambos lados de la relación quedan consistentes.

Líneas 56-59: método helper removeDocumento. Hace lo contrario.

Líneas 61-91: getters y setters.



Paso 7.2.04 — Crear la entidad Documento

Crea la clase Documento:

```
package es.mecd.demo.miproyecto.documento;

import es.mecd.demo.miproyecto.solicitud.SolicitudBeca;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.Lob;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.LocalDateTime;

@Entity
@Table(name = "documentos")
public class Documento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "solicitud_id", nullable = false)
    private SolicitudBeca solicitud;

    @Column(nullable = false, length = 255)
    private String nombre;

    @Column(nullable = false, length = 50)
    private String tipo;

    @Column(nullable = false)
    private Long tamano;

    @Lob
    @Column(name = "contenido")
    private byte[] contenido;

    @Column(name = "fecha_subida", nullable = false)
    private LocalDateTime fechaSubida;

    public Documento() {
    }

    public Documento(String nombre, String tipo, Long tamano, byte[] contenido) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.tamano = tamano;
        this.contenido = contenido == null ? null : contenido.clone();
        this.fechaSubida = LocalDateTime.now();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
```

```

    public SolicitudBeca getSolicitud() { return solicitud; }
    public void setSolicitud(SolicitudBeca solicitud) { this.solicitud = solicitud; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }
    public Long getTamano() { return tamano; }
    public void setTamano(Long tamano) { this.tamano = tamano; }
    public byte[] getContenido() { return contenido == null ? null : contenido.clone(); }
    public void setContenido(byte[] contenido) { this.contenido = contenido == null ? null :
contenido.clone(); }
    public LocalDateTime getFechaSubida() { return fechaSubida; }
    public void setFechaSubida(LocalDateTime fechaSubida) { this.fechaSubida = fechaSubida; }
}

```

Explicación línea por línea

La implementación usa copia defensiva (`clone()`) al recibir y devolver `byte[]`, evitando que código externo modifique el contenido almacenado por referencia.

Líneas 1-7: paquete e imports.

Línea 9-10: `@Entity` y `@Table(name = "documentos")`.

Líneas 12-13: `@Id` y `@GeneratedValue`.

Línea 15: `@ManyToOne(fetch = FetchType.LAZY)` → relación muchos-a-uno hacia `SolicitudBeca`. Esta es la parte propietaria de la relación.

Línea 16: `@JoinColumn(name = "solicitud_id", nullable = false)` → la columna de clave foránea en la tabla `documentos` se llama `solicitud_id` y es obligatoria.

Línea 17: `private SolicitudBeca solicitud;` → el campo.

Línea 19: `@Column(nullable = false, length = 255)` → el nombre del documento es obligatorio.

Línea 20: `private String nombre;` → el campo.

Línea 22-23: tipo del documento (PDF, JPG, PNG). Obligatorio.

Línea 25-26: tamaño en bytes. Obligatorio.

Línea 28: `@Lob` → el contenido es un objeto grande. Con `byte[]`, se mapea a BLOB.

Línea 29: `@Column(name = "contenido")` → el nombre de la columna. Línea 30: `private byte[] contenido;` → el contenido del fichero. Opcional (puede ser nulo si el fichero se guarda en disco y solo se guarda la ruta).

Línea 32-33: fecha de subida. Obligatoria. Se asigna en el constructor con argumentos.

Líneas 35-36: constructor sin argumentos. Obligatorio.

Líneas 38-44: constructor con argumentos. Fíjate en que asigna `fechaSubida = LocalDateTime.now()` automáticamente. Así no hay que hacerlo en el servicio.

Líneas 46-70: getters y setters.



Paso 7.2.05 — Ampliar el Alumno con la relación inversa (opcional)

En el punto 7.1 decidimos **no añadir** una relación inversa `@OneToMany` en `Alumno`. No es necesaria para las operaciones del proyecto y añadirla complicaría el modelo.

Como referencia, una relación inversa se podría expresar así:

```

@OneToMany(mappedBy = "alumno", fetch = FetchType.LAZY)
private List<SolicitudBeca> solicitudes = new ArrayList<>();

```

No añadas ese código en este proyecto. Para listar las solicitudes de un alumno utilizaremos consultas en `SolicitudBecaRepository`, que se crea en 7.3.

Paso 7.2.06 — Arrancar y ver el esquema generado

Reinicia la aplicación. En los logs verás el SQL que Hibernate genera:

```
create table becas (
  id bigint generated by default as identity,
  activa boolean not null,
  anio integer not null,
  descripcion varchar(500),
  importe_maximo decimal(10,2) not null,
  nombre varchar(100) not null,
  primary key (id)
)

create table solicitudes (
  id bigint generated by default as identity,
  fecha_solicitud timestamp(6) not null,
  fecha_resolucion timestamp(6),
  importe_concedido decimal(10,2),
  observaciones varchar(1000),
  estado varchar(30) not null,
  alumno_id bigint not null,
  beca_id bigint not null,
  primary key (id)
)

create table documentos (
  id bigint generated by default as identity,
  contenido blob,
  fecha_subida timestamp(6) not null,
  nombre varchar(255) not null,
  tamano bigint not null,
  tipo varchar(50) not null,
  solicitud_id bigint not null,
  primary key (id)
)

Y después las restricciones:

sql
alter table if exists becas add constraint UK_... unique (nombre)
alter table if exists solicitudes add constraint FK_... foreign key (alumno_id) references alumnos
alter table if exists solicitudes add constraint FK_... foreign key (beca_id) references becas
alter table if exists documentos add constraint FK_... foreign key (solicitud_id) references solicitudes
```

Observa:

- La tabla becas tiene las columnas que definimos, con los tipos y constraints correctos.
- La tabla solicitudes tiene dos columnas de clave foránea (alumno_id y beca_id), porque tiene dos relaciones @ManyToOne.
- La tabla documentos tiene una columna de clave foránea (solicitud_id), porque tiene una relación @ManyToOne.
- importe_maximo es decimal(10,2), no double. La precisión está garantizada.
- estado es varchar(30), no int. Se guarda como string.
- No hay tablas intermedias. Las relaciones @ManyToOne no las necesitan.

Paso 7.2.07 — Verificar en la consola de H2

Abre la consola de H2 (<http://localhost:8080/h2-console>) y ejecuta:

```
SELECT * FROM becas;
SELECT * FROM solicitudes;
SELECT * FROM documentos;
```

Las tres tablas existen, están vacías (todavía no hemos cargado datos) y tienen las columnas esperadas.

Ejecuta también:

```
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_name IN ('BECAS', 'SOLICITUDES', 'DOCUMENTOS');
```

Verás las restricciones de clave primaria, clave foránea y unique.

Paso 7.2.08 — Añadir datos iniciales

Amplía `DatosInicialesBecasConfig` para cargar las becas **sin adelantar** `BecaRepository`, que no se crea hasta 7.3.1. En este punto `Beca` todavía no tiene código, por lo que se utiliza su constructor de cuatro argumentos.

Sustituye el contenido de `DatosInicialesBecasConfig.java` por:

```
package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.alumnoCurso;
import es.mecd.demo.miproyecto.alumnoCursoRepository;
import es.mecd.demo.miproyecto.beca.Beca;
import jakarta.persistence.EntityManager;
import jakarta.transaction.Transactional;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.Year;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
public class DatosInicialesBecasConfig {
    @Bean
    @Profile("dev")
    CommandLineRunner cargarDatos(
        EntityManager entityManager,
        CursoRepository cursos,
        AlumnoRepository alumnos) {
        return args -> cargar(entityManager, cursos, alumnos);
    }

    @Transactional
    void cargar(EntityManager entityManager, CursoRepository cursos, AlumnoRepository alumnos) {
        Long totalBecas = entityManager.createQuery("select count(b) from Beca b", Long.class)
            .getSingleResult();
        if (totalBecas == 0) {
            int anio = Year.now().getValue();
            entityManager.persist(new Beca("Beca General",
                "Ayuda económica general para estudiantes", new BigDecimal("1500.00"), anio));
            entityManager.persist(new Beca("Ayuda de Libros",
                "Ayuda para la compra de libros de texto", new BigDecimal("300.00"), anio));
            entityManager.persist(new Beca("Beca de Comedor",
                "Ayuda para el servicio de comedor escolar", new BigDecimal("800.00"), anio));
        }
        if (alumnos.count() == 0) {
            Curso curso = cursos.findByNombre("5º Primaria")
                .orElseGet(() -> cursos.save(new Curso("5º Primaria")));
            alumnos.save(new Alumno(
                "Ana", "García", "12345678A", LocalDate.of(2010, 5, 12), curso));
        }
    }
}
```

`EntityManager` es temporal. En 7.3.1, cuando exista `BecaRepository`, lo sustituiremos por el repositorio definitivo. El alumno y el curso del PREP se conservan.

Paso 7.2.09 — Verificar los datos iniciales

Reinicia la aplicación. En los logs verás los INSERT:

```
insert into becas (activa, anio, descripcion, importe_maximo, nombre, id) values (?, ?, ?, ?, ?, default)
```

Y si consultas en la consola de H2:

```
sql
SELECT * FROM becas;
```

Verás las tres becas.

Paso 7.2.10 — Errores comunes del ejercicio

Error	Causa	Solución
Table not found	ddl-auto no está configurado	Usar create-drop o update en desarrollo
No default constructor for entity	Falta el constructor sin argumentos	Añadirlo
Unable to determine type for column	Falta @Enumerated en el enum	Usar EnumType.STRING
Unique index or primary key violation	Se intenta guardar una beca duplicada	Validar antes en el servicio
Could not determine type for byte[]	Falta @Lob	Añadirlo
Foreign key constraint fails	Alumno o beca inexistente	Validar referencias antes de guardar
LazyInitializationException	Se accede a una relación LAZY fuera de transacción	Cargar dentro de @Transactional
Cannot resolve symbol Documento	La clase todavía no existe	Crearla en el paso 4
MappingException: Property 'solicitud' not found	mappedBy no coincide con el campo	Corregir el nombre
StackOverflowError al serializar	Relación bidireccional expuesta directamente	Usar DTOs

Paso 7.2.11 — Verificar las relaciones con un test rápido

En este punto todavía no existe `SolicitudBecaRepository`; se crea en 7.3.3. Verifica las relaciones con `TestEntityManager`. Como código todavía no se ha añadido a `Beca`, este test usa el constructor de cuatro argumentos.

```
package es.mecd.demo.miproyecto.solicitud;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.Curso;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.documento.Documento;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;

@DataJpaTest
class SolicitudBecaRelacionesTest {

    @Autowired
    private TestEntityManager entityManager;

    @Test
    void guardarSolicitudConDocumentos_debePersistirLaRelacion() {
        int anio = java.time.Year.now().getValue();
        Curso curso = entityManager.persistAndFlush(new Curso("5º Primaria"));
        Alumno alumno = entityManager.persistAndFlush(
            new Alumno("Ana", "García", "12345678A", LocalDate.of(2010, 5, 12), curso));
        Beca beca = entityManager.persistAndFlush(
```



```

        new Beca("Beca General", "Descripción", new BigDecimal("1500.00"), anio));

    SolicitudBeca solicitud = new SolicitudBeca();
    solicitud.setAlumno(alumno);
    solicitud.setBeca(beca);
    solicitud.setFechaSolicitud(LocalDate.now());
    solicitud.addDocumento(new Documento("DNI.pdf", "application/pdf", 3L, new byte[]{1, 2, 3}));
    solicitud.addDocumento(new Documento("Notas.pdf", "application/pdf", 3L, new byte[]{4, 5, 6}));

    entityManager.persistAndFlush(solicitud);
    Long id = solicitud.getId();
    entityManager.clear();

    SolicitudBeca recuperada = entityManager.find(SolicitudBeca.class, id);
    assertNotNull(recuperada);
    assertEquals(2, recuperada.getDocumentos().size());
    assertEquals("Beca General", recuperada.getBeca().getNombre());
    assertEquals("Ana", recuperada.getAlumno().getNombre());
    }
}

```

En 7.2.12, al incorporar `Beca.codigo`, actualizaremos la construcción de `Beca` en este test. Así el test es ejecutable tanto antes como después del reto.

Paso 7.2.12 — Reto resuelto — Añadir un campo codigo a Beca

Reto. Añade a `Beca` un campo `codigo` único, obligatorio y de máximo 20 caracteres, por ejemplo `BECA-GEN-2026`.

Paso 1 — Añadir el campo.

```

@Column(nullable = false, unique = true, length = 20)
private String codigo;

```

Paso 2 — Añadir getter y setter.

```

public String getCodigo() { return codigo; }
public void setCodigo(String codigo) { this.codigo = codigo; }

```

Paso 3 — Sustituir el constructor por la versión con código.

```

public Beca(String codigo, String nombre, String descripcion,
            BigDecimal importeMaximo, Integer anio) {
    this.codigo = codigo;
    this.nombre = nombre;
    this.descripcion = descripcion;
    this.importeMaximo = importeMaximo;
    this.anio = anio;
    this.activa = true;
}

```

Paso 4 — Actualizar `DatosInicialesBecasConfig` sin adelantar repositorios.

En las tres llamadas `entityManager.persist(new Beca(...))`, añade el código como primer argumento:

```

entityManager.persist(new Beca("BECA-GEN-" + anio, "Beca General",
    "Ayuda económica general para estudiantes", new BigDecimal("1500.00"), anio));
entityManager.persist(new Beca("LIBROS-" + anio, "Ayuda de Libros",
    "Ayuda para la compra de libros de texto", new BigDecimal("300.00"), anio));
entityManager.persist(new Beca("COMEDOR-" + anio, "Beca de Comedor",
    "Ayuda para el servicio de comedor escolar", new BigDecimal("800.00"), anio));

```

Paso 5 — Actualizar el test de relaciones.

Sustituye la construcción de la beca del paso 7.2.11 por:

```
Beca beca = entityManager.persistAndFlush(
    new Beca("TEST-" + anio, "Beca General", "Descripción",
        new BigDecimal("1500.00"), anio));
```

Paso 6 — Reiniciar y verificar en H2.

```
SELECT codigo, nombre FROM becas;
```

No crees todavía DTOs ni servicios que pertenecen a 7.4. Cuando llegues a 7.3.1, `BecaRepository` incluirá `findByCodigo` y `existsByCodigo`; y en 7.4.1–7.4.2 los DTOs y `BecaService` incorporarán `codigo` desde su primera versión.

✓ Resultado esperado global

Al final del ejercicio tienes `EstadoSolicitud`, `Beca`, `SolicitudBeca` y `Documento`; los datos de desarrollo se cargan sin adelantar repositorios; `Beca.codigo` ya es obligatorio y único; y el test JPA de relaciones utiliza el constructor vigente.

🔧 Paso 7.3.01 — Crear el `BecaRepository`

Crea `BecaRepository` ya coherente con el reto `Beca.codigo` de 7.2.12:

```
package es.mecd.demo.miproyecto.beca;

import java.util.List;
import java.util.Optional;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface BecaRepository extends JpaRepository<Beca, Long> {
    Optional<Beca> findByNombre(String nombre);
    Optional<Beca> findByCodigo(String codigo);
    boolean existsByNombre(String nombre);
    boolean existsByCodigo(String codigo);
    List<Beca> findByActivaTrue();
    List<Beca> findByAnioAndActivaTrue(Integer anio);
    List<Beca> findByAnioBetween(Integer anioDesde, Integer anioHasta);

    @Query("SELECT b FROM Beca b "
        + "WHERE (:activa IS NULL OR b.activa = :activa) "
        + "AND (:anio IS NULL OR b.anio = :anio)")
    Page<Beca> buscarConFiltros(
        @Param("activa") Boolean activa,
        @Param("anio") Integer anio,
        Pageable pageable);
}
```

Ahora que el repositorio existe, elimina el acceso temporal mediante `EntityManager` de `DatosInicialesBecasConfig` y sustituye el fichero por su versión definitiva:

```
package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.alumnoCurso;
import es.mecd.demo.miproyecto.alumnoCursoRepository;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.beca.BecaRepository;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.Year;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
```

```

@Configuration
public class DatosInicialesBecasConfig {
    @Bean @Profile("dev")
    CommandLineRunner cargarDatos(BecaRepository becas, CursoRepository cursos, AlumnoRepository alumnos)
    {
        return args -> {
            int anio = Year.now().getValue();
            if (becas.count() == 0) {
                becas.save(new Beca("BECA-GEN-" + anio, "Beca General",
                    "Ayuda económica general para estudiantes", new BigDecimal("1500.00"), anio));
                becas.save(new Beca("LIBROS-" + anio, "Ayuda de Libros",
                    "Ayuda para la compra de libros de texto", new BigDecimal("300.00"), anio));
                becas.save(new Beca("COMEDOR-" + anio, "Beca de Comedor",
                    "Ayuda para el servicio de comedor escolar", new BigDecimal("800.00"), anio));
            }
            if (alumnos.count() == 0) {
                Curso curso = cursos.findByNombre("5º Primaria").orElseGet(() -> cursos.save(new Curso("5º
Primaria")));
                alumnos.save(new Alumno("Ana", "García", "12345678A", LocalDate.of(2010, 5, 12), curso));
            }
        };
    }
}

```

A partir de este punto los datos iniciales de becas se gestionan con `BecaRepository`; no vuelve a utilizarse `EntityManager` para esta carga.

Paso 7.3.02 — Crear el DocumentoRepository

Crea la interfaz `DocumentoRepository` en el paquete `documento`:

```

package es.mecd.demo.miproyecto.documento;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DocumentoRepository extends JpaRepository<Documento, Long> {

    List<Documento> findBySolicitudId(Long solicitudId);

    long countBySolicitudId(Long solicitudId);

    void deleteBySolicitudId(Long solicitudId);
}

```

 Explicación línea por línea Línea 1: `package es.mecd.demo.miproyecto.documento;` → el paquete del recurso.

Líneas 3-5: imports.

Línea 7: `public interface DocumentoRepository extends JpaRepository<Documento, Long> {` → hereda CRUD, paginación y ordenación.

Línea 9: `List findBySolicitudId(Long solicitudId);` → busca los documentos de una solicitud.

- Qué hace: genera `SELECT d FROM Documento d WHERE d.solicitud.id = ?`. Fíjate en que Spring Data navega por la relación: `SolicitudId` se interpreta como `solicitud.id`.
- Por qué: para listar los documentos de una solicitud.
- Error común: poner `findBySolicitud(Long id)` → Spring Data busca el campo `solicitud` de tipo `Solicitud`, no su `id`. Hay que añadir `Id` al final.

Línea 11: `long countBySolicitudId(Long solicitudId);` → cuenta los documentos de una solicitud.

- Qué hace: genera `SELECT COUNT(d) FROM Documento d WHERE d.solicitud.id = ?`.
- Por qué: para el campo `numeroDocumentos` del DTO de respuesta de solicitud.
- Error común: el tipo de retorno es `long`, no `int`. Spring Data lo devuelve como `long`.

Línea 13: `void deleteBySolicitudId(Long solicitudId);` → elimina los documentos de una solicitud.

- Qué hace: genera `DELETE FROM Documento d WHERE d.solicitud.id = ?`.
- Por qué: útil cuando se quiere eliminar los documentos de una solicitud sin eliminar la solicitud. Aunque el `cascade` se encarga de esto automáticamente al eliminar la solicitud, este método es útil para otros casos.
- Error común: no envolver la llamada en una transacción → Spring lanza `InvalidDataAccessApiUsageException` porque `deleteBy` requiere una transacción. Se soluciona anotando el método del servicio con `@Transactional`.



Paso 7.3.03 — Crear el `SolicitudBecaRepository` (parte 1)

Crea la interfaz `SolicitudBecaRepository` en el paquete `solicitud`:

```
package es.mecd.demo.miproyecto.solicitud;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

public interface SolicitudBecaRepository extends JpaRepository<SolicitudBeca, Long> {

    // Consultas derivadas simples

    List<SolicitudBeca> findByAlumnoId(Long alumnoId);

    List<SolicitudBeca> findByBecaId(Long becaId);

    List<SolicitudBeca> findByEstado(EstadoSolicitud estado);

    Page<SolicitudBeca> findByEstado(EstadoSolicitud estado, Pageable pageable);

    List<SolicitudBeca> findByFechaSolicitudBetween(LocalDateTime desde, LocalDateTime hasta);

    boolean existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween(
        Long alumnoId, Long becaId,
        LocalDateTime desde, LocalDateTime hasta);

    boolean existsByBecaId(Long becaId);
}
```



Explicación línea por línea

Líneas 1-11: `package` e imports. Fíjate en que importamos `Page`, `Pageable`, `@EntityGraph`, `@Query` y `@Param`. Línea 13: `public interface SolicitudBecaRepository extends JpaRepository<SolicitudBeca, Long> {` → hereda CRUD, paginación y ordenación.

Línea 17: `List findByAlumnoId(Long alumnoId);` → solicitudes de un alumno.

- Qué hace: `SELECT s FROM SolicitudBeca s WHERE s.alumno.id = ?`.
- Por qué: un ciudadano solo ve sus solicitudes (filtradas por el alumno al que pertenecen).
- Error común: olvidar `Id` al final → Spring Data busca el campo `alumno` completo.

Línea 19: `List findByBecaId(Long becaId);` → solicitudes de una beca.

- Qué hace: `SELECT s FROM SolicitudBeca s WHERE s.beca.id = ?`.
- Por qué: para listar todas las solicitudes de un tipo de beca.

Línea 21: `List findByEstado(EstadoSolicitud estado);` → solicitudes en un estado concreto.

- Qué hace: `SELECT s FROM SolicitudBeca s WHERE s.estado = ?`.

- Por qué: los gestores filtran por estado (por ejemplo, ENVIADA para ver las pendientes).

Línea 23: `Page findByEstado(EstadoSolicitud estado, Pageable pageable);` → versión paginada del anterior.

- Qué hace: igual que el anterior, pero paginado. Spring Data genera la consulta de conteo automáticamente.
- Por qué: para listados grandes, se pagina.
- Error común: confundir List con Page. Si el método devuelve Page, hay que aceptar Pageable.

Línea 25: `List findByFechaSolicitudBetween(LocalDateTime desde, LocalDateTime hasta);` → solicitudes en un rango de fechas.

- Qué hace: `SELECT s FROM SolicitudBeca s WHERE s.fechaSolicitud BETWEEN ? AND ?.`
- Por qué: para informes y estadísticas por periodo.

Líneas 27-29: `boolean existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween(Long alumnoId, Long becaId, LocalDateTime desde, LocalDateTime hasta);` → comprueba si existe una solicitud de un alumno para una beca en un rango de fechas.

- Qué hace: `SELECT COUNT(s) > 0 FROM SolicitudBeca s WHERE s.alumno.id = ? AND s.beca.id = ? AND s.fechaSolicitud BETWEEN ? AND ?.`
- Por qué: implementa la regla de negocio “un alumno no puede tener dos solicitudes activas para la misma beca en el mismo año”.
- Error común: el nombre del método se hace muy largo. Es una limitación de los métodos derivados; cuando el nombre supera 4-5 condiciones, conviene pasar a @Query.

Línea 31: `boolean existsByBecaId(Long becaId);` → comprueba si existe alguna solicitud para una beca.

- Qué hace: `SELECT COUNT(s) > 0 FROM SolicitudBeca s WHERE s.beca.id = ?.`
- Por qué: lo usa el BecaService para comprobar si una beca tiene solicitudes antes de eliminarla.
- Error común: olvidar Id al final → Spring Data busca el campo beca completo.

Paso 7.3.04 — Añadir consultas con @Query y JOIN FETCH al SolicitudBecaRepository (parte 2)

Añade al SolicitudBecaRepository los siguientes métodos:

```
@Query("SELECT s FROM SolicitudBeca s "
      + "JOIN FETCH s.alumno "
      + "JOIN FETCH s.beca "
      + "WHERE s.id = :id")
Optional<SolicitudBeca> findByIdConRelaciones(@Param("id") Long id);

@Query("SELECT s FROM SolicitudBeca s JOIN FETCH s.alumno JOIN FETCH s.beca")
List<SolicitudBeca> findAllConRelaciones();

@Query(
    value = "SELECT s FROM SolicitudBeca s "
      + "JOIN FETCH s.alumno a "
      + "JOIN FETCH s.beca b "
      + "WHERE (:estado IS NULL OR s.estado = :estado) "
      + "AND (:alumnoId IS NULL OR a.id = :alumnoId) "
      + "AND (:becaId IS NULL OR b.id = :becaId)",
    countQuery = "SELECT COUNT(s) FROM SolicitudBeca s "
      + "WHERE (:estado IS NULL OR s.estado = :estado) "
      + "AND (:alumnoId IS NULL OR s.alumno.id = :alumnoId) "
      + "AND (:becaId IS NULL OR s.beca.id = :becaId)")
Page<SolicitudBeca> buscarConFiltros(
    @Param("estado") EstadoSolicitud estado,
```

```
@Param("alumnoId") Long alumnoId,
@Param("becaId") Long becaId,
Pageable pageable);
```

Explicación línea por línea

La consulta paginada declara una `countQuery` explícita sin `JOIN FETCH`; así la consulta de conteo de `Page` es válida y no arrastra asociaciones que sólo pertenecen a la carga de datos.

Línea 1-5: consulta `findByIdConRelaciones`.

- `@Query("SELECT s FROM SolicitudBeca s JOIN FETCH s.alumno JOIN FETCH s.beca WHERE s.id = :id")` → JPQL que carga la solicitud, su alumno y su beca en una sola consulta.
- `JOIN FETCH s.alumno` → carga el alumno en la misma consulta. Sin el `FETCH`, solo se uniría para filtrar, pero no se cargaría.
- `JOIN FETCH s.beca` → lo mismo para la beca.
- `WHERE s.id = :id` → filtra por ID.
- `:id` → parámetro nombrado.
- `Optional findByIdConRelaciones(@Param("id") Long id)`; → devuelve `Optional` porque puede no existir.
- Por qué: al consultar una solicitud, casi siempre se necesitan el alumno y la beca. Con `JOIN FETCH` se cargan en una sola consulta.
- Error común: usar `JOIN` (sin `FETCH`) → la relación no se carga, y al acceder a ella se lanza una consulta adicional (problema N+1).

Línea 7-10: consulta `findAllConRelaciones`.

- `@Query("SELECT s FROM SolicitudBeca s JOIN FETCH s.alumno JOIN FETCH s.beca")` → carga todas las solicitudes con sus relaciones.
- Sin `WHERE` → devuelve todas.
- Por qué: para listados sin filtros donde se necesitan el alumno y la beca.
- Error común: olvidar `JOIN FETCH` → problema N+1.

Línea 12-20: consulta `buscarConFiltros`.

- `@Query(...)` → JPQL con filtros condicionales.
- `JOIN FETCH s.alumno a` → carga el alumno con alias `a`.
- `JOIN FETCH s.beca b` → carga la beca con alias `b`.
- `WHERE (:estado IS NULL OR s.estado = :estado)` → filtro condicional. Si `:estado` es nulo, la condición es verdadera y no se filtra. Si no, se filtra por estado.
- `AND (:alumnoId IS NULL OR a.id = :alumnoId)` → lo mismo para el alumno.
- `AND (:becaId IS NULL OR b.id = :becaId)` → lo mismo para la beca.
- `Page buscarConFiltros(...)` → devuelve `Page` porque acepta `Pageable`.
- Por qué: permite combinar filtros opcionales en una sola consulta.
- Error común: olvidar el `IS NULL OR` → si un filtro es nulo, la consulta no devuelve nada.

Paso 7.3.05 — Añadir `@EntityGraph` para listados simples (parte 3)

Añade métodos con `@EntityGraph` al `SolicitudBecaRepository`:

```
@EntityGraph(attributePaths = {"alumno", "beca"})
Page<SolicitudBeca> findAll(Pageable pageable);

@EntityGraph(attributePaths = {"alumno", "beca"})
Page<SolicitudBeca> findByAlumnoId(Long alumnoId, Pageable pageable);

@EntityGraph(attributePaths = {"alumno", "beca"})
Page<SolicitudBeca> findByBecaId(Long becaId, Pageable pageable);
```

 Explicación línea por línea `@EntityGraph(attributePaths = {"alumno", "beca"})` → anotación que indica a JPA que cargue esas relaciones de forma ansiosa en la consulta generada. Es una alternativa a JOIN FETCH cuando no queremos escribir la consulta a mano.

`Page findAll(Pageable pageable);` → sobrescribe el `findAll` de `JpaRepository` para añadir el `@EntityGraph`. Spring Data detecta la anotación y genera la consulta con los JOIN FETCH correspondientes.

`Page findByAlumnoId(Long alumnoId, Pageable pageable);` → lista paginada de solicitudes de un alumno, con relaciones cargadas.

`Page findByBecaId(Long becaId, Pageable pageable);` → lo mismo para una beca.

Motivo para usar `@EntityGraph` en lugar de `@Query`:

- `@EntityGraph` es más declarativo. Solo indicas qué relaciones cargar, y Spring Data genera la consulta.
- `@Query` te da más control, pero tienes que escribir el JPQL a mano.

Para listados simples, `@EntityGraph` es más limpio. Para consultas complejas (con filtros condicionales), `@Query` es más potente.

Error común: usar `@EntityGraph` con relaciones `@OneToMany` → puede provocar filas duplicadas. En nuestro caso, solo cargamos `@ManyToOne`, así que no hay problema.

Paso 7.3.06 — Añadir consultas de agregación

Añade al `SolicitudBecaRepository` métodos de agregación:

```
@Query("SELECT s.estado, COUNT(s) FROM SolicitudBeca s GROUP BY s.estado")
List<Object[]> contarPorEstado();

@Query("SELECT s.beca.nombre, COUNT(s) FROM SolicitudBeca s " +
        "GROUP BY s.beca.nombre ORDER BY COUNT(s) DESC")
List<Object[]> contarPorBeca();
```

 Explicación línea por línea

Línea 1-2: `contarPorEstado`.

- `SELECT s.estado, COUNT(s)` → devuelve dos columnas: el estado y el número de solicitudes en ese estado.
- `GROUP BY s.estado` → agrupa por estado.
- `List<Object[]>` → cada elemento de la lista es un array de dos objetos: el estado y el count.
- Por qué: para estadísticas (cuántas solicitudes hay en cada estado).
- Error común: devolver `List` → Spring Data no puede mapear dos columnas a una entidad. Hay que devolver `Object[]` o un DTO.

Línea 4-6: `contarPorBeca`.

- `SELECT s.beca.nombre, COUNT(s)` → nombre de la beca y número de solicitudes.
- `GROUP BY s.beca.nombre` → agrupa por nombre de beca.
- `ORDER BY COUNT(s) DESC` → ordena por número de solicitudes, de mayor a menor.

- Por qué: para saber qué becas son las más solicitadas.
- Error común: olvidar el ORDER BY → el orden es indefinido.

Paso 7.3.07 — Arrancar y verificar

Reinicia la aplicación. En los logs verás que Spring Data JPA valida todos los métodos derivados y las consultas @Query al arrancar. Si algún nombre de método está mal o alguna consulta tiene un error de sintaxis, verás un error y la aplicación no arrancará.

Si todo va bien:

```
Started MiProyectoApplication in X.XXX seconds
```

Paso 7.3.08 — Verificar las consultas con un test

Crea un test para verificar que las consultas funcionen. Crea `SolicitudBecaRepositoryTest`:

```
package es.mecd.demo.miproyecto.solicitud;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.beca.BecaRepository;
import es.mecd.demo.miproyecto.alumnoCurso;
import es.mecd.demo.miproyecto.alumnoCursoRepository;
import es.mecd.demo.miproyecto.documento.Documento;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
class SolicitudBecaRepositoryTest {
    @Autowired
    private SolicitudBecaRepository solicitudRepository;

    @Autowired
    private BecaRepository becaRepository;

    @Autowired
    private AlumnoRepository alumnoRepository;

    @Autowired
    private CursoRepository cursoRepository;

    private Beca beca;
    private Alumno alumno;

    @BeforeEach
    void setUp() {
        Curso curso = cursoRepository.save(new Curso("5º Primaria"));
        alumno = new Alumno("Ana", "García", "12345678A",
            LocalDate.of(2010, 5, 12), curso);
        alumno = alumnoRepository.save(alumno);

        beca = becaRepository.save(new Beca("TEST-2026", "Beca General", "Descripción",
            new BigDecimal("1500.00"), 2026));
    }

    @Test
    void guardarSolicitudConDocumentos_debePersistirLaRelacion() {
        SolicitudBeca solicitud = new SolicitudBeca();
        solicitud.setAlumno(alumno);
        solicitud.setBeca(becca);
        solicitud.setFechaSolicitud(LocalDateTime.now());
    }
}
```



```

        Documento doc = new Documento("DNI.pdf", "application/pdf", 12345L,
            new byte[]{1, 2, 3});
        solicitud.addDocumento(doc);

        SolicitudBeca guardada = solicitudRepository.save(solicitud);

        assertNotNull(guardada.getId());
        assertEquals(1, guardada.getDocumentos().size());
    }

    @Test
    void findByIdConRelaciones_debeCargarAlumnoYBeca() {
        SolicitudBeca solicitud = new SolicitudBeca();
        solicitud.setAlumno(alumno);
        solicitud.setBeca(beca);
        solicitud.setFechaSolicitud(LocalDateTime.now());
        SolicitudBeca guardada = solicitudRepository.save(solicitud);

        SolicitudBeca recuperada = solicitudRepository
            .findByIdConRelaciones(guardada.getId())
            .orElseThrow();

        assertEquals("Ana", recuperada.getAlumno().getNombre());
        assertEquals("Beca General", recuperada.getBeca().getNombre());
    }

    @Test
    void findByIdEstado_debeDevolverSolicitudesEnEseEstado() {
        SolicitudBeca enviada = new SolicitudBeca();
        enviada.setAlumno(alumno);
        enviada.setBeca(beca);
        enviada.setFechaSolicitud(LocalDateTime.now());
        enviada.setEstado(EstadoSolicitud.ENVIADA);
        solicitudRepository.save(enviada);

        SolicitudBeca borrador = new SolicitudBeca();
        borrador.setAlumno(alumno);
        borrador.setBeca(beca);
        borrador.setFechaSolicitud(LocalDateTime.now());
        borrador.setEstado(EstadoSolicitud.BORRADOR);
        solicitudRepository.save(borrador);

        List<SolicitudBeca> resultado = solicitudRepository
            .findByIdEstado(EstadoSolicitud.ENVIADA);

        assertEquals(1, resultado.size());
        assertEquals(EstadoSolicitud.ENVIADA, resultado.get(0).getEstado());
    }

    @Test
    void existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween_debeDetectarDuplicado() {
        SolicitudBeca solicitud = new SolicitudBeca();
        solicitud.setAlumno(alumno);
        solicitud.setBeca(beca);
        solicitud.setFechaSolicitud(LocalDateTime.now());
        solicitudRepository.save(solicitud);

        boolean existe = solicitudRepository
            .existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween(
                alumno.getId(), beca.getId(),
                LocalDateTime.now().minusDays(1),
                LocalDateTime.now().plusDays(1));

        assertTrue(existe);
    }

    @Test
    void buscarConFiltros_debeFiltrarPorEstado() {
        for (int i = 0; i < 3; i++) {
            SolicitudBeca s = new SolicitudBeca();
            s.setAlumno(alumno);
            s.setBeca(beca);
            s.setFechaSolicitud(LocalDateTime.now());
            s.setEstado(i == 0 ? EstadoSolicitud.ENVIADA : EstadoSolicitud.BORRADOR);
            solicitudRepository.save(s);
        }

        Page<SolicitudBeca> resultado = solicitudRepository
            .buscarConFiltros(EstadoSolicitud.ENVIADA, null, null,
                PageRequest.of(0, 10));

        assertEquals(1, resultado.getTotalElements());
    }

```

```
}
}
```

Paso 7.3.09 — Verificar las consultas en los logs

Activa el log de SQL en application-dev.properties:

```
spring.jpa.show-sql=true
```

Reinicia y ejecuta los tests. En los logs verás las consultas SQL que Hibernate genera. Por ejemplo, para `findByIdConRelaciones` verás algo como:

```
sql
SELECT s.*, a.*, b.*
FROM solicitudes s
INNER JOIN alumnos a ON s.alumno_id = a.id
INNER JOIN becas b ON s.beca_id = b.id
WHERE s.id = ?
```

Una sola consulta con dos JOIN. Fíjate en que no hay consultas adicionales para el alumno y la beca. Eso es porque hemos usado JOIN FETCH.

Paso 7.3.10 — Errores comunes del ejercicio

Error	Causa	Solución
PropertyReferenceException	El nombre del método derivado no coincide con el modelo	Revisar los nombres de propiedades
Error en <code>deleteBySolicitudId</code>	Falta transacción	Ejecutar desde un servicio <code>@Transactional</code>
La consulta <code>@Query</code> no devuelve datos	JPQL o parámetros incorrectos	Revisar consulta y parámetros
<code>LazyInitializationException</code>	Relación LAZY fuera de transacción	Usar <code>JOIN FETCH</code> o <code>@EntityGraph</code>
Filas duplicadas	<code>JOIN FETCH</code> sobre colección sin <code>DISTINCT</code>	Añadir <code>DISTINCT</code>
<code>MultipleBagFetchException</code>	Se cargan dos colecciones tipo bag simultáneamente	Usar <code>Set</code> o consultas separadas
Página vacía	Parámetros de <code>Pageable</code> incorrectos	Revisar <code>page</code> y <code>size</code>
El método derivado no compila	Error tipográfico	Corregir el nombre
Parámetros desordenados	Firma y nombre del método no coinciden	Revisar el orden
<code>Object[]</code> no se puede castear	Tipo de retorno incorrecto	Usar <code>Object[]</code> y castear elementos

Paso 7.3.11 — Reto resuelto — Consulta con filtro de texto libre

Reto. Añade una consulta que busque texto libre en `observaciones`, ignorando mayúsculas y minúsculas.

Paso 1 — Añadir la consulta al repositorio.

```
@Query("SELECT s FROM SolicitudBeca s " +
        "JOIN FETCH s.alumno " +
        "JOIN FETCH s.beca " +
        "WHERE LOWER(s.observaciones) LIKE LOWER(CONCAT('%', :texto, '%'))")
List<SolicitudBeca> buscarPorTextoEnObservaciones(@Param("texto") String texto);
```

`LOWER(...)` normaliza el texto y `CONCAT('%', :texto, '%')` permite encontrar coincidencias en cualquier posición.

Paso 2 — Probar el método.

```

@Test
void buscarPorTextoEnObservaciones_debeEncontrarCoincidencias() {
    SolicitudBeca s1 = new SolicitudBeca();
    s1.setAlumno(alumno);
    s1.setBeca(beca);
    s1.setFechaSolicitud(LocalDate.now());
    s1.setObservaciones("Falta el DNI del alumno");
    solicitudRepository.save(s1);

    SolicitudBeca s2 = new SolicitudBeca();
    s2.setAlumno(alumno);
    s2.setBeca(beca);
    s2.setFechaSolicitud(LocalDate.now());
    s2.setObservaciones("Solicitud completa");
    solicitudRepository.save(s2);

    List<SolicitudBeca> resultado = solicitudRepository
        .buscarPorTextoEnObservaciones("dni");

    assertEquals(1, resultado.size());
    assertEquals("Falta el DNI del alumno", resultado.get(0).getObservaciones());
}

```

Paso 7.3.12 — Verificar la estructura completa

Al final del ejercicio, el paquete solicitud debe tener:

```

solicitud/
├── SolicitudBeca.java
├── EstadoSolicitud.java
├── SolicitudBecaRepository.java
Y el paquete beca:

text
beca/
├── Beca.java
├── BecaRepository.java

Y el paquete documento:

text
documento/
├── Documento.java
├── DocumentoRepository.java

```

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un BecaRepository con métodos derivados y buscarConFiltros para consultas con paginación.
- Un DocumentoRepository con métodos derivados para consultas por solicitud.
- Un SolicitudBecaRepository con:
 - Métodos derivados simples.
 - Consultas @Query con JOIN FETCH. o Métodos con @EntityGraph. o Consultas de agregación.
- Tests de repositorio que verifican las consultas.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- JpaRepository: interfaz base con CRUD, paginación y ordenación.
- Métodos derivados: Spring Data genera la consulta a partir del nombre.
- @Query con JPQL: para consultas personalizadas.

- JOIN FETCH: carga relaciones en la misma consulta.
- @EntityGraph: alternativa declarativa a JOIN FETCH.
- Pageable y Page: paginación.
- @Param: parámetros nombrados.
- Agregaciones: GROUP BY, COUNT, ORDER BY.
- LOWER y LIKE: búsquedas de texto.
- existsBy...: comprobaciones de existencia.



Conclusión y enlace al siguiente punto

En este punto 7.3 hemos implementado los repositorios Spring Data JPA del proyecto:

- BecaRepository: consultas simples y filtrado con paginación.
- DocumentoRepository: consultas por solicitud.
- SolicitudBecaRepository: mezcla de métodos derivados, @Query, @EntityGraph y agregaciones.

Hemos visto cómo Spring Data genera las consultas a partir de los nombres, cómo escribir consultas con JOIN FETCH para evitar el problema N+1, y cómo verificar todo con tests de @DataJpaTest.

La idea clave: los repositorios son la capa de acceso a datos. Solo contienen operaciones sobre datos, nada de lógica de negocio. La lógica va en el servicio.

En el siguiente punto, 7.4 – Implementación de servicios, implementaremos los servicios del proyecto: BecaService, SolicitudService y DocumentoService. Veremos la lógica de negocio, las validaciones, la transformación a DTOs, las transacciones y las excepciones personalizadas.



Fin del Punto 7.3.

◆ Punto 7.4 — Implementación de servicios



Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Crear los DTOs de entrada y salida del proyecto.
2. Implementar servicios con lógica de negocio y transacciones.
3. Aplicar las reglas de negocio del diseño con excepciones personalizadas.
4. Transformar entre entidades y DTOs dentro del servicio.
5. Manejar relaciones LAZY dentro de la transacción.
6. Validar transiciones de estado.
7. Escribir tests unitarios de los servicios.
8. Diagnosticar y resolver los errores más comunes al implementar servicios.

✂ IMPLEMENTACIÓN



Paso 7.4.01 — Crear los DTOs de Beca

Como `codigo` ya forma parte de `Beca` desde 7.2.12, los DTOs lo incluyen desde su creación.

Crea BecaRequestDTO:

```

package es.mecd.demo.miproyecto.beca;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;
import java.math.BigDecimal;

public class BecaRequestDTO {
    @NotBlank(message = "El código es obligatorio")
    @Size(max = 20, message = "El código no puede superar 20 caracteres")
    private String codigo;

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 100, message = "El nombre no puede superar 100 caracteres")
    private String nombre;

    @Size(max = 500, message = "La descripción no puede superar 500 caracteres")
    private String descripcion;

    @NotNull(message = "El importe máximo es obligatorio")
    @PositiveOrZero(message = "El importe máximo debe ser positivo o cero")
    private BigDecimal importeMaximo;

    @NotNull(message = "El año es obligatorio")
    @Min(value = 2020, message = "El año debe ser igual o posterior a 2020")
    @Max(value = 2100, message = "El año debe ser igual o anterior a 2100")
    private Integer anio;

    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
    public BigDecimal getImporteMaximo() { return importeMaximo; }
    public void setImporteMaximo(BigDecimal importeMaximo) { this.importeMaximo = importeMaximo; }
    public Integer getAnio() { return anio; }
    public void setAnio(Integer anio) { this.anio = anio; }
}

```

Crea BecaResponseDTO:

```

package es.mecd.demo.miproyecto.beca;

import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.math.BigDecimal;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class BecaResponseDTO {
    @JsonProperty("id")
    private String identificador;
    private String codigo;
    private String nombre;
    private String descripcion;
    private BigDecimal importeMaximo;
    private Integer anio;
    private Boolean activa;

    public String getIdentificador() { return identificador; }
    public void setIdentificador(String identificador) { this.identificador = identificador; }
    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
    public BigDecimal getImporteMaximo() { return importeMaximo; }
    public void setImporteMaximo(BigDecimal importeMaximo) { this.importeMaximo = importeMaximo; }
    public Integer getAnio() { return anio; }
    public void setAnio(Integer anio) { this.anio = anio; }
    public Boolean getActiva() { return activa; }
}

```

```

    public void setActiva(Boolean activa) { this.activa = activa; }
}

```

BecaRequestDTO valida código, nombre, importe y año. BecaResponseDTO expone código junto al resto del recurso.

Paso 7.4.02 — Crear el BecaService

Crea BecaService con las reglas de duplicidad por nombre y código, la validación del año actual/siguiente y la protección frente a borrado con solicitudes:

```

package es.mecd.demo.miproyecto.beca;

import es.mecd.demo.miproyecto.common.exception.OperationNoPermitidaException;
import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import es.mecd.demo.miproyecto.common.exception.ValidacionNegocioException;
import es.mecd.demo.miproyecto.solicitud.SolicitudBecaRepository;
import java.time.Year;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class BecaService {
    private final BecaRepository becaRepository;
    private final SolicitudBecaRepository solicitudRepository;

    public BecaService(BecaRepository becaRepository, SolicitudBecaRepository solicitudRepository) {
        this.becaRepository = becaRepository;
        this.solicitudRepository = solicitudRepository;
    }

    @Transactional(readOnly = true)
    public Page<BecaResponseDTO> listar(Boolean activa, Integer anio, Pageable pageable) {
        return becaRepository.buscarConFiltros(activa, anio, pageable).map(this::toDTO);
    }

    @Transactional(readOnly = true)
    public BecaResponseDTO consultar(Long id) {
        return becaRepository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new RecursoNoEncontradoException("Beca", id));
    }

    @Transactional
    public BecaResponseDTO crear(BecaRequestDTO request) {
        validarAnio(request.getAnio());
        validarDuplicados(null, request);
        Beca beca = new Beca(
            request.getCodigo(),
            request.getNombre(),
            request.getDescripcion(),
            request.getImporteMaximo(),
            request.getAnio());
        return toDTO(becaRepository.save(beca));
    }

    @Transactional
    public BecaResponseDTO actualizar(Long id, BecaRequestDTO request) {
        Beca beca = becaRepository.findById(id)
            .orElseThrow(() -> new RecursoNoEncontradoException("Beca", id));
        validarAnio(request.getAnio());
        validarDuplicados(beca, request);
        beca.setCodigo(request.getCodigo());
        beca.setNombre(request.getNombre());
        beca.setDescripcion(request.getDescripcion());
        beca.setImporteMaximo(request.getImporteMaximo());
        beca.setAnio(request.getAnio());
        return toDTO(becaRepository.save(beca));
    }

    @Transactional
    public void eliminar(Long id) {
        Beca beca = becaRepository.findById(id)
            .orElseThrow(() -> new RecursoNoEncontradoException("Beca", id));
        if (solicitudRepository.existsByBecaId(id)) {

```

```

        throw new OperacionNoPermitidaException(
            "No se puede eliminar la beca " + id + ": tiene solicitudes asociadas");
    }
    becaRepository.delete(beca);
}

private void validarAnio(Integer anio) {
    int actual = Year.now().getValue();
    if (anio == null || anio < actual || anio > actual + 1) {
        throw new ValidacionNegocioException(
            "anio",
            "El año de la beca debe ser el actual o el siguiente");
    }
}

private void validarDuplicados(Beca actual, BecaRequestDTO request) {
    if ((actual == null || !actual.getNombre().equals(request.getNombre()))
        && becaRepository.existsByNombre(request.getNombre())) {
        throw new RecursoDuplicadoException("Beca", "nombre", request.getNombre());
    }
    if ((actual == null || !actual.getCodigo().equals(request.getCodigo()))
        && becaRepository.existsByCodigo(request.getCodigo())) {
        throw new RecursoDuplicadoException("Beca", "codigo", request.getCodigo());
    }
}

private BecaResponseDTO toDTO(Beca beca) {
    BecaResponseDTO dto = new BecaResponseDTO();
    dto.setIdentificador(beca.getId() == null ? null : beca.getId().toString());
    dto.setCodigo(beca.getCodigo());
    dto.setNombre(beca.getNombre());
    dto.setDescripcion(beca.getDescripcion());
    dto.setImporteMaximo(beca.getImporteMaximo());
    dto.setAnio(beca.getAnio());
    dto.setActiva(beca.getActiva());
    return dto;
}
}

```

Este servicio materializa las reglas de negocio de `Beca` y propaga `codigo` al crear, actualizar y transformar a DTO.



Paso 7.4.03 — Crear los DTOs de Solicitud

Crea `SolicitudRequestDTO` en el paquete `solicitud`:

```

package es.mecd.demo.miproyecto.solicitud;

import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class SolicitudRequestDTO {

    @NotNull(message = "El alumno es obligatorio")
    private Long alumnoId;

    @NotNull(message = "La beca es obligatoria")
    private Long becaId;

    @Size(max = 1000, message = "Las observaciones no pueden superar 1000 caracteres")
    private String observaciones;

    public Long getAlumnoId() { return alumnoId; }
    public void setAlumnoId(Long alumnoId) { this.alumnoId = alumnoId; }

    public Long getBecaId() { return becaId; }
    public void setBecaId(Long becaId) { this.becaId = becaId; }

    public String getObservaciones() { return observaciones; }
    public void setObservaciones(String observaciones) { this.observaciones = observaciones; }
}

```

Crea `CambioEstadoRequestDTO`:

```

package es.mecd.demo.miproyecto.solicitud;

```

```

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;

import java.math.BigDecimal;

public class CambioEstadoRequestDTO {

    @NotBlank(message = "El estado es obligatorio")
    private String estado;

    @Size(max = 1000, message = "Las observaciones no pueden superar 1000 caracteres")
    private String observaciones;

    @PositiveOrZero(message = "El importe concedido debe ser positivo o cero")
    private BigDecimal importeConcedido;

    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }

    public String getObservaciones() { return observaciones; }
    public void setObservaciones(String observaciones) { this.observaciones = observaciones; }

    public BigDecimal getImporteConcedido() { return importeConcedido; }
    public void setImporteConcedido(BigDecimal importeConcedido) { this.importeConcedido =
importeConcedido; }
}

```

Crea AlumnoResumenDTO:

```

package es.mecd.demo.miproyecto.solicitud;

public class AlumnoResumenDTO {
    private String id;
    private String nombreCompleto;
    private String dni;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getNombreCompleto() { return nombreCompleto; }
    public void setNombreCompleto(String nombreCompleto) { this.nombreCompleto = nombreCompleto; }

    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }
}

```

Crea BecaResumenDTO:

```

package es.mecd.demo.miproyecto.solicitud;

import java.math.BigDecimal;

public class BecaResumenDTO {
    private String id;
    private String codigo;
    private String nombre;
    private BigDecimal importeMaximo;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public BigDecimal getImporteMaximo() { return importeMaximo; }
    public void setImporteMaximo(BigDecimal importeMaximo) { this.importeMaximo = importeMaximo; }
}

```

Crea SolicitudResponseDTO:

```

package es.mecd.demo.miproyecto.solicitud;
import com.fasterxml.jackson.annotation.JsonFormat;

```



```

import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class SolicitudResponseDTO {

    @JsonProperty("id")
    private String identificador;

    private AlumnoResumenDTO alumno;
    private BecaResumenDTO beca;
    private String estado;

    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
    private LocalDateTime fechaSolicitud;

    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
    private LocalDateTime fechaResolucion;

    private BigDecimal importeConcedido;
    private String observaciones;
    private Integer numeroDocumentos;

    public String getIdentificador() { return identificador; }
    public void setIdentificador(String identificador) { this.identificador = identificador; }

    public AlumnoResumenDTO getAlumno() { return alumno; }
    public void setAlumno(AlumnoResumenDTO alumno) { this.alumno = alumno; }

    public BecaResumenDTO getBeca() { return beca; }
    public void setBeca(BecaResumenDTO beca) { this.beca = beca; }

    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }

    public LocalDateTime getFechaSolicitud() { return fechaSolicitud; }
    public void setFechaSolicitud(LocalDateTime fechaSolicitud) { this.fechaSolicitud = fechaSolicitud; }

    public LocalDateTime getFechaResolucion() { return fechaResolucion; }
    public void setFechaResolucion(LocalDateTime fechaResolucion) { this.fechaResolucion = fechaResolucion; }

    public BigDecimal getImporteConcedido() { return importeConcedido; }
    public void setImporteConcedido(BigDecimal importeConcedido) { this.importeConcedido = importeConcedido; }

    public String getObservaciones() { return observaciones; }
    public void setObservaciones(String observaciones) { this.observaciones = observaciones; }

    public Integer getNumeroDocumentos() { return numeroDocumentos; }
    public void setNumeroDocumentos(Integer numeroDocumentos) { this.numeroDocumentos =
numeroDocumentos; }
}

```

Paso 7.4.04 — Crear el SolicitudService (parte 1: consultar y listar)

Crea SolicitudService:

```

package es.mecd.demo.miproyecto.solicitud;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.beca.BecaRepository;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class SolicitudService {

    private final SolicitudBecaRepository solicitudRepository;
    private final AlumnoRepository alumnoRepository;
    private final BecaRepository becaRepository;

    public SolicitudService(SolicitudBecaRepository solicitudRepository,

```

```

        AlumnoRepository alumnoRepository,
        BecaRepository becaRepository) {
    this.solicitudRepository = solicitudRepository;
    this.alumnoRepository = alumnoRepository;
    this.becaRepository = becaRepository;
}

@Transactional(readOnly = true)
public Page<SolicitudResponseDTO> listar(EstadoSolicitud estado, Long alumnoId,
                                         Long becaId, Pageable pageable) {
    return solicitudRepository.buscarConFiltros(estado, alumnoId, becaId, pageable)
        .map(this::toDTO);
}

@Transactional(readOnly = true)
public SolicitudResponseDTO consultar(Long id) {
    return solicitudRepository.findByIdConRelaciones(id)
        .map(this::toDTO)
        .orElseThrow(() -> new RecursoNoEncontradoException("Solicitud", id));
}
}

```

Explicación línea por línea

Líneas 1-11: package e imports.

Línea 13: @Service.

Líneas 15-22: las tres dependencias y el constructor:

- solicitudRepository → acceso a las solicitudes.
- alumnoRepository → para cargar el alumno al crear una solicitud.
- becaRepository → para cargar la beca al crear una solicitud.

Línea 24-29: método listar.

- buscarConFiltros(...) → llama al método del repositorio que ya tiene JOIN FETCH y filtros condicionales.
- .map(this::toDTO) → transforma cada entidad a DTO.

Línea 31-36: método consultar.

- findByIdConRelaciones(id) → usa el método con JOIN FETCH para cargar alumno y beca en una consulta.
- .orElseThrow(...) → si no existe, lanza excepción.

Paso 7.4.05 — Crear el SolicitudService (parte 2: crear solicitud)

Añade el método crear:

```

@Transactional
public SolicitudResponseDTO crear(SolicitudRequestDTO request) {
    Alumno alumno = alumnoRepository.findById(request.getAlumnoId())
        .orElseThrow(() -> new RecursoNoEncontradoException("Alumno", request.getAlumnoId()));
    Beca beca = becaRepository.findById(request.getBecaId())
        .orElseThrow(() -> new RecursoNoEncontradoException("Beca", request.getBecaId()));

    if (!Boolean.TRUE.equals(beca.getActiva())) {
        throw new OperacionNoPermitidaException("No se puede solicitar una beca inactiva");
    }

    LocalDateTime ahora = LocalDateTime.now();
    LocalDateTime inicioAnio = LocalDateTime.of(ahora.getYear(), 1, 1, 0, 0, 0);
    LocalDateTime finAnio = LocalDateTime.of(ahora.getYear() + 1, 1, 1, 0, 0, 0);

    boolean existeDuplicado = solicitudRepository
        .existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween(
            alumno.getId(), beca.getId(), inicioAnio, finAnio);

    if (existeDuplicado) {
        throw new RecursoDuplicadoException(
            "Solicitud", "alumnoId+becaId+año",
            alumno.getId() + "+" + beca.getId() + "+" + ahora.getYear());
    }
}

```

```

    }

    SolicitudBeca solicitud = new SolicitudBeca();
    solicitud.setAlumno(alumno);
    solicitud.setBeca(beca);
    solicitud.setEstado(EstadoSolicitud.BORRADOR);
    solicitud.setFechaSolicitud(ahora);
    solicitud.setObservaciones(request.getObservaciones());

    return toDTO(solicitudRepository.save(solicitud));
}

```

Añade los imports:

```

import es.mecd.demo.miproyecto.common.exception.OperationNoPermitidaException;
import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import java.time.LocalDateTime;

```

Explicación línea por línea

Línea 1: `@Transactional` → transacción completa.

Línea 2: `public SolicitudResponseDTO crear(SolicitudRequestDTO request) {` → el método.

Línea 3-4: `alumnoRepository.findById(request.getAlumnoId()).orElseThrow(...)` → busca el alumno. Si no existe, lanza `RecursoNoEncontradoException("Alumno", ...)`.

Línea 5-6: `becaRepository.findById(request.getBecaId()).orElseThrow(...)` → busca la beca.

A continuación se comprueba `beca.getActiva()`: una beca inactiva no admite nuevas solicitudes y produce `OperationNoPermitidaException`.

Línea 8: `LocalDateTime ahora = LocalDateTime.now();` → captura el momento actual una sola vez. Se usará para la fecha de solicitud y para calcular el rango del año.

Línea 9: `LocalDateTime inicioAnio = LocalDateTime.of(ahora.getYear(), 1, 1, 0, 0, 0);` → primer instante del año actual.

Línea 10: `LocalDateTime finAnio = LocalDateTime.of(ahora.getYear() + 1, 1, 1, 0, 0, 0);` → primer instante del año siguiente. Fíjate en que el rango es semiabierto `[inicioAnio, finAnio)`. Es decir, incluye todo el año actual y excluye el 1 de enero del año siguiente. Así no se pierde ningún instante (la versión anterior con 23:59 dejaba fuera el último minuto del 31 de diciembre).

Línea 12-15: `existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween(...)` → comprueba si ya existe una solicitud del mismo alumno, para la misma beca, en el año actual.

Línea 17-21: si existe duplicado, lanza `RecursoDuplicadoException`. El tercer parámetro del constructor es el valor duplicado (una combinación legible de los tres datos).

Línea 23-28: crea la solicitud con los datos. Fíjate en que no se asigna el `importeConcedido` (todavía no se ha resuelto) ni la fecha de resolución. El estado por defecto es `BORRADOR`.

Línea 30: guarda y transforma.

Paso 7.4.06 — Crear el SolicitudService (parte 3: cambiar estado con validación de transiciones)

Añade el método `cambiarEstado` y el método privado `validarTransicion`:

```

@Transactional
public SolicitudResponseDTO cambiarEstado(Long id, CambioEstadoRequestDTO request) {
    SolicitudBeca solicitud = solicitudRepository.findByIdConRelaciones(id)
        .orElseThrow(() -> new RecursoNoEncontradoException("Solicitud", id));

    if (solicitud.getEstado() != EstadoSolicitud.ENVIADA
        && solicitud.getEstado() != EstadoSolicitud.EN_REVISION) {
        throw new OperationNoPermitidaException(
            "No se puede cambiar el estado de una solicitud en estado "

```

```

        + solicitud.getEstado());
    }

    EstadoSolicitud nuevoEstado;
    try {
        nuevoEstado = EstadoSolicitud.valueOf(request.getEstado());
    } catch (IllegalArgumentException e) {
        throw new ValidacionNegocioException("estado",
            "El estado " + request.getEstado() + " no es válido");
    }

    validarTransicion(solicitud.getEstado(), nuevoEstado);
    if (nuevoEstado == EstadoSolicitud.APROBADA) {
        if (request.getImporteConcedido() == null) {
            throw new ValidacionNegocioException("importeConcedido",
                "El importe concedido es obligatorio al aprobar una solicitud");
        }
        if (request.getImporteConcedido().compareTo(solicitud.getBeca().getImporteMaximo()) > 0) {
            throw new ValidacionNegocioException("importeConcedido",
                "El importe concedido no puede superar el importe máximo de la beca");
        }
        solicitud.setImporteConcedido(request.getImporteConcedido());
    }

    solicitud.setEstado(nuevoEstado);

    if (request.getObservaciones() != null) {
        solicitud.setObservaciones(request.getObservaciones());
    }

    if (nuevoEstado == EstadoSolicitud.APROBADA
        || nuevoEstado == EstadoSolicitud.DENEGADA) {
        solicitud.setFechaResolucion(LocalDate.now());
    }

    return toDTO(solicitudRepository.save(solicitud));
}

private void validarTransicion(EstadoSolicitud actual, EstadoSolicitud nuevo) {
    Map<EstadoSolicitud, Set<EstadoSolicitud>> transiciones = Map.of(
        EstadoSolicitud.ENVIADA, Set.of(
            EstadoSolicitud.EN_REVISION,
            EstadoSolicitud.DENEGADA),
        EstadoSolicitud.EN_REVISION, Set.of(
            EstadoSolicitud.SOLICITUD_DOCUMENTACION,
            EstadoSolicitud.APROBADA,
            EstadoSolicitud.DENEGADA)
    );
    Set<EstadoSolicitud> permitidos = transiciones.getOrDefault(actual, Set.of());
    if (!permitidos.contains(nuevo)) {
        throw new OperacionNoPermitidaException(
            "No se puede pasar de " + actual + " a " + nuevo);
    }
}

```

Añade los imports:

```

import es.mecd.demo.miproyecto.common.exception.OperacionNoPermitidaException;
import es.mecd.demo.miproyecto.common.exception.ValidacionNegocioException;
import java.util.Map;
import java.util.Set;

```

Explicación línea por línea

Línea 1: @Transactional → transacción completa.

Línea 3-4: busca la solicitud con relaciones. Si no existe, 404.

Línea 6-10: comprueba que la solicitud está en un estado que permite cambios (ENVIADA o EN_REVISION). Si no, OperacionNoPermitidaException → 409.

Línea 12-17: convierte el string del DTO a EstadoSolicitud con valueOf. Si el string no coincide con ningún valor del enum, lanza IllegalArgumentException, que capturamos y convertimos a ValidacionNegocioException → 400.

Línea 19: `validarTransicion(solicitud.getEstado(), nuevoEstado);` → valida que la transición de estado sea permitida. Es la nueva regla que añadimos.

Línea 21-30: si el nuevo estado es APROBADA:

- `if (request.getImporteConcedido() == null)` → el importe es obligatorio.
- `if (request.getImporteConcedido().compareTo(...) > 0)` → el importe no puede superar el máximo de la beca.
- `solicitud.setImporteConcedido(...)` → asigna el importe.

Línea 32: `solicitud.setEstado(nuevoEstado);` → cambia el estado.

Línea 34-36: si el DTO trae observaciones, actualiza las de la solicitud.

Línea 38-42: si el nuevo estado es un estado final (APROBADA o DENEGADA), asigna la fecha de resolución.

Línea 44: guarda y transforma. Líneas 46-57: método `validarTransicion`.

- `Map<EstadoSolicitud, Set> transiciones` → mapa que define, para cada estado actual, el conjunto de estados a los que se puede pasar.
- `Set.of(...)` → el conjunto de estados permitidos desde cada uno.
- `transiciones.getOrDefault(actual, Set.of())` → obtiene los permitidos. Si el estado actual no está en el mapa, devuelve un conjunto vacío (no se permite ninguna transición).
- `if (!permitidos.contains(nuevo))` → si el nuevo estado no está en el conjunto, lanza `OperacionNoPermitidaException` → 409.

Las transiciones permitidas son:

- ENVIADA → EN_REVISION o DENEGADA.
- EN_REVISION → SOLICITUD_DOCUMENTACION, APROBADA o DENEGADA.

Paso 7.4.07 — Crear el `SolicitudService` (parte 4: cancelar)

Añade el método `cancelar`:

```
@Transactional
public void cancelar(Long id) {
    SolicitudBeca solicitud = solicitudRepository.findByIdConRelaciones(id)
        .orElseThrow(() -> new RecursoNoEncontradoException("Solicitud", id));

    if (solicitud.getEstado() == EstadoSolicitud.APROBADA
        || solicitud.getEstado() == EstadoSolicitud.DENEGADA
        || solicitud.getEstado() == EstadoSolicitud.CANCELADA) {
        throw new OperacionNoPermitidaException(
            "No se puede cancelar una solicitud en estado " + solicitud.getEstado());
    }

    solicitud.setEstado(EstadoSolicitud.CANCELADA);
    solicitud.setFechaResolucion(LocalDate.now());
    solicitudRepository.save(solicitud);
}
```

Explicación línea por línea

Línea 1: `@Transactional`.

Línea 3-4: busca la solicitud. Si no existe, 404.

Línea 6-11: si el estado es uno de los estados finales (APROBADA, DENEGADA, CANCELADA), lanza `OperacionNoPermitidaException` → 409. No se puede cancelar una solicitud ya resuelta o ya cancelada.

Línea 13-15: cambia el estado a CANCELADA y asigna la fecha de resolución.

Paso 7.4.08 — Añadir el método toDTO al SolicitudService

Añade el método de transformación:

```
private SolicitudResponseDTO toDTO(SolicitudBeca solicitud) {
    SolicitudResponseDTO dto = new SolicitudResponseDTO();
    dto.setIdentificador(solicitud.getId() != null ? solicitud.getId().toString() : null);
    dto.setEstado(solicitud.getEstado().name());
    dto.setFechaSolicitud(solicitud.getFechaSolicitud());
    dto.setFechaResolucion(solicitud.getFechaResolucion());
    dto.setImporteConcedido(solicitud.getImporteConcedido());
    dto.setObservaciones(solicitud.getObservaciones());

    if (solicitud.getAlumno() != null) {
        AlumnoResumenDTO alumnoResumen = new AlumnoResumenDTO();
        alumnoResumen.setId(solicitud.getAlumno().getId().toString());
        alumnoResumen.setNombreCompleto(solicitud.getAlumno().getNombre() + " "
            + solicitud.getAlumno().getApellidos());
        alumnoResumen.setDni(solicitud.getAlumno().getDni());
        dto.setAlumno(alumnoResumen);
    }

    if (solicitud.getBeca() != null) {
        BecaResumenDTO becaResumen = new BecaResumenDTO();
        becaResumen.setId(solicitud.getBeca().getId().toString());
        becaResumen.setCodigo(solicitud.getBeca().getCodigo());
        becaResumen.setNombre(solicitud.getBeca().getNombre());
        becaResumen.setImporteMaximo(solicitud.getBeca().getImporteMaximo());
        dto.setBeca(beccaResumen);
    }

    dto.setNumeroDocumentos(solicitud.getDocumentos().size());

    return dto;
}
```

Explicación línea por línea

Línea 1: `private SolicitudResponseDTO toDTO(SolicitudBeca solicitud) {` → método privado.

Línea 2-8: construye el DTO con los campos simples.

Línea 10-16: si el alumno está cargado, construye el `AlumnoResumenDTO`. Fíjate en que se accede a `solicitud.getAlumno()`, que es una relación LAZY. Como estamos dentro de la transacción (el método está anotado con `@Transactional` o se llama desde uno que lo está), Hibernate puede cargarlo.

Línea 18-25: lo mismo para la beca, incluyendo `codigo`, que quedó incorporado al dominio en 7.2.12.

Línea 26: `solicitud.getDocumentos().size()` → accede a la colección de documentos para contar cuántos hay. Al ser LAZY, se carga dentro de la transacción.

Paso 7.4.09 — Crear el DocumentoResponseDTO y el DocumentoService

Crea `DocumentoResponseDTO`:

```
package es.mecd.demo.miproyecto.documento;

import com.fasterxml.jackson.annotation.JsonFormat;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.LocalDateTime;

public class DocumentoResponseDTO {
    @JsonProperty("id")
    private String identificador;
    private String nombre;
    private String tipo;
    private Long tamano;
    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
    private LocalDateTime fechaSubida;

    public String getIdentificador() { return identificador; }
    public void setIdentificador(String identificador) { this.identificador = identificador; }
    public String getNombre() { return nombre; }
```

```

    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }
    public Long getTamano() { return tamano; }
    public void setTamano(Long tamano) { this.tamano = tamano; }
    public LocalDateTime getFechaSubida() { return fechaSubida; }
    public void setFechaSubida(LocalDateTime fechaSubida) { this.fechaSubida = fechaSubida; }
}

```

Crea DocumentoService:

```

package es.mecd.demo.miproyecto.documento;

import es.mecd.demo.miproyecto.common.exception.OperationNoPermitidaException;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import es.mecd.demo.miproyecto.common.exception.ValidacionNegocioException;
import es.mecd.demo.miproyecto.solicitud.EstadoSolicitud;
import es.mecd.demo.miproyecto.solicitud.SolicitudBeca;
import es.mecd.demo.miproyecto.solicitud.SolicitudBecaRepository;
import java.io.IOException;
import java.util.List;
import java.util.Set;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.multipart.MultipartFile;

@Service
public class DocumentoService {
    private static final long TAMANO_MAXIMO = 10L * 1024 * 1024;
    private static final Set<String> TIPOS_PERMITIDOS = Set.of(
        "application/pdf", "image/jpeg", "image/png");

    private final DocumentoRepository documentoRepository;
    private final SolicitudBecaRepository solicitudRepository;

    public DocumentoService(
        DocumentoRepository documentoRepository,
        SolicitudBecaRepository solicitudRepository) {
        this.documentoRepository = documentoRepository;
        this.solicitudRepository = solicitudRepository;
    }

    @Transactional(readOnly = true)
    public List<DocumentoResponseDTO> listarPorSolicitud(Long solicitudId) {
        if (!solicitudRepository.existsById(solicitudId)) {
            throw new RecursoNoEncontradoException("Solicitud", solicitudId);
        }
        return documentoRepository.findBySolicitudId(solicitudId).stream()
            .map(this::toDTO)
            .toList();
    }

    @Transactional
    public DocumentoResponseDTO anadir(Long solicitudId, MultipartFile fichero) {
        SolicitudBeca solicitud = solicitudRepository.findById(solicitudId)
            .orElseThrow(() -> new RecursoNoEncontradoException("Solicitud", solicitudId));
        validarEditable(solicitud);
        if (fichero == null || fichero.isEmpty()) {
            throw new ValidacionNegocioException("fichero", "El fichero es obligatorio");
        }
        if (fichero.getSize() > TAMANO_MAXIMO) {
            throw new ValidacionNegocioException("fichero", "El fichero supera el tamaño máximo de 10
MB");
        }
        if (!TIPOS_PERMITIDOS.contains(fichero.getContentType())) {
            throw new ValidacionNegocioException(
                "fichero",
                "El tipo de fichero no está permitido. Tipos válidos: PDF, JPG, PNG");
        }
        try {
            Documento documento = new Documento(
                fichero.getOriginalFilename(),
                fichero.getContentType(),
                fichero.getSize(),
                fichero.getBytes());
            solicitud.addDocumento(documento);
            Documento guardado = documentoRepository.saveAndFlush(documento);
            return toDTO(guardado);
        } catch (IOException ex) {
            throw new IllegalStateException("Error al leer el fichero", ex);
        }
    }
}

```



```

    }
}

@Transactional
public void eliminar(Long solicitudId, Long documentoId) {
    SolicitudBeca solicitud = solicitudRepository.findById(solicitudId)
        .orElseThrow(() -> new RecursoNoEncontradoException("Solicitud", solicitudId));
    validarEditable(solicitud);
    Documento documento = documentoRepository.findById(documentoId)
        .orElseThrow(() -> new RecursoNoEncontradoException("Documento", documentoId));
    if (documento.getSolicitud() == null
        || !solicitudId.equals(documento.getSolicitud().getId())) {
        throw new RecursoNoEncontradoException("Documento", documentoId);
    }
    documentoRepository.delete(documento);
}

private void validarEditable(SolicitudBeca solicitud) {
    if (solicitud.getEstado() != EstadoSolicitud.BORRADOR
        && solicitud.getEstado() != EstadoSolicitud.ENVIADA) {
        throw new OperacionNoPermitidaException(
            "No se pueden modificar documentos de una solicitud en estado "
            + solicitud.getEstado());
    }
}

private DocumentoResponseDTO toDTO(Documento documento) {
    DocumentoResponseDTO dto = new DocumentoResponseDTO();
    dto.setIdentificador(documento.getId() == null ? null : documento.getId().toString());
    dto.setNombre(documento.getNombre());
    dto.setTipo(documento.getTipo());
    dto.setTamano(documento.getTamano());
    dto.setFechaSubida(documento.getFechaSubida());
    return dto;
}
}

```

El servicio aplica las reglas de estado, tamaño máximo de 10 MB, tipos PDF/JPG/PNG y pertenencia del documento a la solicitud. La simplificación de ownership se mantiene: el modelo no inventa una relación `Usuario` → `Alumno`.

Paso 7.4.10 — Escribir tests del SolicitudService

Crea `SolicitudServiceTest`:

```

package es.mecd.demo.miproyecto.solicitud;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.beca.BecaRepository;
import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Optional;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class SolicitudServiceTest {

    @Mock
    private SolicitudBecaRepository solicitudRepository;
    @Mock
    private AlumnoRepository alumnoRepository;
    @Mock
    private BecaRepository becaRepository;

    private SolicitudService service;
}

```



```

@BeforeEach
void setUp() {
    service = new SolicitudService(solicitudRepository, alumnoRepository, becaRepository);
}

@Test
void consultar_debeLanzarRecursoNoEncontrado_cuandoNoExiste() {
    when(solicitudRepository.findByIdConRelaciones(999L))
        .thenReturn(Optional.empty());

    assertThrows(RecursoNoEncontradoException.class,
        () -> service.consultar(999L));
}

@Test
void crear_debeLanzarRecursoNoEncontrado_cuandoAlumnoNoExiste() {
    SolicitudRequestDTO request = new SolicitudRequestDTO();
    request.setAlumnoId(1L);
    request.setBecaId(1L);

    when(alumnoRepository.findById(1L)).thenReturn(Optional.empty());

    assertThrows(RecursoNoEncontradoException.class,
        () -> service.crear(request));
}

@Test
void crear_debeLanzarRecursoDuplicado_cuandoYaExisteSolicitud() {
    Alumno alumno = mock(Alumno.class);
    when(alumno.getId()).thenReturn(1L);
    Beca beca = new Beca("TEST-2026", "Beca General", "Desc",
        new BigDecimal("1500.00"), 2026);
    beca.setId(1L);

    SolicitudRequestDTO request = new SolicitudRequestDTO();
    request.setAlumnoId(1L);
    request.setBecaId(1L);

    when(alumnoRepository.findById(1L)).thenReturn(Optional.of(alumno));
    when(beccaRepository.findById(1L)).thenReturn(Optional.of(becca));
    when(solicitudRepository.existsByAlumnoIdAndBecaIdAndFechaSolicitudBetween(
        eq(1L), eq(1L), any(LocalDateTime.class), any(LocalDateTime.class)))
        .thenReturn(true);

    assertThrows(RecursoDuplicadoException.class,
        () -> service.crear(request));
}
}

```

`@ExtendWith(MockitoExtension.class)` activa Mockito para JUnit 5; `@Mock` crea las dependencias simuladas y `when(...)` configura su comportamiento.

Paso 7.4.11 — Errores comunes del ejercicio

Error	Causa	Solución
<code>LazyInitializationException</code>	Acceso a relación LAZY fuera de transacción	Transformar dentro de <code>@Transactional</code>
<code>NullPointerException</code> en <code>toDTO</code>	Una relación puede ser nula	Comprobar antes de acceder
<code>RecursoDuplicadoException</code> mal lanzada	Constructor incorrecto	Revisar la firma
Comparación incorrecta de <code>BigDecimal</code>	Se usa <code>></code>	Usar <code>compareTo</code>
<code>IllegalArgumentException</code> no capturada	<code>valueOf</code> recibe un valor inválido	Capturar y traducir el error
Fechas distintas en una misma operación	Se llama varias veces a <code>now()</code>	Capturar una única fecha
Cascade no guarda documentos	Falta <code>cascade = ALL</code>	Añadirlo
Los documentos no se guardan	Se persisten fuera de la solicitud	Guardar a través de la solicitud

Error	Causa	Solución
existsByBecaId no existe	Falta método de repositorio	Añadirlo
Transición no validada	Falta validarTransicion	Aplicar la validación

Paso 7.4.12 — Reto resuelto — Método para obtener la solicitud más reciente de un alumno

Reto. Añade un método de servicio que devuelva la solicitud más reciente de un alumno, o `Optional.empty()` si no tiene ninguna.

Paso 1 — Añadir el método al repositorio.

```
Optional<SolicitudBeca> findFirstByAlumnoIdOrderByFechaSolicitudDesc(Long alumnoId);
```

Spring Data ordena por `fechaSolicitud` en sentido descendente y devuelve el primer resultado.

Paso 2 — Añadir el método al servicio.

```
@Transactional(readOnly = true)
public Optional<SolicitudResponseDTO> buscarUltimaDeAlumno(Long alumnoId) {
    if (!alumnoRepository.existsById(alumnoId)) {
        throw new RecursoNoEncontradoException("Alumno", alumnoId);
    }

    return solicitudRepository.findFirstByAlumnoIdOrderByFechaSolicitudDesc(alumnoId)
        .map(this::toDTO);
}
```

Paso 3 — Probar el caso sin solicitudes.

```
@Test
void buscarUltimaDeAlumno_debeDevolverVacio_cuandoAlumnoSinSolicitudes() {
    when(alumnoRepository.existsById(1L)).thenReturn(true);
    when(solicitudRepository.findFirstByAlumnoIdOrderByFechaSolicitudDesc(1L))
        .thenReturn(Optional.empty());

    Optional<SolicitudResponseDTO> resultado = service.buscarUltimaDeAlumno(1L);

    assertTrue(resultado.isEmpty());
}
```

Resultado esperado global

Al final del ejercicio, deberías tener:

- Los DTOs de Beca, Solicitud y Documento (entrada y salida).
- Un `BecaService` con CRUD, validaciones de duplicado y comprobación de solicitudes asociadas.
- Un `SolicitudService` con listar, consultar, crear, cambiar estado y cancelar, con todas las reglas de negocio.
- Un `DocumentoService` con listar, añadir y eliminar, con validaciones de estado, tamaño y tipo.
- Tests unitarios de los servicios con Mockito.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- DTOs de entrada y salida: separados, con validaciones y resúmenes.
- `@Service`: servicios con lógica de negocio.

- @Transactional y @Transactional(readOnly = true): transacciones en el servicio.
- Transformación toDTO: privada, dentro del servicio.
- Reglas de negocio: con RecursoDuplicadoException, OperacionNoPermitidaException, ValidacionNegocioException.
- Relaciones LAZY: cargadas dentro de la transacción.
- BigDecimal.compareTo: para comparaciones de precisión.
- MultipartFile: recepción de ficheros.
- Cascade: guardar documentos a través de la solicitud.
- validarTransicion: validación de transiciones de estado.
- Rango semiabierto: [inicioAnio, finAnio) para no perder instantes.
- Tests con Mockito: @ExtendWith, @Mock, when, assertThrows.



Conclusión y enlace al siguiente punto

En este punto has construido la capa de servicios y has concentrado en ella las reglas de negocio, las transacciones, las transformaciones entre entidades y DTOs y las validaciones de estado. El siguiente punto expone estas operaciones mediante controladores REST.

◆ Punto 7.5 — Implementación de controladores



Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Implementar controladores REST que deleguen en los servicios.
2. Aplicar códigos de estado correctos en cada operación.
3. Construir respuestas con ResponseEntity y cabecera Location.
4. Combinar @Valid, @PathVariable, @RequestParam y @RequestBody.
5. Documentar los endpoints con OpenAPI.
6. Implementar el UsuarioController para la gestión de usuarios (solo ADMIN).
7. Verificar los controladores con MockMvc.
8. Diagnosticar y resolver los errores más comunes al implementar controladores.

✂ IMPLEMENTACIÓN



Paso 7.5.01 — Crear el BecaController

Crea la clase BecaController en el paquete beca:

```
package es.mecd.demo.miproyecto.beca;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import jakarta.validation.Valid;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
import java.net.URI;
```

```

@RestController
@RequestMapping("/api/v1/becas")
public class BecaController {

    private final BecaService becaService;

    public BecaController(BecaService becaService) {
        this.becaService = becaService;
    }

    @Operation(summary = "Listar becas")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Lista de becas devuelta")
    })
    @GetMapping
    public Page<BecaResponseDTO> listar(
        @RequestParam(required = false) Boolean activa,
        @RequestParam(required = false) Integer anio,
        @PageableDefault(size = 20, sort = "id") Pageable pageable) {
        return becaService.listar(activa, anio, pageable);
    }

    @Operation(summary = "Consultar beca por ID")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Beca encontrada"),
        @ApiResponse(responseCode = "404", description = "Beca no encontrada")
    })
    @GetMapping("/{id}")
    public ResponseEntity<BecaResponseDTO> consultar(@PathVariable Long id) {
        return ResponseEntity.ok(beccaService.consultar(id));
    }

    @Operation(summary = "Crear beca")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "201", description = "Beca creada"),
        @ApiResponse(responseCode = "400", description = "Datos inválidos"),
        @ApiResponse(responseCode = "409", description = "Ya existe una beca con ese nombre")
    })
    @PostMapping
    public ResponseEntity<BecaResponseDTO> crear(
        @Valid @RequestBody BecaRequestDTO request) {
        BecaResponseDTO creada = becaService.crear(request);
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(creada.getIdentificador())
            .toUri();
        return ResponseEntity.created(location).body(creada);
    }

    @Operation(summary = "Actualizar beca")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Beca actualizada"),
        @ApiResponse(responseCode = "404", description = "Beca no encontrada"),
        @ApiResponse(responseCode = "409", description = "Ya existe otra beca con ese nombre")
    })
    @PutMapping("/{id}")
    public ResponseEntity<BecaResponseDTO> actualizar(
        @PathVariable Long id,
        @Valid @RequestBody BecaRequestDTO request) {
        return ResponseEntity.ok(beccaService.actualizar(id, request));
    }

    @Operation(summary = "Eliminar beca")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "204", description = "Beca eliminada"),
        @ApiResponse(responseCode = "404", description = "Beca no encontrada"),
        @ApiResponse(responseCode = "409", description = "La beca tiene solicitudes asociadas")
    })
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        becaService.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}

```

Explicación línea por línea

Líneas 1-17: package e imports. Fíjate en los de OpenAPI (Operation, ApiResponse, ApiResponses) y en ServletUriComponentsBuilder para construir la URI.

Línea 19: `@RestController` → marca la clase como controlador REST.

Línea 20: `@RequestMapping("/api/v1/becas")` → prefijo común para todos los endpoints de la clase.

Línea 21: `public class BecaController {` → declara la clase.

Líneas 23-27: la dependencia y el constructor. Inyección por constructor, como en todo el curso.

Líneas 29-33: anotaciones OpenAPI. `@Operation` describe el endpoint; `@ApiResponse`s agrupa las respuestas posibles.

Línea 34: `@GetMapping` → mapea a GET `/api/v1/becas`.

Línea 35-38: parámetros del método:

- `@RequestParam(required = false)` Boolean activa → filtro opcional por estado activa/inactiva.
- `@RequestParam(required = false)` Integer anio → filtro opcional por año.
- `@PageableDefault(size = 20, sort = "id")` Pageable pageable → paginación con valores por defecto.
 - `size = 20` → 20 elementos por página.
 - `sort = "id"` → ordenado por ID ascendente.
 - `@PageableDefault` es de Spring Data. Si el cliente no envía parámetros, se usan estos valores.

Línea 39: `return becaService.listar(activa, anio, pageable);` → delega en el servicio. El servicio devuelve Page. Fíjate en que no se envuelve en `ResponseEntity`: Spring MVC lo serializa directamente y devuelve 200.

Línea 41-46: anotaciones OpenAPI para el endpoint de consulta.

Línea 47: `@GetMapping("/{id}")` → mapea a GET `/api/v1/becas/{id}`.

Línea 48: `public ResponseEntity consultar(@PathVariable Long id) {` → recibe el ID de la URL.

Línea 49: `return ResponseEntity.ok(beccaService.consultar(id));` → delega en el servicio. Si el servicio lanza `RecursoNoEncontradoException`, el manejador global la captura y devuelve 404. Si no, devuelve 200 con el DTO.

Línea 51-56: anotaciones OpenAPI.

Línea 57: `@PostMapping` → mapea a POST `/api/v1/becas`.

Línea 58-60: parámetros del método:

- `@Valid @RequestBody BecaRequestDTO request` → deserializa el cuerpo y activa la validación de Bean Validation. Si hay errores, Spring MVC lanza `MethodArgumentNotValidException` antes de ejecutar el método.

Línea 61: `BecaResponseDTO creada = becaService.crear(request);` → delega en el servicio.

Línea 62-65: construye la URI del recurso creado con `ServletUriComponentsBuilder`.

- `fromCurrentRequest()` → toma la URL actual (`/api/v1/becas`).
- `.path("/{id}")` → añade el path del recurso.
- `.buildAndExpand(creada.getIdentificador())` → reemplaza `{id}` con el ID.
- `.toUri()` → construye la URI final.

Línea 66: `return ResponseEntity.created(location).body(creada);` → devuelve 201 con cabecera Location y el DTO en el cuerpo.

Línea 68-73: anotaciones OpenAPI.

Línea 74: `@PutMapping("/{id}")` → mapea a PUT `/api/v1/becas/{id}`.

Línea 75-77: parámetros: id de la URL, request del cuerpo (con `@Valid`).

Línea 78: `return ResponseEntity.ok(becaService.actualizar(id, request));` → delega y devuelve 200 con el DTO actualizado.

Línea 80-85: anotaciones OpenAPI.

Línea 86: `@DeleteMapping("/{id}")` → mapea a DELETE `/api/v1/becas/{id}`.

Línea 87: `public ResponseEntity eliminar(@PathVariable Long id) {` → método.

Línea 88: `becaService.eliminar(id);` → delega. Si el servicio lanza excepción (404 o 409), el manejador global la captura.

Línea 89: `return ResponseEntity.noContent().build();` → devuelve 204 sin cuerpo.

Paso 7.5.02 — Crear el SolicitudesController

Crea la clase `SolicitudesController` en el paquete `solicitud`:

```
package es.mecd.demo.miproyecto.solicitud;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import jakarta.validation.Valid;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;

@RestController
@RequestMapping("/api/v1/solicitudes")
public class SolicitudesController {

    private final SolicitudesService solicitudesService;

    public SolicitudesController(SolicitudesService solicitudesService) {
        this.solicitudesService = solicitudesService;
    }

    @Operation(summary = "Listar solicitudes")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Lista de solicitudes devuelta")
    })
    @GetMapping
    public Page<SolicitudResponseDTO> listar(
        @RequestParam(required = false) EstadoSolicitud estado,
        @RequestParam(required = false) Long alumnoId,
        @RequestParam(required = false) Long becaId,
        @PageableDefault(size = 20, sort = "fechaSolicitud",
            direction = Sort.Direction.DESC) Pageable pageable) {
        return solicitudesService.listar(estado, alumnoId, becaId, pageable);
    }

    @Operation(summary = "Consultar solicitud por ID")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Solicitud encontrada"),
        @ApiResponse(responseCode = "404", description = "Solicitud no encontrada")
    })
    @GetMapping("/{id}")
    public ResponseEntity<SolicitudResponseDTO> consultar(@PathVariable Long id) {
        return ResponseEntity.ok(solicitudesService.consultar(id));
    }

    @Operation(summary = "Crear solicitud")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "201", description = "Solicitud creada"),
```

```

        @ApiResponse(responseCode = "400", description = "Datos inválidos"),
        @ApiResponse(responseCode = "404", description = "Alumno o beca no encontrados"),
        @ApiResponse(responseCode = "409", description = "Ya existe una solicitud para ese alumno y beca
este año")
    })
    @PostMapping
    public ResponseEntity<SolicitudResponseDTO> crear(
        @Valid @RequestBody SolicitudRequestDTO request) {
        SolicitudResponseDTO creada = solicitudService.crear(request);
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(creada.getIdentificador())
            .toUri();
        return ResponseEntity.created(location).body(creada);
    }

    @Operation(summary = "Cambiar estado de una solicitud")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Estado cambiado"),
        @ApiResponse(responseCode = "400", description = "Estado o importe inválidos"),
        @ApiResponse(responseCode = "404", description = "Solicitud no encontrada"),
        @ApiResponse(responseCode = "409", description = "La solicitud no permite cambio de estado")
    })
    @PatchMapping("/{id}/estado")
    public ResponseEntity<SolicitudResponseDTO> cambiarEstado(
        @PathVariable Long id,
        @Valid @RequestBody CambioEstadoRequestDTO request) {
        return ResponseEntity.ok(solicitudService.cambiarEstado(id, request));
    }

    @Operation(summary = "Cancelar solicitud")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "204", description = "Solicitud cancelada"),
        @ApiResponse(responseCode = "404", description = "Solicitud no encontrada"),
        @ApiResponse(responseCode = "409", description = "La solicitud no se puede cancelar")
    })
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> cancelar(@PathVariable Long id) {
        solicitudService.cancelar(id);
        return ResponseEntity.noContent().build();
    }
}

```

Explicación línea por línea

Líneas 1-17: package e imports. Igual que en BecaController. Añadimos el import de Sort.

Línea 19-20: @RestController y @RequestMapping("/api/v1/solicitudes").

Líneas 22-26: dependencia y constructor.

Líneas 28-33: anotaciones OpenAPI.

Línea 34: @GetMapping → GET /api/v1/solicitudes.

Línea 35-39: parámetros:

- @RequestParam(required = false) EstadoSolicitud estado → filtro por estado. Fíjate en que el tipo es el enum, no un String. Spring MVC convierte el string de la query al enum automáticamente. Si el string no coincide con ningún valor, devuelve 400.
- @RequestParam(required = false) Long alumnoId → filtro por alumno.
- @RequestParam(required = false) Long becaId → filtro por beca.
- @PageableDefault(size = 20, sort = "fechaSolicitud", direction = Sort.Direction.DESC) Pageable pageable → paginación por defecto, ordenado por fecha de solicitud descendente. Así las solicitudes más recientes aparecen primero, que es lo habitual.

Línea 40: return solicitudService.listar(estados, alumnoId, becaId, pageable); → delega.

Línea 42-47: anotaciones OpenAPI.

Línea 48: @GetMapping("/{id}") → GET /api/v1/solicitudes/{id}.

Línea 49: `public ResponseEntity consultar(@PathVariable Long id) {` → método. Línea 50: `return ResponseEntity.ok(solicitudService.consultar(id));` → delega.

Línea 52-58: anotaciones OpenAPI.

Línea 59: `@PostMapping` → POST `/api/v1/solicitudes`.

Línea 60-62: parámetros.

Línea 63: `SolicitudResponseDTO creada = solicitudService.crear(request);` → delega.

Línea 64-67: construye la URI con `ServletUriComponentsBuilder`.

Línea 68: `return ResponseEntity.created(location).body(creada);` → 201 con Location.

Línea 70-77: anotaciones OpenAPI.

Línea 78: `@PatchMapping("/{id}/estado")` → PATCH `/api/v1/solicitudes/{id}/estado`.

Línea 79-81: parámetros.

Línea 82: `return ResponseEntity.ok(solicitudService.cambiarEstado(id, request));` → delega.

Línea 84-90: anotaciones OpenAPI.

Línea 91: `@DeleteMapping("/{id}")` → DELETE `/api/v1/solicitudes/{id}`.

Línea 92-93: método y delegación.

Línea 94: `return ResponseEntity.noContent().build();` → 204.

Paso 7.5.03 — Crear el DocumentoController

Crea la clase `DocumentoController` en el paquete `documento`:

```
package es.mecd.demo.miproyecto.documento;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import java.net.URI;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.util.List;

@RestController
@RequestMapping("/api/v1/solicitudes/{solicitudId}/documentos")
public class DocumentoController {
    private final DocumentoService documentoService;

    public DocumentoController(DocumentoService documentoService) {
        this.documentoService = documentoService;
    }

    @Operation(summary = "Listar documentos de una solicitud")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Lista de documentos"),
        @ApiResponse(responseCode = "404", description = "Solicitud no encontrada")
    })
    @GetMapping
    public List<DocumentoResponseDTO> listar(@PathVariable Long solicitudId) {
        return documentoService.listarPorSolicitud(solicitudId);
    }

    @Operation(summary = "Añadir documento a una solicitud")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "201", description = "Documento añadido"),
        @ApiResponse(responseCode = "400", description = "Fichero inválido"),
        @ApiResponse(responseCode = "404", description = "Solicitud no encontrada"),
        @ApiResponse(responseCode = "409", description = "La solicitud no permite añadir documentos")
    })
}
```



```

@PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<DocumentoResponseDTO> anadir(
    @PathVariable Long solicitudId,
    @RequestParam("fichero") MultipartFile fichero) {
    DocumentoResponseDTO creado = documentoService.anadir(solicitudId, fichero);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(creado.getIdificador())
        .toUri();
    return ResponseEntity.created(location).body(creado);
}

@Operation(summary = "Eliminar documento de una solicitud")
@ApiResponses(value = {
    @ApiResponse(responseCode = "204", description = "Documento eliminado"),
    @ApiResponse(responseCode = "404", description = "Solicitud o documento no encontrados"),
    @ApiResponse(responseCode = "409", description = "La solicitud no permite eliminar documentos")
})
@DeleteMapping("/{documentoId}")
public ResponseEntity<Void> eliminar(
    @PathVariable Long solicitudId,
    @PathVariable Long documentoId) {
    documentoService.eliminar(solicitudId, documentoId);
    return ResponseEntity.noContent().build();
}
}

```

Explicación línea por línea

Líneas 1-12: package e imports. Fíjate en los de MultipartFile y MediaType. Línea 14: @RestController.

Línea 15: @RequestMapping("/api/v1/solicitudes/{solicitudId}/documentos") → prefijo con path variable. Todos los endpoints de esta clase incluyen {solicitudId} en la ruta.

Línea 16: public class DocumentoController { → declara la clase.

Líneas 18-22: dependencia y constructor.

Líneas 24-28: anotaciones OpenAPI.

Línea 29: @GetMapping → GET /api/v1/solicitudes/{solicitudId}/documentos.

Línea 30: public List listar(@PathVariable Long solicitudId) { → recibe el ID de la solicitud. Devuelve una lista (no Page, porque los documentos de una solicitud no suelen ser muchos).

Línea 31: return documentoService.listarPorSolicitud(solicitudId); → delega.

Línea 33-39: anotaciones OpenAPI.

Línea 40: @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE) → mapea a POST /api/v1/solicitudes/{solicitudId}/documentos. consumes = MULTIPART_FORM_DATA_VALUE indica que este endpoint acepta peticiones multipart/form-data, no JSON. Es lo que se usa para subir ficheros.

Línea 41-43: parámetros:

- @PathVariable Long solicitudId → el ID de la solicitud.
- @RequestParam("fichero") MultipartFile fichero → el fichero subido. @RequestParam porque en multipart/form-data los ficheros son parámetros del formulario, no cuerpo JSON. Línea 44: DocumentoResponseDTO creado = documentoService.anadir(solicitudId, fichero); → delega.

Después de crear el documento se construye una URI Location con ServletUriComponentsBuilder y se devuelve 201 Created con el DTO. Así el comportamiento coincide con el resto de recursos creados por la API.

Línea 47-52: anotaciones OpenAPI.

Línea 53: `@DeleteMapping("/{documentoId}") → DELETE /api/v1/solicitudes/{solicitudId}/documentos/{documentoId}`.

Línea 54-56: parámetros: dos `@PathVariable`.

Línea 57: `documentoService.eliminar(solicitudId, documentoId); → delega`.

Línea 58: `return ResponseEntity.noContent().build(); → 204`.

Paso 7.5.04 — Crear el DTO de cambio de roles

El `UsuarioController` necesita un DTO para cambiar los roles. Crea `CambioRolesRequestDTO` en el paquete `auth`:

```
package es.mecd.demo.miproyecto.auth;
import jakarta.validation.constraints.NotEmpty;

import java.util.List;

public class CambioRolesRequestDTO {

    @NotEmpty(message = "Debe indicar al menos un rol")
    private List<String> roles;

    public List<String> getRoles() { return roles; }
    public void setRoles(List<String> roles) { this.roles = roles; }
}
```

`@NotEmpty` obliga a indicar al menos un rol.

Paso 7.5.05 — Crear el UsuarioService

Crea primero `UsuarioResponseDTO`, que es el tipo devuelto por el servicio:

```
package es.mecd.demo.miproyecto.auth;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.util.List;
public class UsuarioResponseDTO {
    @JsonProperty("id") private String identificador;
    private String username; private String email; private List<String> roles; private boolean activo;
    public String getIdentificador() { return identificador; }
    public void setIdentificador(String identificador) { this.identificador = identificador; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public List<String> getRoles() { return roles; }
    public void setRoles(List<String> roles) { this.roles = roles; }
    public boolean isActive() { return activo; }
    public void setActivo(boolean activo) { this.activo = activo; }
}
```

El `UsuarioController` necesita un `UsuarioService`. Créalo ahora:

```
package es.mecd.demo.miproyecto.auth;

import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.HashSet;
import java.util.List;
import java.util.Set;

@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;
    private final RolRepository rolRepository;

    public UsuarioService(UsuarioRepository usuarioRepository,
```

```

        RolRepository rolRepository) {
            this.usuarioRepository = usuarioRepository;
            this.rolRepository = rolRepository;
        }

        @Transactional(readOnly = true)
        public Page<UsuarioResponseDTO> listar(Pageable pageable) {
            return usuarioRepository.findAll(pageable).map(this::toDTO);
        }

        @Transactional(readOnly = true)
        public UsuarioResponseDTO consultar(Long id) {
            return usuarioRepository.findById(id)
                .map(this::toDTO)
                .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", id));
        }

        @Transactional
        public UsuarioResponseDTO cambiarRoles(Long id, List<String> nombresRoles) {
            Usuario usuario = usuarioRepository.findById(id)
                .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", id));

            Set<Rol> roles = new HashSet<>();
            for (String nombre : nombresRoles) {
                Rol rol = rolRepository.findByNombre(nombre)
                    .orElseThrow(() -> new RecursoNoEncontradoException("Rol", nombre));
                roles.add(rol);
            }

            usuario.setRoles(roles);
            return toDTO(usuarioRepository.save(usuario));
        }

        private UsuarioResponseDTO toDTO(Usuario usuario) {
            UsuarioResponseDTO dto = new UsuarioResponseDTO();
            dto.setIdentificador(usuario.getId() != null ? usuario.getId().toString() : null);
            dto.setUsername(usuario.getUsername());
            dto.setEmail(usuario.getEmail());
            dto.setActivo(usuario.isActivo());
            dto.setRoles(usuario.getRoles().stream()
                .map(Rol::getNombre)
                .sorted()
                .toList());
            return dto;
        }
    }
}

```

Explicación línea por línea

Línea 1: package es.mecd.demo.miproyecto.auth;

Líneas 3-12: imports.

Línea 14: @Service.

Líneas 16-22: dos dependencias y constructor.

- usuarioRepository → acceso a usuarios.
- rolRepository → para buscar los roles al cambiarlos. Línea 24-27: método listar.
- @Transactional(readOnly = true) → consulta.
- usuarioRepository.findAll(pageable).map(this::toDTO) → obtiene la página y transforma.

Línea 29-34: método consultar.

- findById(id).map(this::toDTO).orElseThrow(...) → busca, transforma o lanza 404.

Línea 36-48: método cambiarRoles.

- findById(id).orElseThrow(...) → busca el usuario.
- Set roles = new HashSet<>() → conjunto de roles. Usamos Set para evitar duplicados.
- for (String nombre : nombresRoles) → recorre los nombres de roles del DTO.

- `rolRepository.findByNombre(nombre).orElseThrow(...)` → busca cada rol. Si no existe, 404.
- `roles.add(rol)` → añade al conjunto.
- `usuario.setRoles(roles)` → asigna los nuevos roles.
- `toDTO(usuarioRepository.save(usuario))` → guarda y devuelve.

Línea 50-58: `toDTO` → transforma el usuario a DTO.

Verificación mínima del servicio de usuarios.

Crea `src/test/java/es/mecd/demo/miproyecto/auth/UsuarioServiceTest.java` para comprobar que el cambio de roles resuelve los nombres contra `RolRepository`:

```
package es.mecd.demo.miproyecto.auth;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;
import java.util.List;
import java.util.Optional;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
@ExtendWith(MockitoExtension.class)
class UsuarioServiceTest {
    @Mock UsuarioRepository usuarios; @Mock RolRepository roles; UsuarioService service;
    @BeforeEach void setUp() { service=new UsuarioService(usuarios,roles); }
    @Test void cambiarRolesDebeResolverRoles() {
        Usuario u=new Usuario(); u.setUsername("u"); u.setEmail("u@e.es");
        Rol admin=new Rol("ADMIN","Admin"); when(usuarios.findById(1L)).thenReturn(Optional.of(u));
        when(roles.findByNombre("ADMIN")).thenReturn(Optional.of(admin));
        when(usuarios.save(u)).thenReturn(u);
        assertEquals(List.of("ADMIN"), service.cambiarRoles(1L,List.of("ADMIN")).getRoles());
    }
}
```



Paso 7.5.06 — Crear el UsuarioController

Crea la clase `UsuarioController` en el paquete `auth`:

```
package es.mecd.demo.miproyecto.auth;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import jakarta.validation.Valid;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/usuarios")
@PreAuthorize("hasRole('ADMIN')")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @Operation(summary = "Listar usuarios")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Lista de usuarios")
    })
    @GetMapping
    public Page<UsuarioResponseDTO> listar(
        @PageableDefault(size = 20, sort = "id") Pageable pageable) {
        return usuarioService.listar(pageable);
    }

    @Operation(summary = "Consultar usuario por ID")
    @ApiResponses(value = {
```

```

        @ApiResponse(responseCode = "200", description = "Usuario encontrado"),
        @ApiResponse(responseCode = "404", description = "Usuario no encontrado")
    })
    @GetMapping("/{id}")
    public UsuarioResponseDTO consultar(@PathVariable Long id) {
        return usuarioService.consultar(id);
    }

    @Operation(summary = "Cambiar roles de un usuario")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Roles cambiados"),
        @ApiResponse(responseCode = "404", description = "Usuario o rol no encontrado")
    })
    @PatchMapping("/{id}/roles")
    public UsuarioResponseDTO cambiarRoles(
        @PathVariable Long id,
        @Valid @RequestBody CambioRolesRequestDTO request) {
        return usuarioService.cambiarRoles(id, request.getRoles());
    }
}

```

Explicación línea por línea

Líneas 1-14: package e imports.

Línea 16: @RestController.

Línea 17: @RequestMapping("/api/v1/usuarios").

Línea 18: @PreAuthorize("hasRole('ADMIN')") → a nivel de clase. Todos los métodos de esta clase requieren el rol ADMIN. Es más limpio que ponerlo en cada método.

Línea 19: public class UsuarioController {

Líneas 21-25: dependencia y constructor.

Líneas 27-31: anotaciones OpenAPI.

Línea 32: @GetMapping → GET /api/v1/usuarios.

Línea 33-35: parámetro Pageable con @PageableDefault. Ordenado por id ascendente. Línea 36: return usuarioService.listar(pageable); → delega.

Línea 38-43: anotaciones OpenAPI.

Línea 44: @GetMapping("/{id}") → GET /api/v1/usuarios/{id}.

Línea 45: public UsuarioResponseDTO consultar(@PathVariable Long id) { → método. Fíjate en que no devuelve ResponseEntity: devuelve directamente el DTO. Si el servicio lanza 404, el manejador global lo captura.

Línea 46: return usuarioService.consultar(id); → delega.

Línea 48-53: anotaciones OpenAPI.

Línea 54: @PatchMapping("/{id}/roles") → PATCH /api/v1/usuarios/{id}/roles.

Línea 55-58: parámetros.

Línea 59: return usuarioService.cambiarRoles(id, request.getRoles()); → delega.



Paso 7.5.07 — Arrancar y probar los endpoints

Reinicia la aplicación. Prueba los endpoints principales:

```

# Listar becas (público)

> **Shell:** los comandos `curl` con comillas simples y `` de continuación están escritos para Bash/Git Bash. En Windows `cmd.exe`, usa comillas dobles escapadas o ejecuta los ejemplos desde Git Bash.
curl -i http://localhost:8080/api/v1/bezas

# Consultar beca

```

```
curl -i http://localhost:8080/api/v1/becas/1

# Listar solicitudes (requiere autenticación, lo veremos en 7.6)
curl -i http://localhost:8080/api/v1/solicitudes

# Listar usuarios (requiere ADMIN, lo veremos en 7.6)
curl -i http://localhost:8080/api/v1/usuarios
```

Paso 7.5.08 — Verificar OpenAPI

Abre <http://localhost:8080/swagger-ui.html> en el navegador. Verifica que:

- Aparecen los cuatro controladores: Beca, Solicitud, Documento, Usuario.
- Cada endpoint tiene su resumen (@Operation).
- Cada endpoint muestra las respuestas posibles (@ApiResponse).

Paso 7.5.09 — Escribir tests del BecaController

Crea BecaControllerTest:

```
package es.mecd.demo.miproyecto.beca;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import com.fasterxml.jackson.databind.ObjectMapper;
import es.mecd.demo.miproyecto.auth.JwtService;
import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import java.math.BigDecimal;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(BecaController.class)
@AutoConfigureMockMvc(addFilters = false)
class BecaControllerTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @MockitoBean private JwtService jwtService;
    @MockitoBean private BecaService becaService;

    @Test
    void consultarDebeDevolver200CuandoExiste() throws Exception {
        BecaResponseDTO dto = new BecaResponseDTO();
        dto.setIdentificador("1");
        dto.setNombre("Beca General");
        when(beccaService.consultar(1L)).thenReturn(dto);
        mockMvc.perform(get("/api/v1/becas/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.nombre").value("Beca General"));
    }

    @Test
    void crearDebeDevolver201ConLocationCuandoDatosValidos() throws Exception {
        BecaResponseDTO creada = new BecaResponseDTO();
        creada.setIdentificador("1");
        creada.setNombre("Beca General");
        when(beccaService.crear(any())).thenReturn(creada);
        BecaRequestDTO request = requestValido();
        mockMvc.perform(post("/api/v1/becas")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(header().exists("Location"))
            .andExpect(jsonPath("$.nombre").value("Beca General"));
    }
}
```

```

@Test
void crearDebeDevolver400CuandoFaltanCampos() throws Exception {
    BecaRequestDTO request = new BecaRequestDTO();
    request.setNombre("");
    mockMvc.perform(post("/api/v1/becas")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.codigo").value("VALIDACION"));
}

@Test
void crearDebeDevolver409CuandoNombreDuplicado() throws Exception {
    when(becaService.crear(any()))
        .thenReturn(new RecursoDuplicadoException("Beca", "nombre", "Beca General"));
    mockMvc.perform(post("/api/v1/becas")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(requestValido())))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.codigo").value("RECURSO_DUPLICADO"));
}

@Test
void eliminarDebeDevolver204CuandoExiste() throws Exception {
    mockMvc.perform(delete("/api/v1/becas/1")).andExpect(status().isNoContent());
}

private BecaRequestDTO requestValido() {
    BecaRequestDTO request = new BecaRequestDTO();
    request.setCodigo("TEST-2026");
    request.setNombre("Beca General");
    request.setImporteMaximo(new BigDecimal("1500.00"));
    request.setAnio(2026);
    return request;
}
}

```

Paso 7.5.10 — Escribir tests del SolicitudController

Crea SolicitudControllerTest:

```

package es.mecd.demo.miproyecto.solicitud;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.patch;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import com.fasterxml.jackson.databind.ObjectMapper;
import es.mecd.demo.miproyecto.auth.JwtService;
import es.mecd.demo.miproyecto.common.exception.OperationNoPermitidaException;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(SolicitudController.class)
@AutoConfigureMockMvc(addFilters = false)
class SolicitudControllerTest {
    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @MockitoBean private JwtService jwtService;
    @MockitoBean private SolicitudService solicitudService;

    @Test
    void crearDebeDevolver201ConLocation() throws Exception {
        SolicitudResponseDTO creada = new SolicitudResponseDTO();
        creada.setIdentificador("1");
        creada.setEstado("BORRADOR");
        when(solicitudService.crear(any())).thenReturn(creada);
        SolicitudRequestDTO request = new SolicitudRequestDTO();
    }
}

```



```

        request.setAlumnoId(1L);
        request.setBecaId(1L);
        mockMvc.perform(post("/api/v1/solicitudes")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(header().exists("Location"))
            .andExpect(jsonPath("$.estado").value("BORRADOR"));
    }

    @Test
    void cambiarEstadoDebeDevolver200() throws Exception {
        SolicitudResponseDTO actualizada = new SolicitudResponseDTO();
        actualizada.setIdentificador("1");
        actualizada.setEstado("ENVIADA");
        when(solicitudService.cambiarEstado(eq(1L), any()))
            .thenReturn(actualizada);
        CambioEstadoRequestDTO request = new CambioEstadoRequestDTO();
        request.setEstado("ENVIADA");
        mockMvc.perform(patch("/api/v1/solicitudes/1/estado")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.estado").value("ENVIADA"));
    }

    @Test
    void cambiarEstadoDebeDevolver409CuandoEstadoNoPermitido() throws Exception {
        when(solicitudService.cambiarEstado(eq(1L), any()))
            .thenThrow(new OperacionNoPermitidaException("No se puede cambiar el estado"));
        CambioEstadoRequestDTO request = new CambioEstadoRequestDTO();
        request.setEstado("ENVIADA");
        mockMvc.perform(patch("/api/v1/solicitudes/1/estado")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.codigo").value("OPERACION_NO_PERMITIDA"));
    }
}

```

Paso 7.5.11 — Errores comunes del ejercicio

Error	Causa	Solución
@RequestBody en un fichero	Se usa JSON para un multipart	Usar @RequestParam MultipartFile
415 Unsupported Media Type	Content-Type o consumes incorrecto	Revisar cabeceras y mapping
400 Bad Request en multipart	Falta el parámetro del fichero	Añadir @RequestParam("fichero")
No aparece Location en POST	Falta ResponseEntity.created(...)	Construir la respuesta correctamente
Página vacía	Pageable mal configurado	Revisar @PageableDefault
El enum no se convierte	El texto no coincide con un valor	Usar un valor válido
NoSuchMethodError con PageableDefault	Dependencia web incompatible o ausente	Revisar pom.xml
La URL no coincide	@RequestMapping incorrecto	Corregir la ruta
No se capturan path variables	Falta @PathVariable	Añadirlo
@PreAuthorize no se aplica	Falta @EnableMethodSecurity	Habilitar seguridad por método

Paso 7.5.12 — Reto resuelto — Endpoint para consultar las solicitudes de un alumno

Reto. Añade GET /api/v1/alumnos/{alumnoId}/solicitudes con paginación.

Paso 1 — Añadir la operación al servicio.

```

@Transactional(readOnly = true)
public Page<SolicitudResponseDTO> listarPorAlumno(Long alumnoId, Pageable pageable) {

```



```

        if (!alumnoRepository.existsById(alumnoId)) {
            throw new RecursoNoEncontradoException("Alumno", alumnoId);
        }

        return solicitudRepository.findByAlumnoId(alumnoId, pageable)
            .map(this::toDTO);
    }

```

Paso 2 — Utilizar la consulta paginada del repositorio.

`SolicitudBecaRepository` ya dispone de `findByAlumnoId(Long alumnoId, Pageable pageable)` con `@EntityGraph`.

Paso 3 — Inyectar `SolicitudService` y añadir el endpoint al `AlumnoController`.

Añade estos imports:

```

import es.mecd.demo.miproyecto.solicitud.SolicitudResponseDTO;
import es.mecd.demo.miproyecto.solicitud.SolicitudService;

```

Amplía el campo y el constructor existentes. El constructor de `AlumnoController` pasa a recibir los dos servicios:

```

private final AlumnoService alumnoService;
private final SolicitudService solicitudService;

public AlumnoController(AlumnoService alumnoService, SolicitudService solicitudService) {
    this.alumnoService = alumnoService;
    this.solicitudService = solicitudService;
}

```

Añade el endpoint:

```

@GetMapping("/{alumnoId}/solicitudes")
public Page<SolicitudResponseDTO> listarSolicitudes(
    @PathVariable Long alumnoId,
    @PageableDefault(size = 20) Pageable pageable) {
    return solicitudService.listarPorAlumno(alumnoId, pageable);
}

```

Paso 4 — Probar.

```

curl -i "http://localhost:8080/api/v1/alumnos/1/solicitudes?page=0&size=10"

```

Paso 5 — Probar el endpoint con `MockMvc`.

Crea `src/test/java/es/mecd/demo/miproyecto/alumno/AlumnoControllerTest.java`. Como es un slice `@WebMvcTest`, se mockean `JwtService`, `AlumnoService` y `SolicitudService` con `@MockitoBean`:

```

package es.mecd.demo.miproyecto.alumno;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import es.mecd.demo.miproyecto.auth.JwtService;
import es.mecd.demo.miproyecto.solicitud.SolicitudService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.data.domain.PageImpl;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(AlumnoController.class)
@AutoConfigureMockMvc(addFilters = false)
class AlumnoControllerTest {

```

```

@Autowired
private MockMvc mockMvc;

@MockitoBean
private JwtService jwtService;

@MockitoBean
private AlumnoService alumnoService;

@MockitoBean
private SolicitudService solicitudService;

@Test
void retolistarSolicitudesAlumnoDebeResponder200() throws Exception {
    when(solicitudService.listarPorAlumno(anyLong(), any()))
        .thenReturn(new PageImpl<>(java.util.List.of()));

    mockMvc.perform(get("/api/v1/alumnos/1/solicitudes"))
        .andExpect(status().isOk());
}
}

```

✓ Resultado esperado global

Al final del ejercicio, deberías tener:

- Un BecaController con CRUD completo.
- Un SolicitudController con listar, consultar, crear, cambiar estado y cancelar.
- Un DocumentoController con listar, añadir y eliminar.
- Un UsuarioController con listar, consultar y cambiar roles, protegido por ADMIN.
- Un UsuarioService con los métodos correspondientes.
- Tests de MockMvc para los controladores.
- Documentación OpenAPI accesible en Swagger UI.
- Los endpoints probados con curl.

✖ Resumen técnico del ejercicio

El ejercicio ha demostrado:

- @RestController y @RequestMapping: estructura base.
- @GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping: métodos HTTP.
- @PathVariable y @RequestParam: extracción de datos.
- @RequestBody y @Valid: entrada de datos con validación.
- @PageableDefault: paginación con valores por defecto.
- ResponseEntity: control de códigos y cabeceras.
- ServletUriComponentsBuilder: construcción de la cabecera Location.
- MultipartFile: subida de ficheros.
- @PreAuthorize a nivel de clase: todos los métodos requieren el rol.
- OpenAPI: documentación automática.
- @WebMvcTest y MockMvc: tests de controladores.

← END Conclusión y enlace al siguiente punto

En este punto 7.5 hemos implementado los controladores REST del proyecto:

- BecaController: CRUD completo.
- SolicitudController: gestión de solicitudes con todas las operaciones.
- DocumentoController: gestión de documentos con subida de ficheros.

- `UsuarioController`: gestión de usuarios (solo ADMIN).

Hemos visto cómo los controladores delegan en los servicios, cómo construyen las respuestas con los códigos de estado correctos, cómo reciben ficheros con `MultipartFile`, y cómo se documentan con OpenAPI.

La idea clave: el controlador es la puerta de entrada HTTP. No tiene lógica de negocio; solo traduce HTTP a Java y viceversa. Todo lo demás está en el servicio.

En el siguiente punto, 7.6 – Seguridad básica, configuraremos la seguridad del proyecto: roles, reglas de URL, `@PreAuthorize` en los endpoints, CORS, y ajustaremos el `SecurityFilterChain` para que todo funcione con JWT.

✓ Fin del Punto 7.5.

◆ Punto 7.6 — Seguridad básica

🎯 Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Integrar Spring Security y JWT en el proyecto completo.
2. Definir roles específicos del dominio (CIUDADANO, GESTOR, ADMIN).
3. Configurar el `SecurityFilterChain` con reglas por endpoint.
4. Aplicar `@PreAuthorize` en las operaciones que lo requieren.
5. Configurar CORS en la cadena de filtros de seguridad.
6. Inicializar usuarios y roles con datos de ejemplo.
7. Verificar la seguridad con curl y con tests.
8. Diagnosticar y resolver los errores más comunes al integrar seguridad en un proyecto.

✂ IMPLEMENTACIÓN

🔧 Paso 7.6.01 — Añadir los roles CIUDADANO y GESTOR al inicializador

Amplía `UsuariosInicialesConfig` para crear los roles y usuarios de dominio. Usa el helper `rol(...)` del PREP para no duplicar registros:

```
Rol ciudadano = rol(roles, "CIUDADANO", "Ciudadano solicitante");
Rol gestor = rol(roles, "GESTOR", "Gestor de solicitudes");
crear(usuarios, encoder, "ciudadano", "ciudadano123",
      "ciudadano@educacion.gob.es", ciudadano);
crear(usuarios, encoder, "gestor", "gestor123",
      "gestor@educacion.gob.es", gestor, user);
```

Actualiza también `AuthService.registrar(...)`: el rol por defecto deja de ser `USER` y pasa a `CIUDADANO`, manteniendo `USER` sólo como fallback defensivo si la base todavía no contiene el rol nuevo.

```
Rol rol = rolRepository.findByNombre("CIUDADANO")
    .orElseGet(() -> rolRepository.findByNombre("USER")
    .orElseThrow(() -> new RecursoNoEncontradoException("Rol", "USER")));
```

A partir de aquí un usuario registrado puede ejecutar el flujo de ciudadano de 7.7.

Paso 7.6.02 — Actualizar el SecurityConfig

Sustituye SecurityConfig por esta versión completa. Incluye el AuthenticationManager necesario por AuthService y las reglas de URL previas al reto CONSULTOR.

Modifica el SecurityConfig con las reglas del proyecto y la configuración de CORS:

```
package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.auth.JwtAccessDeniedHandler;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationEntryPoint;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationFilter;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() { return
PasswordEncoderFactories.createDelegatingPasswordEncoder(); }
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws
Exception {
        return configuration.getAuthenticationManager();
    }
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationFilter filter,
        JwtAuthenticationEntryPoint entryPoint, JwtAccessDeniedHandler deniedHandler) throws Exception
    {
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST,
                    "/api/v1/auth/registro",
                    "/api/v1/auth/login",
                    "/api/v1/auth/refresh").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/becas", "/api/v1/becas/**").permitAll()
                .requestMatchers("/h2-console/**", "/swagger-ui/**", "/swagger-ui.html", "/v3/api-
docs/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/v1/becas").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PUT, "/api/v1/becas/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/api/v1/becas/**").hasRole("ADMIN")
                .requestMatchers("/api/v1/usuarios/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/v1/solicitudes/*/
estado").hasAnyRole("GESTOR", "ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/v1/solicitudes").hasRole("CIUDADANO")
                .requestMatchers(HttpMethod.DELETE, "/api/v1/solicitudes/*").hasRole("CIUDADANO")
                .requestMatchers(HttpMethod.POST, "/api/v1/solicitudes/*
documentos").hasRole("CIUDADANO")
                .requestMatchers(HttpMethod.DELETE, "/api/v1/solicitudes/*documentos/
**").hasRole("CIUDADANO")
                .requestMatchers(HttpMethod.GET, "/api/v1/solicitudes/**")
                    .hasAnyRole("CIUDADANO", "GESTOR", "ADMIN")
                .requestMatchers("/api/v1/solicitudes/**").authenticated()
                .anyRequest().authenticated()
            )
            .exceptionHandling(ex ->
ex.authenticationEntryPoint(entryPoint).accessDeniedHandler(deniedHandler))
            .addFilterBefore(filter, UsernamePasswordAuthenticationFilter.class);
    }
}
```

```

        return http.build();
    }
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:4200"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*")); config.setExposedHeaders(List.of("Location"));
        config.setAllowCredentials(true); config.setMaxAge(3600L);
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config); return source;
    }
}

```

Explicación línea por línea

Línea 24: @Configuration, @EnableWebSecurity, @EnableMethodSecurity → configuración de seguridad y seguridad por método.

Línea 26-29: bean PasswordEncoder.

Línea 31-38: bean SecurityFilterChain con las dependencias inyectadas.

Línea 40: .cors(cors -> cors.configurationSource(corsConfigurationSource())) → habilita CORS en la cadena de filtros de seguridad. Sin esta línea, Spring Security bloquea las peticiones preflight OPTIONS.

CORS se configura aquí, y no solo en `WebMvcConfigurer`, porque cuando se usa Spring Security la configuración debe registrarse en la cadena de filtros de seguridad. Si solo se configura en `WebMvcConfigurer`, Spring Security bloquea las peticiones preflight OPTIONS con un 401, y el navegador no puede consumir la API. Por eso usamos `CorsConfigurationSource` como bean y lo referenciamos en `http.cors(...)`.

Línea 41: .csrf(csrf -> csrf.disable()) → desactiva CSRF. Para APIs REST sin sesión, no es necesaria. Línea 42-43: .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) → no se crean sesiones HTTP.

Línea 45-47: endpoints públicos.

- `POST /api/v1/auth/registro`, `POST /api/v1/auth/login` y `POST /api/v1/auth/refresh` → allowlist pública explícita. No se abre todo `/api/v1/auth/**`.
- `HttpMethod.GET`, `"/api/v1/becas"`, `"/api/v1/becas/**"` → el listado y la consulta de becas son públicos. Fíjate en que se especifica el método HTTP para que solo el GET sea público.
- `/h2-console/`, `/swagger-ui/`, `/v3/api-docs/**` → consola y documentación.

Línea 49-52: gestión de becas. POST, PUT y DELETE sobre becas requieren rol ADMIN.

Línea 54-55: gestión de usuarios. Solo ADMIN.

Línea 57-59: cambio de estado de solicitudes. Requiere GESTOR o ADMIN. La URL `/api/v1/solicitudes//estado usa` para un segmento variable (el ID).

Línea 61-65: documentos.

- POST sobre documentos requiere CIUDADANO.
- DELETE sobre documentos requiere CIUDADANO.

Línea 67-68: el resto de solicitudes requiere autenticación.

Línea 70-71: cualquier otra ruta requiere autenticación.

Línea 73-77: manejadores de error personalizados. Línea 79-80: filtro JWT antes del `UsernamePasswordAuthenticationFilter`.

Línea 83-95: bean CorsConfigurationSource.

- `config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:4200"))` → orígenes permitidos (front-end en desarrollo).
- `config.setAllowedMethods(...)` → métodos permitidos, incluido OPTIONS.
- `config.setAllowedHeaders(List.of("*"))` → todas las cabeceras.
- `config.setExposedHeaders(List.of("Location"))` → el front-end puede leer la cabecera Location.
- `config.setAllowCredentials(true)` → permite cookies y cabecera Authorization.
- `config.setMaxAge(3600L)` → cachea el preflight durante 1 hora.
- `UrlBasedCorsConfigurationSource` → registra la configuración para todas las rutas `/api/**`.



Paso 7.6.03 — Añadir @PreAuthorize en las operaciones contextuales

Refuerza las reglas de URL con seguridad por método.

En `SolicitudController`:

```
@PostMapping
@PreAuthorize("hasRole('CIUDADANO')")
public ResponseEntity<SolicitudResponseDTO> crear(
    @Valid @RequestBody SolicitudRequestDTO request) {
    // cuerpo ya creado en 7.5.2
}

@PatchMapping("/{id}/estado")
@PreAuthorize("hasAnyRole('GESTOR', 'ADMIN')")
public ResponseEntity<SolicitudResponseDTO> cambiarEstado(
    @PathVariable Long id,
    @Valid @RequestBody CambioEstadoRequestDTO request) {
    // cuerpo ya creado en 7.5.2
}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('CIUDADANO')")
public ResponseEntity<Void> cancelar(@PathVariable Long id) {
    // cuerpo ya creado en 7.5.2
}
```

En `AlumnoController`, aplica las reglas definidas en el diseño: crear/actualizar requieren CIUDADANO y eliminar requiere ADMIN.

```
@PostMapping
@PreAuthorize("hasRole('CIUDADANO')")
// ... método crear existente

@PutMapping("/{id}")
@PreAuthorize("hasRole('CIUDADANO')")
// ... método actualizar existente

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
// ... método eliminar existente
```

Estas anotaciones complementan el `SecurityFilterChain`; no sustituyen las reglas de URL.



Paso 7.6.04 — Añadir @PreAuthorize en el BecaController

Añade las anotaciones al `BecaController`:

```
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<BecaResponseDTO> crear(
    @Valid @RequestBody BecaRequestDTO request) {
```

```

    // ...
}

@PutMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<BecaResponseDTO> actualizar(
    @PathVariable Long id,
    @Valid @RequestBody BecaRequestDTO request) {
    // ...
}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    // ...
}

```

Paso 7.6.05 — Añadir @PreAuthorize en el DocumentoController

Añade las anotaciones al DocumentoController:

```

@PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
@PreAuthorize("hasRole('CIUDADANO')")
public ResponseEntity<DocumentoResponseDTO> anadir(
    @PathVariable Long solicitudId,
    @RequestParam("fichero") MultipartFile fichero) {
    // ...
}

@DeleteMapping("/{documentoId}")
@PreAuthorize("hasRole('CIUDADANO')")
public ResponseEntity<Void> eliminar(
    @PathVariable Long solicitudId,
    @PathVariable Long documentoId) {
    // ...
}

```

Paso 7.6.06 — Arrancar y probar con curl

Reinicia la aplicación. Prueba los escenarios:

```

# 1. Endpoint público sin token → 200
curl -i http://localhost:8080/api/v1/becas

# 2. Endpoint protegido sin token → 401
curl -i http://localhost:8080/api/v1/solicitudes

# 3. Login como ciudadano
TOKEN_CIUDADANO=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"ciudadano","password":"ciudadano123"}' | jq -r '.access_token')

# 4. Ciudadano lista solicitudes → 200
curl -i -H "Authorization: Bearer $TOKEN_CIUDADANO" \
http://localhost:8080/api/v1/solicitudes

# 5. Ciudadano intenta crear una beca → 403
curl -i -X POST -H "Authorization: Bearer $TOKEN_CIUDADANO" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/becas \
-d '{"codigo":"OTRA-2026","nombre":"Otra Beca","importeMaximo":1000.00,"anio":2026}'

# 6. Login como gestor
TOKEN_GESTOR=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"gestor","password":"gestor123"}' | jq -r '.access_token')

# 7. Gestor cambia estado de una solicitud → 200
curl -i -X PATCH -H "Authorization: Bearer $TOKEN_GESTOR" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/solicitudes/1/estado \
-d '{"estado":"EN_REVISION'}

# 8. Login como admin
TOKEN_ADMIN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin123"}' | jq -r '.access_token')

```



```
# 9. Admin crea una beca → 201
curl -i -X POST -H "Authorization: Bearer $TOKEN_ADMIN" \
  -H "Content-Type: application/json" \
  http://localhost:8080/api/v1/becas \
  -d '{"codigo":"EXCELENCIA-2026","nombre":"Beca de Excelencia","descripcion":"Para alumnos destacados","importeMaximo":2000.00,"anio":2026}'
```

Observa los códigos de estado. Cada operación devuelve el código correcto según el rol del usuario.

Paso 7.6.07 — Probar con un token inválido

```
curl -i -H "Authorization: Bearer tokenInvalido" \
  http://localhost:8080/api/v1/solicitudes
```

Verás un 401 con el JSON del JwtAuthenticationEntryPoint. El filtro JWT detecta que el token no es válido y no autentica. El AuthorizationFilter rechaza la petición.

Paso 7.6.08 — Verificar CORS

Prueba un preflight OPTIONS desde un origen permitido:

```
curl -i -X OPTIONS http://localhost:8080/api/v1/solicitudes \
  -H "Origin: http://localhost:3000" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: authorization,content-type"
```

Verás las cabeceras CORS:

```
HTTP/1.1 200
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
Access-Control-Allow-Headers: authorization,content-type
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 3600
```

El servidor responde al preflight con las cabeceras que el navegador necesita. Sin la configuración de CORS en la cadena de filtros de seguridad, este preflight habría devuelto 401 y el navegador no habría enviado la petición real.

Paso 7.6.09 — Escribir tests de seguridad

Crea SolicitudSecurityTest:

```
package es.mecd.demo.miproyecto.solicitud;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.patch;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import es.mecd.demo.miproyecto.auth.JwtAccessDeniedHandler;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationEntryPoint;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationFilter;
import es.mecd.demo.miproyecto.auth.JwtService;
import es.mecd.demo.miproyecto.auth.SecurityErrorWriter;
import es.mecd.demo.miproyecto.config.SecurityConfig;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.data.domain.PageImpl;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(SolicitudController.class)
```



```

@Import({SecurityConfig.class, SecurityErrorWriter.class, JwtAuthenticationEntryPoint.class,
        JwtAccessDeniedHandler.class, JwtAuthenticationFilter.class})
class SolicitudSecurityTest {
    @Autowired private MockMvc mockMvc;
    @MockitoBean private SolicitudService service;
    @MockitoBean private JwtService jwt;

    @Test
    void sinAutenticacionDebeDar401() throws Exception {
        mockMvc.perform(get("/api/v1/solicitudes")).andExpect(status().isUnauthorized());
    }

    @Test
    void ciudadanoPuedeListar() throws Exception {
        when(service.listar(any(), any(), any(), any()))
            .thenReturn(new PageImpl<>(java.util.List.of()));
        mockMvc.perform(get("/api/v1/solicitudes").with(user("c").roles("CIUDADANO")))
            .andExpect(status().isOk());
    }

    @Test
    void ciudadanoPuedeCrear() throws Exception {
        SolicitudResponseDTO creada = new SolicitudResponseDTO();
        creada.setIdentificador("1");
        when(service.crear(any())).thenReturn(creada);
        mockMvc.perform(post("/api/v1/solicitudes").with(user("c").roles("CIUDADANO"))
            .contentType("application/json")
            .content("{\"alumnoId\":1,\"becaId\":1}"))
            .andExpect(status().isCreated());
    }

    @Test
    void gestorNoPuedeCrear() throws Exception {
        mockMvc.perform(post("/api/v1/solicitudes").with(user("g").roles("GESTOR"))
            .contentType("application/json")
            .content("{\"alumnoId\":1,\"becaId\":1}"))
            .andExpect(status().isForbidden());
    }

    @Test
    void consultorPuedeListar() throws Exception {
        when(service.listar(any(), any(), any(), any()))
            .thenReturn(new PageImpl<>(java.util.List.of()));
        mockMvc.perform(get("/api/v1/solicitudes").with(user("c").roles("CONSULTOR")))
            .andExpect(status().isOk());
    }

    @Test
    void consultorNoPuedeCambiarEstado() throws Exception {
        mockMvc.perform(patch("/api/v1/solicitudes/1/estado").with(user("c").roles("CONSULTOR"))
            .contentType("application/json").content("{\"estado\":\"EN_REVISION\"}"))
            .andExpect(status().isForbidden());
    }
}

```

Paso 7.6.10 — Escribir tests del BecaController con roles

Crea BecaControllerSecurityTest:

```

package es.mecd.demo.miproyecto.beca;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static
    org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import es.mecd.demo.miproyecto.auth.JwtAccessDeniedHandler;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationEntryPoint;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationFilter;
import es.mecd.demo.miproyecto.auth.JwtService;
import es.mecd.demo.miproyecto.auth.SecurityErrorWriter;
import es.mecd.demo.miproyecto.config.SecurityConfig;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;

```

```

import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(BecaController.class)
@Import({SecurityConfig.class, SecurityErrorWriter.class, JwtAuthenticationEntryPoint.class,
        JwtAccessDeniedHandler.class, JwtAuthenticationFilter.class})
class BecaControllerSecurityTest {
    @Autowired private MockMvc mockMvc;
    @MockitoBean private BecaService service;
    @MockitoBean private JwtService jwt;

    @Test
    void listarEsPublico() throws Exception {
        mockMvc.perform(get("/api/v1/becas")).andExpect(status().isOk());
    }

    @Test
    void ciudadanoNoPuedeCrearBeca() throws Exception {
        mockMvc.perform(post("/api/v1/becas").with(user("c").roles("CIUDADANO"))
            .contentType("application/json")
            .content("{\"codigo\":\"TEST-2026\",\"nombre\":\"Beca\",
                \"importeMaximo\":1000.00,\"anio\":2026}"))
            .andExpect(status().isForbidden());
    }

    @Test
    void adminPuedeCrearBeca() throws Exception {
        BecaResponseDTO creada = new BecaResponseDTO();
        creada.setIdentificador("1");
        when(service.crear(any())).thenReturn(creada);
        mockMvc.perform(post("/api/v1/becas").with(user("a").roles("ADMIN"))
            .contentType("application/json")
            .content("{\"codigo\":\"TEST-2026\",\"nombre\":\"Beca\",
                \"importeMaximo\":1000.00,\"anio\":2026}"))
            .andExpect(status().isCreated());
    }
}

```

Paso 7.6.11 — Errores comunes del ejercicio

Error	Causa	Solución
@PreAuthorize se ignora	Falta @EnableMethodSecurity	Añadirlo
Endpoints públicos devuelven 401	Reglas mal ordenadas	Colocar anyRequest().authenticated() al final
Endpoint de admin accesible	Patrón de URL incorrecto	Revisar matcher y método HTTP
El rol no coincide	Confusión entre ROLE_ y hasRole	Usar hasRole("ADMIN")
Varios roles no funcionan	Se usa hasRole	Usar hasAnyRole
Tests devuelven 403	Rol de @WithMockUser incorrecto	Ajustar el rol esperado
Endpoint de documentos no acepta fichero	Falta MULTIPART_FORM_DATA	Añadir consumes
Ciudadano no puede listar solicitudes	Regla de autenticación o rol incorrecta	Revisar matcher
Endpoint público devuelve 401	Regla de URL mal colocada	Revisar el orden
CORS bloquea preflight	Falta http.cors()	Configurarlo en SecurityFilterChain
El front-end recibe 401 en OPTIONS	Security bloquea preflight	Configurar CORS en la cadena

Paso 7.6.12 — Reto resuelto — Añadir un rol CONSULTOR

Reto. Añade un rol `CONSULTOR` que pueda consultar solicitudes mediante `GET`, pero no modificarlas.

Paso 1 — Crear el rol de forma idempotente con el helper del PREP.

```
Rol consultor = rol(roles, "CONSULTOR", "Consultor");
```

Paso 2 — Crear el usuario consultor con el mismo helper de usuarios.

```
crear(usuarios, encoder, "consultor", "consultor123",  
      "consultor@educacion.gob.es", consultor);
```

Paso 3 — Permitir consultas GET.

```
.requestMatchers(HttpMethod.GET, "/api/v1/solicitudes/**")  
  .hasAnyRole("CIUDADANO", "GESTOR", "ADMIN", "CONSULTOR")
```

Las operaciones de modificación mantienen sus permisos específicos; por ejemplo, el cambio de estado sigue reservado a `GESTOR` y `ADMIN`.

Paso 4 — Probar el rol.

```
TOKEN_CONSULTOR=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"consultor","password":"consultor123"}' | jq -r '.access_token')  
  
# Listar solicitudes → 200  
curl -i -H "Authorization: Bearer $TOKEN_CONSULTOR" \  
http://localhost:8080/api/v1/solicitudes  
  
# Cambiar estado → 403  
curl -i -X PATCH -H "Authorization: Bearer $TOKEN_CONSULTOR" \  
-H "Content-Type: application/json" \  
http://localhost:8080/api/v1/solicitudes/1/estado \  
-d '{"estado":"EN_REVISION"}'
```

✓ Resultado esperado global

Al final del ejercicio, deberías tener:

- Los roles CIUDADANO, GESTOR, ADMIN, CONSULTOR y USER añadidos al proyecto.
- Usuarios con cada rol.
- Un SecurityFilterChain con reglas por URL y por método.
- CORS configurado en la cadena de filtros de seguridad.
- Anotaciones `@PreAuthorize` en los controladores.
- Tests de seguridad para los controladores.
- Los endpoints protegidos según los roles.

✚ Resumen técnico del ejercicio

El ejercicio ha demostrado:

- Roles del dominio: CIUDADANO, GESTOR, ADMIN, CONSULTOR, USER.
- SecurityFilterChain completo: reglas de URL con HttpMethod.
- CORS en la cadena de filtros: `http.cors()` y `CorsConfigurationSource`.
- `@PreAuthorize`: refuerzo en los métodos.
- `hasRole` y `hasAnyRole`: con prefijo automático.
- `permitAll` para endpoints públicos.
- `authenticated` para endpoints autenticados.
- `@WithMockUser` en tests: simular roles.
- Diferencia 401 vs 403.



Conclusión y enlace al siguiente punto

En este punto 7.6 hemos integrado Spring Security y JWT en el proyecto completo:

- Roles del dominio: CIUDADANO, GESTOR, ADMIN, CONSULTOR, USER.
- Reglas de URL: por endpoint y por método HTTP.
- Reglas de método: con @PreAuthorize.
- CORS en la cadena de filtros: CorsConfigurationSource + http.cors().
- Datos iniciales: usuarios con cada rol.
- Verificación: con curl y tests.

La idea clave: la seguridad se integra en todas las capas. Las reglas de URL protegen áreas generales. Las anotaciones protegen operaciones específicas. CORS permite al front-end consumir la API. Y los tests garantizan que las reglas se cumplen.

◆ Punto 7.7 — Pruebas finales del API



Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Verificar el proyecto completo con una batería de pruebas manuales.
2. Ejecutar todos los tests automatizados y analizar los resultados.
3. Revisar la documentación de OpenAPI y validar los contratos.
4. Preparar una colección de Postman lista para entregar.
5. Realizar una prueba de carga básica con ab.
6. Documentar el proyecto con un README completo.
7. Diagnosticar y resolver los errores más comunes antes de la entrega.

✂ IMPLEMENTACIÓN



Paso 7.7.01 — Ejecutar todos los tests

Antes del cierre, añade un test de aceptación integral que recorra los escenarios principales del módulo. Este test complementa —no sustituye— los tests unitarios, de repositorio, MVC y seguridad creados en los puntos anteriores.

```
package es.mecd.demo.miproyecto;

import static
    org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.multipart;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.patch;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.auth.UsuarioRepository;
import es.mecd.demo.miproyecto.beca.Beca;
import es.mecd.demo.miproyecto.beca.BecaRepository;
import es.mecd.demo.miproyecto.documento.DocumentoRepository;
import es.mecd.demo.miproyecto.solicitud.EstadoSolicitud;
import es.mecd.demo.miproyecto.solicitud.SolicitudBeca;
import es.mecd.demo.miproyecto.solicitud.SolicitudBecaRepository;
import java.time.Year;
import org.junit.jupiter.api.Test;
```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.mock.web.MockMultiPartFile;
import org.springframework.test.annotation.DirtiesContext;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("dev")
@DirtiesContext(classMode = DirtiesContext.ClassMode.BEFORE_EACH_TEST_METHOD)
class ApiBecasAcceptanceIntegrationTest {
    @Autowired
    private MockMvc mockMvc;
    @Autowired
    private AlumnoRepository alumnos;
    @Autowired
    private BecaRepository becas;
    @Autowired
    private SolicitudBecaRepository solicitudes;
    @Autowired
    private DocumentoRepository documentos;
    @Autowired
    private UsuarioRepository usuarios;

    @Test
    void flujoPrincipalDebeCubrirCiudadanoGestorYConsultor() throws Exception {
        mockMvc.perform(get("/api/v1/becas"))
            .andExpect(status().isOk());

        mockMvc.perform(post("/api/v1/auth/login")
            .contentType("application/json")
            .content("{\"username\":\"ciudadano\",\"password\":\"ciudadano123\"}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.access_token").exists());

        Alumno alumno = alumnos.findAll().get(0);
        Beca beca = becas.findAll().get(0);
        String crearSolicitud = "{\"alumnoId\":\"" + alumno.getId()
            + "\",\"becaId\":\"" + beca.getId()
            + "\",\"observaciones\":\"Solicitud de aceptación\"}";

        mockMvc.perform(post("/api/v1/solicitudes")
            .with(user("ciudadano").roles("CIUDADANO"))
            .contentType("application/json")
            .content(crearSolicitud))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.estado").value("BORRADOR"));

        SolicitudBeca solicitud = solicitudes.findAll().get(0);
        MockMultiPartFile fichero = new MockMultiPartFile(
            "fichero", "dni.pdf", "application/pdf", "PDF".getBytes());
        mockMvc.perform(multipart("/api/v1/solicitudes/{id}/documentos", solicitud.getId())
            .file(fichero)
            .with(user("ciudadano").roles("CIUDADANO")))
            .andExpect(status().isCreated());

        org.junit.jupiter.api.Assertions.assertEquals(1, documentos.count());

        mockMvc.perform(get("/api/v1/solicitudes")
            .with(user("consultor").roles("CONSULTOR")))
            .andExpect(status().isOk());

        solicitud.setEstado(EstadoSolicitud.ENVIADA);
        solicitudes.saveAndFlush(solicitud);
        mockMvc.perform(patch("/api/v1/solicitudes/{id}/estado", solicitud.getId())
            .with(user("gestor").roles("GESTOR"))
            .contentType("application/json")
            .content("{\"estado\":\"EN_REVISION\"}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.estado").value("EN_REVISION"));

        mockMvc.perform(patch("/api/v1/solicitudes/{id}/estado", solicitud.getId())
            .with(user("gestor").roles("GESTOR"))
            .contentType("application/json")
            .content("{\"estado\":\"APROBADA\",\"importeConcedido\":1200.00}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.estado").value("APROBADA"));

        mockMvc.perform(get("/api/v1/estadisticas/solicitudes-por-estado")

```

```

        .with(user("gestor").roles("GESTOR")))
        .andExpect(status().isOk());
    }

    @Test
    void administracionRegistroYErroresDebenRespetarElContrato() throws Exception {
        mockMvc.perform(post("/api/v1/auth/registro")
            .contentType("application/json")
            .content("{\"username\":\"nuevo_ciudadano\", \"password\":\"nuevo12345\", \"email\":\"nuevo@educacion.gob.es\"}"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.roles[0]").value("CIUDADANO"));

        mockMvc.perform(get("/api/v1/usuarios")
            .with(user("admin").roles("ADMIN")))
            .andExpect(status().isOk());

        int anio = Year.now().getValue();
        String nuevaBeca = "{\"codigo\":\"ACEPT-\" + anio + \"\", \"nombre\":\"Beca aceptación\", \"descripcion\":\"Integración M7\", \"importeMaximo\":900.00, \"anio\":\" + anio + \"\"}";
        mockMvc.perform(post("/api/v1/becas")
            .with(user("admin").roles("ADMIN"))
            .contentType("application/json")
            .content(nuevaBeca))
            .andExpect(status().isCreated());

        Beca creada = becas.findByCodigo("ACEPT-" + anio).orElseThrow();
        mockMvc.perform(delete("/api/v1/becas/{id}", creada.getId())
            .with(user("admin").roles("ADMIN")))
            .andExpect(status().isNoContent());

        mockMvc.perform(get("/api/v1/solicitudes"))
            .andExpect(status().isUnauthorized());
        mockMvc.perform(post("/api/v1/becas")
            .with(user("gestor").roles("GESTOR"))
            .contentType("application/json")
            .content(nuevaBeca))
            .andExpect(status().isForbidden());
        mockMvc.perform(get("/api/v1/solicitudes/999999")
            .with(user("ciudadano").roles("CIUDADANO")))
            .andExpect(status().isNotFound());

        org.junit.jupiter.api.Assertions.assertTrue(usuarios.existsByUsername("nuevo_ciudadano"));
    }
}

```

Ejecuta toda la batería:

```
./mvnw test
```

En Windows CMD:

```
mvnw.cmd test
```

El resultado esperado es **0 failures, 0 errors**. El número exacto de tests se contabiliza al ejecutar la suite completa.



Paso 7.7.02 — Ejecutar los tests con cobertura

Añade el plugin de JaCoCo al pom.xml:

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

```

        <id>report</id>
        <phase>test</phase>
        <goals>
            <goal>report</goal>
        </goals>
    </execution>
</executions>
</plugin>

```

Ejecuta:

```
./mvnw test
```

Al final, abre `target/site/jacoco/index.html` en el navegador. Verás el porcentaje de cobertura por paquete y por clase.

Paso 7.7.03 — Arrancar la aplicación

Arranca la aplicación en modo desarrollo:

```
./mvnw spring-boot:run
```

Verifica que arranca sin errores. La consola debe mostrar:

```

Tomcat started on port(s): 8080 (http)
Started MiProyectoApplication in X.XXX seconds

```

Paso 7.7.04 — Probar el escenario del ciudadano

Ejecuta los siguientes comandos en orden:

```

# 1. Registro
curl -i -X POST http://localhost:8080/api/v1/auth/registro \
-H "Content-Type: application/json" \
-d '{
    "username":"nuevo_ciudadano",
    "password":"nuevo12345",
    "email":"nuevo@educacion.gob.es"
}'

# 2. Login
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"nuevo_ciudadano","password":"nuevo12345"}' | jq -r '.access_token')

echo "Token: $TOKEN"

# 3. Consultar becas (público)
curl -i http://localhost:8080/api/v1/becas

# 4. Crear una solicitud
curl -i -X POST http://localhost:8080/api/v1/solicitudes \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"alumnoId":1,"becaId":1,"observaciones":"Solicitud de prueba"}'

# 5. Consultar la solicitud (sustituye {id} por el ID devuelto)
curl -i -H "Authorization: Bearer $TOKEN" \
http://localhost:8080/api/v1/solicitudes/1
# 6. Cancelar la solicitud
curl -i -X DELETE -H "Authorization: Bearer $TOKEN" \
http://localhost:8080/api/v1/solicitudes/1

```

Observa los códigos de estado:

- Registro: 201.
- Login: 200.
- Consultar becas: 200.
- Crear solicitud: 201.

- Consultar solicitud: 200.
- Cancelar: 204.

Paso 7.7.05 — Probar el escenario del gestor

Preparar los datos del escenario. Para probar el flujo del gestor necesitas una solicitud en estado ENVIADA. En el entorno de desarrollo, abre la consola H2 y crea una solicitud de prueba antes de ejecutar los comandos del gestor:

```
INSERT INTO solicitudes (alumno_id, beca_id, estado, fecha_solicitud, observaciones)
VALUES (1, 2, 'ENVIADA', CURRENT_TIMESTAMP, 'Solicitud de prueba para el escenario gestor');
```

Comprueba en H2 el identificador asignado y úsalo en las peticiones siguientes. Si el identificador no es 2, sustituye 2 por el valor real.

```
# Login como gestor
TOKEN_GESTOR=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"gestor","password":"gestor123"}' | jq -r '.access_token')

# Listar solicitudes
curl -i -H "Authorization: Bearer $TOKEN_GESTOR" \
http://localhost:8080/api/v1/solicitudes

# Cambiar estado a EN_REVISION
curl -i -X PATCH -H "Authorization: Bearer $TOKEN_GESTOR" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/solicitudes/2/estado \
-d '{"estado":"EN_REVISION"}'

# Cambiar estado a APROBADA con importe
curl -i -X PATCH -H "Authorization: Bearer $TOKEN_GESTOR" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/solicitudes/2/estado \
-d '{"estado":"APROBADA","importeConcedido":250.00}'

# Verificar la solicitud
curl -i -H "Authorization: Bearer $TOKEN_GESTOR" \
http://localhost:8080/api/v1/solicitudes/2
```

Observa que el gestor puede cambiar estados pero no puede crear solicitudes (lo veremos en el siguiente paso).

Paso 7.7.06 — Probar el escenario del administrador

```
# Login como admin
TOKEN_ADMIN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin123"}' | jq -r '.access_token')

# Crear una beca
curl -i -X POST -H "Authorization: Bearer $TOKEN_ADMIN" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/becas \
-d '{
  "codigo":"EXCELENCIA-2026",
  "nombre":"Beca de Excelencia",
  "descripcion":"Para alumnos destacados",
  "importeMaximo":2000.00,
  "anio":2026
}'

# Listar usuarios
curl -i -H "Authorization: Bearer $TOKEN_ADMIN" \
http://localhost:8080/api/v1/usuarios

# Eliminar una beca sin solicitudes
curl -i -X DELETE -H "Authorization: Bearer $TOKEN_ADMIN" \
http://localhost:8080/api/v1/becas/4
```


Paso 7.7.07 — Probar errores y seguridad

```
# 1. Endpoint protegido sin token → 401
curl -i http://localhost:8080/api/v1/solicitudes

# 2. Endpoint de admin con rol CIUDADANO → 403
TOKEN_CIUADANO=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"ciudadano","password":"ciudadano123"}' | jq -r '.access_token')

curl -i -X POST -H "Authorization: Bearer $TOKEN_CIUADANO" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/becas \
-d '{"codigo":"OTRA-2026","nombre":"Otra","importeMaximo":1000.00,"anio":2026}'

# 3. Crear una solicitud duplicada → 409
curl -i -X POST -H "Authorization: Bearer $TOKEN_CIUADANO" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/solicitudes \
-d '{"alumnoId":1,"becaId":1}'

# 4. Cambiar el estado de una solicitud aprobada → 409
curl -i -X PATCH -H "Authorization: Bearer $TOKEN_GESTOR" \
-H "Content-Type: application/json" \
http://localhost:8080/api/v1/solicitudes/2/estado \
-d '{"estado":"EN_REVISION"}'
```

Observa los códigos de estado:

- Sin token: 401.
- Con rol incorrecto: 403.
- Duplicado: 409.
- Transición no permitida: 409.

Paso 7.7.08 — Revisar la documentación de OpenAPI

Para que Swagger UI pueda autenticar peticiones protegidas con JWT, crea primero `OpenApiConfig`:

```
package es.mecd.demo.miproyecto.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.PathItem;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import java.util.List;
import org.springdoc.core.customizers.OpenApiCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {
    public static final String BEARER_AUTH = "bearerAuth";

    @Bean
    public OpenAPI apiBecasOpenAPI() {
        SecurityScheme bearer = new SecurityScheme()
            .type(SecurityScheme.Type.HTTP)
            .scheme("bearer")
            .bearerFormat("JWT");
        return new OpenAPI()
            .components(new Components().addSecuritySchemes(BEARER_AUTH, bearer))
            .addSecurityItem(new SecurityRequirement().addList(BEARER_AUTH));
    }

    @Bean
    public OpenApiCustomizer publicEndpointsWithoutSecurity() {
        return openApi -> {
            clearPostSecurity(openApi, "/api/v1/auth/registro");
            clearPostSecurity(openApi, "/api/v1/auth/login");
            clearPostSecurity(openApi, "/api/v1/auth/refresh");
            clearGetSecurity(openApi, "/api/v1/becas");
            clearGetSecurity(openApi, "/api/v1/becas/{id}");
        };
    }
}
```

```

    }

    private void clearPostSecurity(OpenAPI openApi, String path) {
        PathItem item = openApi.getPaths() == null ? null : openApi.getPaths().get(path);
        if (item != null && item.getPost() != null) item.getPost().setSecurity(List.of());
    }

    private void clearGetSecurity(OpenAPI openApi, String path) {
        PathItem item = openApi.getPaths() == null ? null : openApi.getPaths().get(path);
        if (item != null && item.getGet() != null) item.getGet().setSecurity(List.of());
    }
}

```

Arranca la aplicación y abre `http://localhost:8080/swagger-ui.html`. Verifica que aparecen los controladores reales de M7 y que el botón **Authorize** utiliza `bearerAuth`.

Prueba el flujo:

1. Ejecuta `POST /api/v1/auth/login`.
2. Copia `access_token`.
3. Pulsa **Authorize**.
4. Introduce el token como Bearer JWT.
5. Ejecuta un endpoint protegido, por ejemplo `GET /api/v1/solicitudes`.



Paso 7.7.09 — Preparar la colección de Postman

Abre Postman y crea una colección llamada “API Becas - Ministerio”. Dentro, crea las siguientes carpetas:

- Auth: registro, login, refresh.
- Becas: listar, consultar, crear, actualizar, eliminar.
- Solicitudes: listar, consultar, crear, cambiar estado, cancelar.
- Documentos: listar, añadir, eliminar.
- Usuarios: listar, consultar, cambiar roles. Para cada carpeta, añade las peticiones correspondientes. Configura un entorno con la variable `baseUrl = http://localhost:8080` y `token` (que se rellena tras el login).

Añade un test a la petición de login para que guarde el token automáticamente:

```

pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

const jsonData = pm.response.json();
pm.environment.set("token", jsonData.access_token);

```

Y añade un test a una petición protegida para verificar que el token funciona:

```

pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

```

Exporta la colección y el entorno dentro de `M7/proyecto/postman/` con estos nombres exactos:

```

API_Becas_Ministerio.postman_collection.json
API_Becas_Local.postman_environment.json

```

No incluyas peticiones a endpoints inexistentes. ### Paso 7.7.10 — Prueba de carga ligera con ab

Instala ab (Apache Benchmark), que viene con el paquete `apache2-utils`:

```

# En Debian/Ubuntu
sudo apt install apache2-utils

```

```
# En macOS (ya viene instalado por defecto)
which ab
```

Ejecuta una prueba ligera:

```
ab -n 100 -c 10 http://localhost:8080/api/v1/becas

Analiza los resultados. Busca:
- Requests per second: cuántas peticiones por segundo soporta.
- Time per request: tiempo medio por petición.
- Failed requests: peticiones fallidas (debería ser 0).
- Percentiles: p50, p90, p99.
```

Nota: ab solo puede hacer peticiones GET sencillas. Para pruebas con tokens o con cuerpos JSON, se necesitan herramientas como hey (que requiere Go) o wrk. Para la verificación básica de que la API no se cae bajo una carga moderada, ab es suficiente.

Paso 7.7.11 — Actualizar el README

Actualiza README.md para que cualquier persona pueda arrancar, probar y comprender el proyecto sin consultar material adicional. Usa como base una estructura como esta:

```
# API de Gestión de Becas - Ministerio de Educación

API REST para la gestión de solicitudes de becas educativas.

## Requisitos previos

- Java 17
- El Maven Wrapper incluido (`mvnw` / `mvnw.cmd`)

## Compilación y ejecución

```bash
./mvnw clean package
java -jar target/api-becas-0.0.1-SNAPSHOT.jar
```

También puedes arrancar con Maven:

```bash
./mvnw spring-boot:run
```

## Tests

```bash
./mvnw test
```

## Documentación de la API

Swagger UI: `http://localhost:8080/swagger-ui.html`

Pulsa **Authorize** e introduce el JWT de acceso obtenido en `/api/v1/auth/login` para probar los endpoints protegidos.

## Usuarios de prueba (perfil dev)



Usuario	Contraseña	Rol
admin	admin123	ADMIN
gestor	gestor123	GESTOR
ciudadano	ciudadano123	CIUDADANO
user	user12345	USER
consultor	consultor123	CONSULTOR



## Endpoints principales

### Autenticación



Método	URL	Descripción
POST	/api/v1/auth/registro	Registrar usuario


```

```

| POST | `/api/v1/auth/login` | Iniciar sesión |
| POST | `/api/v1/auth/refresh` | Renovar token |

### Becas

Método	URL	Rol
GET	`/api/v1/becas`	Público
GET	`/api/v1/becas/{id}`	Público
POST	`/api/v1/becas`	ADMIN
PUT	`/api/v1/becas/{id}`	ADMIN
DELETE	`/api/v1/becas/{id}`	ADMIN

### Solicitudes

Método	URL	Rol
GET	`/api/v1/solicitudes`	Autenticado
GET	`/api/v1/solicitudes/{id}`	Autenticado
POST	`/api/v1/solicitudes`	CIUDADANO
PATCH	`/api/v1/solicitudes/{id}/estado`	GESTOR, ADMIN
DELETE	`/api/v1/solicitudes/{id}`	CIUDADANO

### Documentos

Método	URL	Rol
GET	`/api/v1/solicitudes/{id}/documentos`	Autenticado
POST	`/api/v1/solicitudes/{id}/documentos`	CIUDADANO
DELETE	`/api/v1/solicitudes/{id}/documentos/{docId}`	CIUDADANO

### Usuarios

Método	URL	Rol
GET	`/api/v1/usuarios`	ADMIN
GET	`/api/v1/usuarios/{id}`	ADMIN
PATCH	`/api/v1/usuarios/{id}/roles`	ADMIN

### Estadísticas

Método	URL	Rol
GET	`/api/v1/estadisticas/solicitudes-por-estado`	GESTOR, ADMIN

## Estructura del proyecto

```text
src/main/java/es/meecd/demo/miproyecto/
├── auth/ → Autenticación, JWT, usuarios y roles
├── alumno/ → Gestión de alumnos
├── beca/ → Gestión de becas
├── solicitud/ → Gestión de solicitudes y estadísticas
├── documento/ → Gestión de documentos
├── config/ → Seguridad, OpenAPI y datos iniciales
├── common/ → DTOs de error y excepciones comunes
└──

```

Revisa el README contra los endpoints reales antes de cerrar el módulo. No documentes rutas que el proyecto no implemente.



## Paso 7.7.12 — Errores comunes antes de la entrega

Error	Causa	Solución
Tests fallan	Alguna regla de negocio mal implementada	Revisar los tests
Cobertura baja	Faltan tests en algún módulo	Añadirlos
Swagger no muestra endpoints	Falta la dependencia o configuración	Verificar <code>pom.xml</code>
La colección de Postman no funciona	Falta el entorno o el token	Configurarlos
La prueba de carga da errores	La API no soporta la concurrencia	Revisar el pool de conexiones
El README está desactualizado	No se ha actualizado	Actualizarlo

Error	Causa	Solución
El JAR no arranca	Falta alguna dependencia en runtime	Revisar el <code>pom.xml</code>
Los usuarios de prueba no existen	El <code>CommandLineRunner</code> no se ejecuta	Verificar el perfil
La base de datos no se crea	<code>ddl-auto</code> mal configurado	Verificar la configuración
La documentación no coincide con la API	Los endpoints han cambiado	Actualizar las anotaciones

### Paso 7.7.13 — Reto resuelto — Añadir un endpoint de estadísticas

**Reto.** Añade `GET /api/v1/estadisticas/solicitudes-por-estado` para devolver el número de solicitudes agrupadas por estado. El endpoint sólo será accesible para `GESTOR` y `ADMIN`.

#### Paso 1 — Añadir la operación al servicio.

```
@Transactional(readOnly = true)
public Map<String, Long> contarPorEstado() {
 return solicitudRepository.contarPorEstado().stream()
 .collect(Collectors.toMap(
 fila -> ((EstadoSolicitud) fila[0]).name(),
 fila -> (Long) fila[1]
));
}
```

Añade los imports necesarios:

```
import java.util.Map;
import java.util.stream.Collectors;
```

#### Paso 2 — Crear EstadisticasController.

```
package es.mecd.demo.miproyecto.solicitud;

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/v1/estadisticas")
public class EstadisticasController {

 private final SolicitudService solicitudService;

 public EstadisticasController(SolicitudService solicitudService) {
 this.solicitudService = solicitudService;
 }

 @GetMapping("/solicitudes-por-estado")
 @PreAuthorize("hasAnyRole('GESTOR', 'ADMIN')")
 public Map<String, Long> solicitudesPorEstado() {
 return solicitudService.contarPorEstado();
 }
}
```

**Completar la matriz de seguridad del endpoint.** Mantén `@PreAuthorize` como defensa en profundidad y añade también la regla HTTP al `SecurityConfig`, antes de la regla general de solicitudes. De este modo un usuario `CONSULTOR` recibe `403 Forbidden` en la propia cadena de seguridad y el endpoint queda reservado a `GESTOR` y `ADMIN` de forma coherente con el resto de la API.

```
.requestMatchers(HttpMethod.GET, "/api/v1/estadisticas/**")
 .hasAnyRole("GESTOR", "ADMIN")
```

### Paso 3 — Probar el endpoint.

```
curl -H "Authorization: Bearer $TOKEN_GESTOR" \
http://localhost:8080/api/v1/estadisticas/solicitudes-por-estado
```

Una respuesta válida tendrá esta forma:

```
{
 "BORRADOR": 5,
 "ENVIADA": 3,
 "EN REVISION": 1,
 "APROBADA": 2,
 "DENEGADA": 1,
 "CANCELADA": 0
}
```

### ✓ Resultado esperado global

Al final del ejercicio, deberías tener:

- Todos los tests pasando (BUILD SUCCESS).
- Cobertura de tests superior al 70%.
- La aplicación arrancando sin errores.
- Todos los escenarios de prueba pasando (ciudadano, gestor, admin, errores).
- Swagger UI mostrando todos los endpoints.
- Una colección de Postman lista para entregar.
- Un README completo.
- Un informe de prueba de carga con ab.
- (Opcional) Un endpoint de estadísticas.

### 🧩 Resumen técnico del ejercicio

El ejercicio ha demostrado:

- Verificación manual: con curl y Postman.
- Tests automatizados: ./mvnw test con JaCoCo.
- Documentación: OpenAPI en Swagger UI.
- Prueba de carga: con ab (Apache Benchmark).
- Documentación del proyecto: README completo.
- Criterios de aceptación: funcionalidad, seguridad, tests, documentación, rendimiento.



## Conclusión del Módulo 7 y del curso

En este punto 7.7 hemos realizado las pruebas finales del API:

- Verificación funcional: todos los escenarios probados.
- Verificación de seguridad: 401, 403 y 200 según el caso.
- Tests automatizados: ejecutados y analizados.
- Documentación: OpenAPI y README.
- Prueba de carga: ligera, con ab, para verificar que la API aguanta.
- Entrega: código, README, colección de Postman, documentación.

## Anexo A — Código exacto del PREP autónomo

Este anexo forma parte de la práctica. Permite reproducir exactamente PREP-01...PREP-08 sin depender de módulos anteriores. Los ficheros `mvnw`, `mvnw.cmd`, `.mvn/wrapper/maven-wrapper.properties` y `.gitignore` los genera Spring Initializr en PREP-01; no se copian manualmente.

Los listados siguientes fijan el estado **antes de 7.1.01**. Las modificaciones posteriores se realizan únicamente en los pasos 7.x indicados en el cuerpo principal.

`pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <parent>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-parent</artifactId>
 <version>3.5.16</version>
 <relativePath/>
 </parent>
 <groupId>es.mecd.demo</groupId>
 <artifactId>api-becas</artifactId>
 <version>0.0.1-SNAPSHOT</version>
 <name>api-becas</name>
 <description>M7 - API REST autónoma de gestión de becas</description>
 <properties>
 <java.version>17</java.version>
 <springdoc.version>2.8.13</springdoc.version>
 <jjwt.version>0.13.0</jjwt.version>
 </properties>
 <dependencies>
 <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</
artifactId></dependency>
 <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</
artifactId></dependency>
 <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</
artifactId></dependency>
 <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</
artifactId></dependency>
 <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></
dependency>
 <dependency><groupId>org.springdoc</groupId><artifactId>springdoc-openapi-starter-webmvc-ui</
artifactId><version>${springdoc.version}</version></dependency>
 <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>${
jjwt.version}</version></dependency>
 <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>${
jjwt.version}</version><scope>runtime</scope></dependency>
```

```

 <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>${jjwt.version}</version><scope>runtime</scope></dependency>
 <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
 <dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</artifactId><scope>test</scope></dependency>
 </dependencies>
 <build>
 <plugins>
 <plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId></plugin>
 </plugins>
 </build>
</project>

```

#### src/main/java/es/mecd/demo/miproyecto/MiProyectoApplication.java

```

package es.mecd.demo.miproyecto;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MiProyectoApplication {
 public static void main(String[] args) {
 SpringApplication.run(MiProyectoApplication.class, args);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/alumno/Alumno.java

```

package es.mecd.demo.miproyecto.alumno;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.LocalDate;

@Entity
@Table(name = "alumnos")
public class Alumno {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 @Column(nullable = false, length = 100)
 private String nombre;
 @Column(nullable = false, length = 150)
 private String apellidos;
 @Column(nullable = false, unique = true, length = 9)
 private String dni;
 @Column(name = "fecha_nacimiento", nullable = false)
 private LocalDate fechaNacimiento;
 @ManyToOne(fetch = FetchType.LAZY, optional = false)
 @JoinColumn(name = "curso_id", nullable = false)
 private Curso curso;

 protected Alumno() { }
 public Alumno(String nombre, String apellidos, String dni, LocalDate fechaNacimiento, Curso curso) {
 this.nombre = nombre;
 this.apellidos = apellidos;
 this.dni = dni;
 this.fechaNacimiento = fechaNacimiento;
 this.curso = curso;
 }
 public Long getId() { return id; }
 public String getNombre() { return nombre; }
 public void setNombre(String nombre) { this.nombre = nombre; }
 public String getApellidos() { return apellidos; }
}

```



```

 public void setApellidos(String apellidos) { this.apellidos = apellidos; }
 public String getDni() { return dni; }
 public void setDni(String dni) { this.dni = dni; }
 public LocalDate getFechaNacimiento() { return fechaNacimiento; }
 public void setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento = fechaNacimiento; }
 public Curso getCurso() { return curso; }
 public void setCurso(Curso curso) { this.curso = curso; }
}

```

#### src/main/java/es/mecd/demo/miproyecto/alumno/AlumnoController.java

```

package es.mecd.demo.miproyecto.alumno;

import jakarta.validation.Valid;
import java.net.URI;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {
 private final AlumnoService alumnoService;

 public AlumnoController(AlumnoService alumnoService) {
 this.alumnoService = alumnoService;
 }

 @GetMapping
 public Page<AlumnoResponseDTO> listar(
 @RequestParam(required = false) String curso,
 @PageableDefault(size = 20, sort = "id") Pageable pageable) {
 return alumnoService.listar(curso, pageable);
 }

 @GetMapping("/{id}")
 public AlumnoResponseDTO consultar(@PathVariable Long id) {
 return alumnoService.consultar(id);
 }

 @PostMapping
 public ResponseEntity<AlumnoResponseDTO> crear(@Valid @RequestBody AlumnoRequestDTO request) {
 AlumnoResponseDTO creado = alumnoService.crear(request);
 URI location = ServletUriComponentsBuilder.fromCurrentRequest().path("/{id}")
 .buildAndExpand(creado.getIdentificador()).toUri();
 return ResponseEntity.created(location).body(creado);
 }

 @PutMapping("/{id}")
 public AlumnoResponseDTO actualizar(@PathVariable Long id, @Valid @RequestBody AlumnoRequestDTO
request) {
 return alumnoService.actualizar(id, request);
 }

 @DeleteMapping("/{id}")
 public ResponseEntity<Void> eliminar(@PathVariable Long id) {
 alumnoService.eliminar(id);
 return ResponseEntity.noContent().build();
 }
}

```

**src/main/java/es/mecd/demo/miproyecto/alumno/AlumnoRepository.java**

```
package es.mecd.demo.miproyecto.alumno;

import java.util.Optional;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface AlumnoRepository extends JpaRepository<Alumno, Long> {
 boolean existsByDni(String dni);
 Optional<Alumno> findByDni(String dni);

 @Query("SELECT a FROM Alumno a JOIN a.curso c "
 + "WHERE (:curso IS NULL OR LOWER(c.nombre) = LOWER(:curso))")
 Page<Alumno> buscar(@Param("curso") String curso, Pageable pageable);
}
```

**src/main/java/es/mecd/demo/miproyecto/alumno/AlumnoRequestDTO.java**

```
package es.mecd.demo.miproyecto.alumno;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.time.LocalDate;

public class AlumnoRequestDTO {
 @NotBlank @Size(max = 100) private String nombre;
 @NotBlank @Size(max = 150) private String apellidos;
 @NotBlank @Size(min = 9, max = 9) private String dni;
 @NotNull private LocalDate fechaNacimiento;
 @NotBlank private String curso;
 public String getNombre() { return nombre; }
 public void setNombre(String nombre) { this.nombre = nombre; }
 public String getApellidos() { return apellidos; }
 public void setApellidos(String apellidos) { this.apellidos = apellidos; }
 public String getDni() { return dni; }
 public void setDni(String dni) { this.dni = dni; }
 public LocalDate getFechaNacimiento() { return fechaNacimiento; }
 public void setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento = fechaNacimiento; }
 public String getCurso() { return curso; }
 public void setCurso(String curso) { this.curso = curso; }
}
```

**src/main/java/es/mecd/demo/miproyecto/alumno/AlumnoResponseDTO.java**

```
package es.mecd.demo.miproyecto.alumno;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.LocalDate;

public class AlumnoResponseDTO {
 @JsonProperty("id") private String identificador;
 private String nombre;
 private String apellidos;
 private String dni;
 private LocalDate fechaNacimiento;
 private String curso;
 private int edad;
 public String getIdentificador() { return identificador; }
 public void setIdentificador(String identificador) { this.identificador = identificador; }
 public String getNombre() { return nombre; }
 public void setNombre(String nombre) { this.nombre = nombre; }
 public String getApellidos() { return apellidos; }
 public void setApellidos(String apellidos) { this.apellidos = apellidos; }
 public String getDni() { return dni; }
 public void setDni(String dni) { this.dni = dni; }
 public LocalDate getFechaNacimiento() { return fechaNacimiento; }
 public void setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento = fechaNacimiento; }
}
```

```

 public String getCurso() { return curso; }
 public void setCurso(String curso) { this.curso = curso; }
 public int getEdad() { return edad; }
 public void setEdad(int edad) { this.edad = edad; }
}

```

#### src/main/java/es/mecd/demo/miproyecto/alumno/AlumnoService.java

```

package es.mecd.demo.miproyecto.alumno;

import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import java.time.LocalDate;
import java.time.Period;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AlumnoService {
 private final AlumnoRepository alumnoRepository;
 private final CursoRepository cursoRepository;
 public AlumnoService(AlumnoRepository alumnoRepository, CursoRepository cursoRepository) {
 this.alumnoRepository = alumnoRepository;
 this.cursoRepository = cursoRepository;
 }
 @Transactional(readOnly = true)
 public Page<AlumnoResponseDTO> listar(String curso, Pageable pageable) {
 return alumnoRepository.buscar(curso, pageable).map(this::toDTO);
 }
 @Transactional(readOnly = true)
 public AlumnoResponseDTO consultar(Long id) {
 return alumnoRepository.findById(id).map(this::toDTO)
 .orElseThrow(() -> new RecursoNoEncontradoException("Alumno", id));
 }
 @Transactional
 public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
 if (alumnoRepository.existsByDni(request.getDni())) {
 throw new RecursoDuplicadoException("Alumno", "dni", request.getDni());
 }
 Curso curso = cursoRepository.findByNombre(request.getCurso())
 .orElseGet(() -> cursoRepository.save(new Curso(request.getCurso())));
 return toDTO(alumnoRepository.save(new Alumno(
 request.getNombre(), request.getApellidos(), request.getDni(),
 request.getFechaNacimiento(), curso)));
 }
 @Transactional
 public AlumnoResponseDTO actualizar(Long id, AlumnoRequestDTO request) {
 Alumno alumno = alumnoRepository.findById(id)
 .orElseThrow(() -> new RecursoNoEncontradoException("Alumno", id));
 Curso curso = cursoRepository.findByNombre(request.getCurso())
 .orElseGet(() -> cursoRepository.save(new Curso(request.getCurso())));
 alumno.setNombre(request.getNombre());
 alumno.setApellidos(request.getApellidos());
 alumno.setDni(request.getDni());
 alumno.setFechaNacimiento(request.getFechaNacimiento());
 alumno.setCurso(curso);
 return toDTO(alumnoRepository.save(alumno));
 }
 @Transactional
 public void eliminar(Long id) {
 Alumno alumno = alumnoRepository.findById(id)
 .orElseThrow(() -> new RecursoNoEncontradoException("Alumno", id));
 alumnoRepository.delete(alumno);
 }
 private AlumnoResponseDTO toDTO(Alumno alumno) {
 AlumnoResponseDTO dto = new AlumnoResponseDTO();
 dto.setIdentificador(alumno.getId() == null ? null : alumno.getId().toString());
 dto.setNombre(alumno.getNombre());
 dto.setApellidos(alumno.getApellidos());
 dto.setDni(alumno.getDni());
 dto.setFechaNacimiento(alumno.getFechaNacimiento());
 dto.setCurso(alumno.getCurso() == null ? null : alumno.getCurso().getNombre());
 dto.setEdad(Period.between(alumno.getFechaNacimiento(), LocalDate.now()).getYears());
 return dto;
 }
}

```

```
}
}
```

#### src/main/java/es/mecd/demo/miproyecto/alumno/Curso.java

```
package es.mecd.demo.miproyecto.alumno;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "cursos")
public class Curso {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 @Column(nullable = false, unique = true, length = 80)
 private String nombre;

 protected Curso() { }
 public Curso(String nombre) { this.nombre = nombre; }
 public Long getId() { return id; }
 public String getNombre() { return nombre; }
 public void setNombre(String nombre) { this.nombre = nombre; }
}
```

#### src/main/java/es/mecd/demo/miproyecto/alumno/CursoRepository.java

```
package es.mecd.demo.miproyecto.alumno;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CursoRepository extends JpaRepository<Curso, Long> {
 Optional<Curso> findByNombre(String nombre);
}
```

#### src/main/java/es/mecd/demo/miproyecto/auth/AuthController.java

```
package es.mecd.demo.miproyecto.auth;

import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {
 private final AuthService authService;
 public AuthController(AuthService authService) { this.authService = authService; }
 @PostMapping("/registro")
 public ResponseEntity<RegistroResponseDTO> registrar(@Valid @RequestBody RegistroRequestDTO request) {
 return ResponseEntity.status(HttpStatus.CREATED).body(authService.registrar(request));
 }
 @PostMapping("/login")
 public LoginResponseDTO login(@Valid @RequestBody LoginRequestDTO request) { return
authService.login(request); }
 @PostMapping("/refresh")
 public LoginResponseDTO refresh(@Valid @RequestBody RefreshRequestDTO request) {
 return authService.refresh(request.getRefreshToken());
 }
}
```

```
}
}
```

#### src/main/java/es/mecd/demo/miproyecto/auth/AuthExceptionHandler.java

```
package es.mecd.demo.miproyecto.auth;
import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import java.time.Instant;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
@RestControllerAdvice(assignableTypes = AuthController.class)
@Order(1)
public class AuthExceptionHandler {
 @ExceptionHandler({CredencialesInvalidasException.class, TokenInvalidoException.class})
 ResponseEntity<ErrorResponse> auth(RuntimeException ex, HttpServletRequest request) {
 ErrorResponse error = new ErrorResponse(); error.setTimestamp(Instant.now().toString());
 error.setStatus(401); error.setCodigo("AUTH_ERROR"); error.setMensaje(ex.getMessage());
 error.setPath(request.getRequestURI()); return
 ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
 }
}
```

#### src/main/java/es/mecd/demo/miproyecto/auth/AuthService.java

```
package es.mecd.demo.miproyecto.auth;

import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuthService {
 private final AuthenticationManager authenticationManager;
 private final JwtService jwtService;
 private final UsuarioRepository usuarioRepository;
 private final RolRepository rolRepository;
 private final PasswordEncoder passwordEncoder;
 public AuthService(AuthenticationManager authenticationManager, JwtService jwtService,
 UsuarioRepository usuarioRepository, RolRepository rolRepository, PasswordEncoder
passwordEncoder) {
 this.authenticationManager = authenticationManager; this.jwtService = jwtService;
 this.usuarioRepository = usuarioRepository; this.rolRepository = rolRepository;
 this.passwordEncoder = passwordEncoder;
 }
 @Transactional
 public RegistroResponseDTO registrar(RegistroRequestDTO request) {
 if (usuarioRepository.existsByUsername(request.getUsername())) {
 throw new RecursoDuplicadoException("Usuario", "username", request.getUsername());
 }
 if (usuarioRepository.existsByEmail(request.getEmail())) {
 throw new RecursoDuplicadoException("Usuario", "email", request.getEmail());
 }
 Rol rol = rolRepository.findByNombre("USER")
 .orElseThrow(() -> new RecursoNoEncontradoException("Rol", "USER"));
 Usuario u = new Usuario(); u.setUsername(request.getUsername());
 u.setPassword(passwordEncoder.encode(request.getPassword())); u.setEmail(request.getEmail());
 u.getRoles().add(rol); u = usuarioRepository.save(u);
 RegistroResponseDTO dto = new RegistroResponseDTO(); dto.setId(u.getId().toString());
 dto.setUsername(u.getUsername()); dto.setEmail(u.getEmail());
 dto.setRoles(u.getRoles().stream().map(Rol::getNombre).sorted().toList()); return dto;
 }
 @Transactional(readOnly = true)
 public LoginResponseDTO login(LoginRequestDTO request) {
 try {
 authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(
```

```

 request.getUsername(), request.getPassword()));
 } catch (AuthenticationException ex) { throw new CredencialesInvalidasException(); }
 Usuario u = usuarioRepository.findByUsername(request.getUsername())
 .orElseThrow(CredencialesInvalidasException::new);
 return tokens(u);
}
@Transactional(readOnly = true)
public LoginResponseDTO refresh(String refreshToken) {
 if (!jwtService.esRefreshValido(refreshToken)) throw new TokenInvalidoException("Refresh token
 inválido");
 Usuario u = usuarioRepository.findByUsername(jwtService.username(refreshToken))
 .orElseThrow(() -> new RecursoNoEncontradoException("Usuario",
 jwtService.username(refreshToken)));
 return tokens(u);
}
private LoginResponseDTO tokens(Usuario u) {
 return new LoginResponseDTO(jwtService.generarAccessToken(u), jwtService.generarRefreshToken(u),
 jwtService.getExpirationMs() / 1000);
}
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/CredencialesInvalidasException.java

```

package es.mecd.demo.miproyecto.auth;
public class CredencialesInvalidasException extends RuntimeException {
 public CredencialesInvalidasException() { super("Credenciales inválidas"); }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/JwtAccessDeniedHandler.java

```

package es.mecd.demo.miproyecto.auth;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;
@Component
public class JwtAccessDeniedHandler implements AccessDeniedHandler {
 private final SecurityErrorWriter writer;
 public JwtAccessDeniedHandler(SecurityErrorWriter writer) { this.writer = writer; }
 @Override
 public void handle(HttpServletRequest request, HttpServletResponse response, AccessDeniedException ex)
 throws IOException { writer.write(request, response, 403, "ACCESO_DENEGADO", "No tiene
 permisos suficientes"); }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/JwtAuthenticationEntryPoint.java

```

package es.mecd.demo.miproyecto.auth;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;
@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {
 private final SecurityErrorWriter writer;
 public JwtAuthenticationEntryPoint(SecurityErrorWriter writer) { this.writer = writer; }
 @Override
 public void commence(HttpServletRequest request, HttpServletResponse response, AuthenticationException
 ex)
 throws IOException { writer.write(request, response, 401, "NO_AUTENTICADO", "Autenticación
 requerida"); }
}

```

**src/main/java/es/mecd/demo/miproyecto/auth/JwtAuthenticationFilter.java**

```

package es.mecd.demo.miproyecto.auth;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
 private final JwtService jwtService;

 public JwtAuthenticationFilter(JwtService jwtService) { this.jwtService = jwtService; }

 @Override
 protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
 throws ServletException, IOException {
 String header = request.getHeader("Authorization");
 if (header != null && header.startsWith("Bearer ")) {
 String token = header.substring(7);
 if (jwtService.esAccessValido(token) && SecurityContextHolder.getContext().getAuthentication() == null) {
 var auth = new UsernamePasswordAuthenticationToken(
 jwtService.username(token), null, jwtService.authorities(token));
 auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
 SecurityContextHolder.getContext().setAuthentication(auth);
 }
 chain.doFilter(request, response);
 }
 }
}

```

**src/main/java/es/mecd/demo/miproyecto/auth/JwtConfig.java**

```

package es.mecd.demo.miproyecto.auth;

import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import javax.crypto.SecretKey;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JwtConfig {
 @Bean
 SecretKey jwtSecretKey(@Value("${jwt.secret}") String secret) {
 return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
 }
}

```

**src/main/java/es/mecd/demo/miproyecto/auth/JwtService.java**

```

package es.mecd.demo.miproyecto.auth;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import java.util.Date;
import java.util.List;
import javax.crypto.SecretKey;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.stereotype.Service;

@Service
public class JwtService {

```



```

private static final String CLAIM_TIPO = "tipo";
private final SecretKey clave;
private final long expirationMs;
private final long refreshExpirationMs;
public JwtService(SecretKey clave, @Value("${jwt.expiration}") long expirationMs,
 @Value("${jwt.refresh-expiration}") long refreshExpirationMs) {
 this.clave = clave; this.expirationMs = expirationMs; this.refreshExpirationMs =
refreshExpirationMs;
}
public String generarAccessToken(Usuario usuario) { return generar(usuario, "access", expirationMs,
true); }
public String generarRefreshToken(Usuario usuario) { return generar(usuario, "refresh",
refreshExpirationMs, false); }
private String generar(Usuario usuario, String tipo, long duracion, boolean incluirRoles) {
 Date ahora = new Date();
 var builder = Jwts.builder().subject(usuario.getUsername()).claim(CLAIM_TIPO, tipo)
 .issuedAt(ahora).expiration(new Date(ahora.getTime() + duracion));
 if (incluirRoles) {
 builder.claim("roles", usuario.getRoles().stream().map(r -> "ROLE_" +
r.getNombre()).sorted().toList());
 }
 return builder.signWith(clave, Jwts.SIG.HS256).compact();
}
public Claims extraerClaims(String token) {
 return Jwts.parser().verifyWith(clave).build().parseSignedClaims(token).getPayload();
}
public boolean esAccessValido(String token) { return esTipoValido(token, "access"); }
public boolean esRefreshValido(String token) { return esTipoValido(token, "refresh"); }
private boolean esTipoValido(String token, String tipo) {
 try { return tipo.equals(extraerClaims(token).get(CLAIM_TIPO, String.class)); }
 catch (JwtException | IllegalArgumentException ex) { return false; }
}
public String username(String token) { return extraerClaims(token).getSubject(); }
public List<GrantedAuthority> authorities(String token) {
 Object raw = extraerClaims(token).get("roles");
 if (!(raw instanceof List<?> lista)) return List.of();
 return lista.stream().filter(String.class::isInstance).map(String.class::cast)
 .map(SimpleGrantedAuthority::new).map(GrantedAuthority.class::cast).toList();
}
public long getExpirationMs() { return expirationMs; }
}

```

#### src/main/java/es/meecd/demo/miproyecto/auth/LoginRequestDTO.java

```

package es.meecd.demo.miproyecto.auth;
import jakarta.validation.constraints.NotBlank;
public class LoginRequestDTO {
 @NotBlank private String username;
 @NotBlank private String password;
 public String getUsername() { return username; }
 public void setUsername(String username) { this.username = username; }
 public String getPassword() { return password; }
 public void setPassword(String password) { this.password = password; }
}

```

#### src/main/java/es/meecd/demo/miproyecto/auth/LoginResponseDTO.java

```

package es.meecd.demo.miproyecto.auth;
import com.fasterxml.jackson.annotation.JsonProperty;
public class LoginResponseDTO {
 @JsonProperty("access_token") private final String accessToken;
 @JsonProperty("refresh_token") private final String refreshToken;
 @JsonProperty("token_type") private final String tokenType = "Bearer";
 @JsonProperty("expires_in") private final long expiresIn;
 public LoginResponseDTO(String accessToken, String refreshToken, long expiresIn) {
 this.accessToken = accessToken; this.refreshToken = refreshToken; this.expiresIn = expiresIn;
 }
 public String getAccessToken() { return accessToken; }
 public String getRefreshToken() { return refreshToken; }
 public String getTokenType() { return tokenType; }
 public long getExpiresIn() { return expiresIn; }
}

```



**src/main/java/es/mecd/demo/miproyecto/auth/RefreshRequestDTO.java**

```
package es.mecd.demo.miproyecto.auth;
import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.NotBlank;
public class RefreshRequestDTO {
 @NotBlank @JsonProperty("refresh_token") private String refreshToken;
 public String getRefreshToken() { return refreshToken; }
 public void setRefreshToken(String refreshToken) { this.refreshToken = refreshToken; }
}
```

**src/main/java/es/mecd/demo/miproyecto/auth/RegistroRequestDTO.java**

```
package es.mecd.demo.miproyecto.auth;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
public class RegistroRequestDTO {
 @NotBlank @Size(min = 3, max = 50) private String username;
 @NotBlank @Size(min = 8, max = 100) private String password;
 @NotBlank @Email private String email;
 public String getUsername() { return username; }
 public void setUsername(String username) { this.username = username; }
 public String getPassword() { return password; }
 public void setPassword(String password) { this.password = password; }
 public String getEmail() { return email; }
 public void setEmail(String email) { this.email = email; }
}
```

**src/main/java/es/mecd/demo/miproyecto/auth/RegistroResponseDTO.java**

```
package es.mecd.demo.miproyecto.auth;
import java.util.List;
public class RegistroResponseDTO {
 private String id; private String username; private String email; private List<String> roles;
 public String getId() { return id; } public void setId(String id) { this.id = id; }
 public String getUsername() { return username; } public void setUsername(String username)
 { this.username = username; }
 public String getEmail() { return email; } public void setEmail(String email) { this.email = email; }
 public List<String> getRoles() { return roles; } public void setRoles(List<String> roles) { this.roles
 = roles; }
}
```

**src/main/java/es/mecd/demo/miproyecto/auth/Rol.java**

```
package es.mecd.demo.miproyecto.auth;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToMany;
import jakarta.persistence.Table;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "roles")
public class Rol {
 @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
 @Column(nullable = false, unique = true, length = 50) private String nombre;
 @Column(length = 200) private String descripcion;
 @ManyToMany(mappedBy = "roles") private Set<Usuario> usuarios = new HashSet<>();
 protected Rol() { }
 public Rol(String nombre, String descripcion) { this.nombre = nombre; this.descripcion =
 descripcion; }
 public Long getId() { return id; }
 public String getNombre() { return nombre; }
```

```

 public void setNombre(String nombre) { this.nombre = nombre; }
 public String getDescripcion() { return descripcion; }
 public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
 public Set<Usuario> getUsuarios() { return usuarios; }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/RolRepository.java

```

package es.mecd.demo.miproyecto.auth;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
public interface RolRepository extends JpaRepository<Rol, Long> { Optional<Rol> findByNombre(String
nombre); }

```

#### src/main/java/es/mecd/demo/miproyecto/auth/SecurityErrorWriter.java

```

package es.mecd.demo.miproyecto.auth;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
@Component
public class SecurityErrorWriter {
 private final ObjectMapper objectMapper;
 public SecurityErrorWriter(ObjectMapper objectMapper) { this.objectMapper = objectMapper; }
 public void write(HttpServletRequest request, HttpServletResponse response, int status,
 String codigo, String mensaje) throws IOException {
 response.setStatus(status); response.setContentType(MediaType.APPLICATION_JSON_VALUE);
 Map<String, Object> body = new LinkedHashMap<>(); body.put("timestamp", Instant.now().toString());
 body.put("status", status); body.put("codigo", codigo); body.put("mensaje", mensaje);
 body.put("path", request.getRequestURI()); objectMapper.writeValue(response.getOutputStream(),
body);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/TokenInvalidoException.java

```

package es.mecd.demo.miproyecto.auth;
public class TokenInvalidoException extends RuntimeException {
 public TokenInvalidoException(String mensaje) { super(mensaje); }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/Usuario.java

```

package es.mecd.demo.miproyecto.auth;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.JoinTable;
import jakarta.persistence.ManyToMany;
import jakarta.persistence.Table;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "usuarios")
public class Usuario {

```

```

@Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
@Column(nullable = false, unique = true, length = 50) private String username;
@Column(nullable = false) private String password;
@Column(nullable = false, unique = true, length = 150) private String email;
@Column(nullable = false) private boolean activo = true;
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "usuarios_rols", joinColumns = @JoinColumn(name = "usuario_id"),
 inverseJoinColumns = @JoinColumn(name = "rol_id"))
private Set<Rol> roles = new HashSet<>();
protected Usuario() { }
public Long getId() { return id; }
public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
public String getPassword() { return password; }
public void setPassword(String password) { this.password = password; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public boolean isActive() { return activo; }
public void setActivo(boolean activo) { this.activo = activo; }
public Set<Rol> getRoles() { return roles; }
public void setRoles(Set<Rol> roles) { this.roles = new HashSet<>(roles); }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/UsuarioDetailsService.java

```

package es.mecd.demo.miproyecto.auth;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class UsuarioDetailsService implements UserDetailsService {
 private final UsuarioRepository usuarioRepository;

 public UsuarioDetailsService(UsuarioRepository usuarioRepository) { this.usuarioRepository =
usuarioRepository; }

 @Override
 public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
 Usuario u = usuarioRepository.findByUsername(username)
 .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
 String[] authorities = u.getRoles().stream().map(r -> "ROLE_" +
r.getNombre()).toArray(String[]::new);
 return User.withUsername(u.getUsername()).password(u.getPassword()).authorities(authorities)
 .disabled(!u.isActive()).build();
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/UsuarioRepository.java

```

package es.mecd.demo.miproyecto.auth;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
 Optional<Usuario> findByUsername(String username);
 boolean existsByUsername(String username);
 boolean existsByEmail(String email);
}

```

#### src/main/java/es/mecd/demo/miproyecto/auth/UsuariosInicialesConfig.java

```

package es.mecd.demo.miproyecto.auth;

import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration

```

```

public class UsuariosInicialesConfig {
 @Bean
 @Profile("dev")
 CommandLineRunner cargarUsuarios(UsuarioRepository usuarios, RolRepository roles, PasswordEncoder
encoder) {
 return args -> {
 Rol user = rol(roles, "USER", "Usuario genérico");
 Rol admin = rol(roles, "ADMIN", "Administrador");
 crear(usuarios, encoder, "admin", "admin123", "admin@educacion.gob.es", admin);
 crear(usuarios, encoder, "user", "user12345", "user@educacion.gob.es", user);
 };
 }

 private Rol rol(RolRepository repo, String nombre, String descripcion) {
 return repo.findByNombre(nombre).orElseGet(() -> repo.save(new Rol(nombre, descripcion)));
 }

 private void crear(UsuarioRepository repo, PasswordEncoder encoder, String username,
 String password, String email, Rol... roles) {
 if (repo.existsByUsername(username)) return;
 Usuario u = new Usuario();
 u.setUsername(username);
 u.setPassword(encoder.encode(password));
 u.setEmail(email);
 for (Rol rol : roles) u.getRoles().add(rol);
 repo.save(u);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/dto/error/ErrorResponse.java

```

package es.mecd.demo.miproyecto.common.dto.error;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.util.List;
import java.util.Map;

/** Contrato estable de error HTTP para la API. */
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {
 private String timestamp;
 private int status;
 private String codigo;
 private String mensaje;
 private String path;
 private String traceId;
 private Map<String, Object> detalles;
 private List<ValidationError> errors;

 public String getTimestamp() {
 return timestamp;
 }

 public void setTimestamp(String timestamp) {
 this.timestamp = timestamp;
 }

 public int getStatus() {
 return status;
 }

 public void setStatus(int status) {
 this.status = status;
 }

 public String getCodigo() {
 return codigo;
 }

 public void setCodigo(String codigo) {
 this.codigo = codigo;
 }

 public String getMensaje() {
 return mensaje;
 }

 public void setMensaje(String mensaje) {

```

```

 this.mensaje = mensaje;
 }

 public String getPath() {
 return path;
 }

 public void setPath(String path) {
 this.path = path;
 }

 public String getTraceId() {
 return traceId;
 }

 public void setTraceId(String traceId) {
 this.traceId = traceId;
 }

 public Map<String, Object> getDetalles() {
 return detalles;
 }

 public void setDetalles(Map<String, Object> detalles) {
 this.detalles = detalles;
 }

 public List<ValidationError> getErrors() {
 return errors;
 }

 public void setErrors(List<ValidationError> errors) {
 this.errors = errors;
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/dto/error/ValidationError.java

```

package es.mecd.demo.miproyecto.common.dto.error;

import com.fasterxml.jackson.annotation.JsonInclude;

/** Error de validación asociado a un campo de entrada. */
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ValidationError {
 private String field;
 private Object rejectedValue;
 private String message;

 public ValidationError() {
 }

 public ValidationError(
 String field,
 Object rejectedValue,
 String message) {
 this.field = field;
 this.rejectedValue = rejectedValue;
 this.message = message;
 }

 public String getField() {
 return field;
 }

 public void setField(String field) {
 this.field = field;
 }

 public Object getRejectedValue() {
 return rejectedValue;
 }

 public void setRejectedValue(Object rejectedValue) {
 this.rejectedValue = rejectedValue;
 }

 public String getMessage() {
 }
}

```

```

 return message;
 }

 public void setMessage(String message) {
 this.message = message;
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/exception/AplicacionException.java

```

package es.mecd.demo.miproyecto.common.exception;

/** Base estable para las excepciones funcionales de la aplicación. */
public abstract class AplicacionException extends RuntimeException {
 private final String codigo;

 protected AplicacionException(String codigo, String mensaje) {
 super(mensaje);
 this.codigo = codigo;
 }

 protected AplicacionException(
 String codigo,
 String mensaje,
 Throwable causa) {
 super(mensaje, causa);
 this.codigo = codigo;
 }

 public String getCodigo() {
 return codigo;
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/exception/BusinessExceptionHandler.java

```

package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import java.util.Map;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/** Traduce excepciones funcionales a códigos HTTP estables. */
@RestControllerAdvice
@Order(2)
public class BusinessExceptionHandler {
 @ExceptionHandler(RecursoNoEncontradoException.class)
 public ResponseEntity<ErrorResponse> handleNoEncontrado(
 RecursoNoEncontradoException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.NOT_FOUND,
 ex.getCodigo(),
 ex.getMessage(),
 request,
 Map.of("recurso", ex.getRecurso(), "id", ex.getId()));
 }

 @ExceptionHandler(RecursoDuplicadoException.class)
 public ResponseEntity<ErrorResponse> handleDuplicado(
 RecursoDuplicadoException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.CONFLICT,
 ex.getCodigo(),
 ex.getMessage(),
 request,
 Map.of(
 "recurso", ex.getRecurso(),

```

```

 "campo", ex.getCampo(),
 "valor", ex.getValor()));
 }

 @ExceptionHandler(OperacionNoPermitidaException.class)
 public ResponseEntity<ErrorResponse> handleOperacionNoPermitida(
 OperacionNoPermitidaException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.CONFLICT,
 ex.getCodigo(),
 ex.getMessage(),
 request,
 null);
 }

 @ExceptionHandler(ValidacionNegocioException.class)
 public ResponseEntity<ErrorResponse> handleValidacionNegocio(
 ValidacionNegocioException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.UNPROCESSABLE_ENTITY,
 ex.getCodigo(),
 ex.getMessage(),
 request,
 Map.of("campo", ex.getCampo(), "motivo", ex.getMotivo()));
 }

 @ExceptionHandler(NegocioException.class)
 public ResponseEntity<ErrorResponse> handleNegocioHeredado(
 NegocioException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.CONFLICT,
 ex.getCodigo(),
 ex.getMessage(),
 request,
 null);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/exception/ErrorResponseFactory.java

```

package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import java.time.Instant;
import java.util.Map;
import java.util.UUID;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

/** Construcción común de respuestas de error. */
final class ErrorResponseFactory {
 private ErrorResponseFactory() {
 }

 static String newTraceId() {
 return UUID.randomUUID().toString();
 }

 static ResponseEntity<ErrorResponse> build(
 HttpStatus status,
 String codigo,
 String mensaje,
 HttpServletRequest request,
 Map<String, Object> detalles) {
 return build(
 status,
 codigo,
 mensaje,
 request,
 detalles,
 newTraceId());
 }

 static ResponseEntity<ErrorResponse> build(

```

```

 HttpStatus status,
 String codigo,
 String mensaje,
 HttpServletRequest request,
 Map<String, Object> detalles,
 String traceId) {
 ErrorResponse error = new ErrorResponse();
 error.setTimestamp(Instant.now().toString());
 error.setStatus(status.value());
 error.setCodigo(codigo);
 error.setMensaje(mensaje);
 error.setPath(request.getRequestURI());
 error.setTraceId(traceId);
 error.setDetalles(detalles);
 return ResponseEntity.status(status).body(error);
}
}

```

#### **src/main/java/es/meecd/demo/miproyecto/common/exception/ErrorTecnicoException.java**

```

package es.meecd.demo.miproyecto.common.exception;

/** Envuelve una causa técnica sin exponer detalles internos al cliente. */
public class ErrorTecnicoException extends AplicacionException {
 public ErrorTecnicoException(String mensaje, Throwable causa) {
 super("ERROR_TECNICO", mensaje, causa);
 }
}

```

#### **src/main/java/es/meecd/demo/miproyecto/common/exception/GenericExceptionHandler.java**

```

package es.meecd.demo.miproyecto.common.exception;

import es.meecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/** Última barrera: errores técnicos e inesperados. */
@RestControllerAdvice
@Order(3)
public class GenericExceptionHandler {
 private static final Logger LOG =
 LoggerFactory.getLogger(GenericExceptionHandler.class);

 @ExceptionHandler(ErrorTecnicoException.class)
 public ResponseEntity<ErrorResponse> handleTecnico(
 ErrorTecnicoException ex,
 HttpServletRequest request) {
 String traceId = ErrorResponseFactory.newTraceId();
 LOG.error(
 "Error técnico en {} - traceId={}",
 request.getRequestURI(),
 traceId,
 ex);
 return ErrorResponseFactory.build(
 HttpStatus.INTERNAL_SERVER_ERROR,
 ex.getCodigo(),
 "Error interno del servidor",
 request,
 null,
 traceId);
 }

 @ExceptionHandler(Exception.class)
 public ResponseEntity<ErrorResponse> handleGenerico(
 Exception ex,
 HttpServletRequest request) {
 String traceId = ErrorResponseFactory.newTraceId();
 }
}

```



```

 LOG.error(
 "Error inesperado en {} - traceId={}",
 request.getRequestURI(),
 traceId,
 ex);
 return ErrorResponseFactory.build(
 HttpStatus.INTERNAL_SERVER_ERROR,
 "ERROR_INTERNO",
 "Error interno del servidor",
 request,
 null,
 traceId);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/exception/NegocioException.java

```

package es.mecd.demo.miproyecto.common.exception;

/**
 * Excepción genérica heredada de M4.
 * Se conserva por compatibilidad; en M5 se prefieren tipos específicos.
 */
@Deprecated(forRemoval = false)
public class NegocioException extends AplicacionException {
 public NegocioException(String mensaje) {
 super("NEGOCIO", mensaje);
 }

 protected NegocioException(String codigo, String mensaje) {
 super(codigo, mensaje);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/exception/OperacionNoPermitidaException.java

```

package es.mecd.demo.miproyecto.common.exception;

/** Operación válida técnicamente pero prohibida por una regla de negocio. */
public class OperacionNoPermitidaException extends AplicacionException {
 public OperacionNoPermitidaException(String mensaje) {
 super("OPERACION_NO_PERMITIDA", mensaje);
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/common/exception/RecursoDuplicadoException.java

```

package es.mecd.demo.miproyecto.common.exception;

/** Conflicto por una clave o dato funcional ya existente. */
public class RecursoDuplicadoException extends NegocioException {
 private final String recurso;
 private final String campo;
 private final Object valor;

 public RecursoDuplicadoException(
 String recurso,
 String campo,
 Object valor) {
 super(
 "RECURSO_DUPLICADO",
 "Ya existe " + recurso + " con " + campo + " = " + valor);
 this.recurso = recurso;
 this.campo = campo;
 this.valor = valor;
 }

 public String getRecurso() {
 return recurso;
 }
}

```

```
 public String getCampo() {
 return campo;
 }

 public Object getValor() {
 return valor;
 }
}
```

#### **src/main/java/es/mecd/demo/miproyecto/common/exception/RecursoNoEncontradoException.java**

```
package es.mecd.demo.miproyecto.common.exception;

/** Recurso solicitado inexistente. */
public class RecursoNoEncontradoException extends AplicacionException {
 private final String recurso;
 private final Object id;

 public RecursoNoEncontradoException(String recurso, Object id) {
 super(
 "RECURSO_NO_ENCONTRADO",
 "No se encontró " + recurso + " con id " + id);
 this.recurso = recurso;
 this.id = id;
 }

 public String getRecurso() {
 return recurso;
 }

 public Object getId() {
 return id;
 }
}
```

#### **src/main/java/es/mecd/demo/miproyecto/common/exception/ValidacionNegocioException.java**

```
package es.mecd.demo.miproyecto.common.exception;

/** Regla funcional incumplida sobre un campo concreto. */
public class ValidacionNegocioException extends AplicacionException {
 private final String campo;
 private final String motivo;

 public ValidacionNegocioException(String campo, String motivo) {
 super(
 "VALIDACION_NEGOCIO",
 "El campo " + campo + " no cumple la regla: " + motivo);
 this.campo = campo;
 this.motivo = motivo;
 }

 public String getCampo() {
 return campo;
 }

 public String getMotivo() {
 return motivo;
 }
}
```

#### **src/main/java/es/mecd/demo/miproyecto/common/exception/ValidationExceptionHandler.java**

```
package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import es.mecd.demo.miproyecto.common.dto.error.ValidationError;
import jakarta.servlet.http.HttpServletRequest;
import java.time.Instant;
import java.util.List;
import java.util.UUID;
```

```

import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

/** Errores de contrato de entrada y validación. */
@RestControllerAdvice
@Order(1)
public class ValidationExceptionHandler {
 @ExceptionHandler(MethodArgumentNotValidException.class)
 public ResponseEntity<ErrorResponse> handleValidacion(
 MethodArgumentNotValidException ex,
 HttpServletRequest request) {
 List<ValidationError> errores = ex.getBindingResult()
 .getFieldErrors()
 .stream()
 .map(error -> new ValidationError(
 error.getField(),
 error.getRejectedValue(),
 error.getDefaultMessage()))
 .toList();
 ErrorResponse response = new ErrorResponse();
 response.setTimestamp(Instant.now().toString());
 response.setStatus(HttpStatus.BAD_REQUEST.value());
 response.setCodigo("VALIDACION");
 response.setMensaje("La petición tiene errores de validación");
 response.setPath(request.getRequestURI());
 response.setTraceId(UUID.randomUUID().toString());
 response.setErrors(errores);
 return ResponseEntity.badRequest().body(response);
 }

 @ExceptionHandler(HttpMessageNotReadableException.class)
 public ResponseEntity<ErrorResponse> handleMensajeNoLegible(
 HttpMessageNotReadableException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.BAD_REQUEST,
 "MENSAJE_NO_LEIBLE",
 "El cuerpo de la petición no se puede leer",
 request,
 null);
 }

 @ExceptionHandler(MissingServletRequestParameterException.class)
 public ResponseEntity<ErrorResponse> handleParametroFaltante(
 MissingServletRequestParameterException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.BAD_REQUEST,
 "PARAMETRO_FALTANTE",
 "Falta el parámetro " + ex.getParameterName(),
 request,
 null);
 }

 @ExceptionHandler(MethodArgumentTypeMismatchException.class)
 public ResponseEntity<ErrorResponse> handleTipoIncorrecto(
 MethodArgumentTypeMismatchException ex,
 HttpServletRequest request) {
 return ErrorResponseFactory.build(
 HttpStatus.BAD_REQUEST,
 "TIPO_INCORRECTO",
 "Valor no válido para " + ex.getName(),
 request,
 null);
 }
}

```

**src/main/java/es/mecd/demo/miproyecto/config/DatosInicialesBecasConfig.java**

```
package es.mecd.demo.miproyecto.config;
```

```

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.alumnoCurso;
import es.mecd.demo.miproyecto.alumnoCursoRepository;
import java.time.LocalDate;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
public class DatosInicialesBecasConfig {
 @Bean
 @Profile("dev")
 CommandLineRunner cargarDatos(CursoRepository cursos, AlumnoRepository alumnos) {
 return args -> {
 if (alumnos.count() == 0) {
 Curso curso = cursos.findByNombre("5º Primaria")
 .orElseGet(() -> cursos.save(new Curso("5º Primaria")));
 alumnos.save(new Alumno(
 "Ana", "García", "12345678A", LocalDate.of(2010, 5, 12), curso));
 }
 };
 }
}

```

#### src/main/java/es/mecd/demo/miproyecto/config/SecurityConfig.java

```

package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.auth.JwtAccessDeniedHandler;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationEntryPoint;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationFilter;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
 @Bean
 public PasswordEncoder passwordEncoder() {
 return PasswordEncoderFactories.createDelegatingPasswordEncoder();
 }

 @Bean
 public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
 return configuration.getAuthenticationManager();
 }

 @Bean
 public SecurityFilterChain filterChain(
 HttpSecurity http,
 JwtAuthenticationFilter filter,
 JwtAuthenticationEntryPoint entryPoint,
 JwtAccessDeniedHandler deniedHandler) throws Exception {
 http.cors(cors -> cors.configurationSource(corsConfigurationSource()))
 .csrf(csrf -> csrf.disable())
 .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
 .sessionManagement(session ->
 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
 }
}

```

```

 .authorizeHttpRequests(auth -> auth
 .requestMatchers(HttpMethod.POST,
 "/api/v1/auth/registro",
 "/api/v1/auth/login",
 "/api/v1/auth/refresh").permitAll()
 .requestMatchers("/h2-console/**", "/swagger-ui/**", "/swagger-ui.html", "/v3/api-
docs/**").permitAll()
 .anyRequest().authenticated())
 .exceptionHandling(ex ->
ex.authenticationEntryPoint(entryPoint).accessDeniedHandler(deniedHandler))
 .addFilterBefore(filter, UsernamePasswordAuthenticationFilter.class);
 return http.build();
 }

 @Bean
 public CorsConfigurationSource corsConfigurationSource() {
 CorsConfiguration config = new CorsConfiguration();
 config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:4200"));
 config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
 config.setAllowedHeaders(List.of("*"));
 config.setExposedHeaders(List.of("Location"));
 config.setAllowCredentials(true);
 config.setMaxAge(3600L);
 UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
 source.registerCorsConfiguration("/api/**", config);
 return source;
 }
}

```

#### src/main/resources/application-dev.properties

```

spring.datasource.url=jdbc:h2:mem:becasdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.h2.console.enabled=true
jwt.secret=MDEyMzQ1Njc4OWFiY2RlZjAxMjM0NTY3ODlhYmNkZWY=
jwt.expiration=900000
jwt.refresh-expiration=604800000

```

#### src/main/resources/application-test.properties

```

spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=false
spring.h2.console.enabled=false
jwt.secret=MDEyMzQ1Njc4OWFiY2RlZjAxMjM0NTY3ODlhYmNkZWY=
jwt.expiration=900000
jwt.refresh-expiration=604800000

```

#### src/main/resources/application.properties

```

spring.application.name=api-becas
spring.profiles.active=dev
spring.jpa.open-in-view=false
spring.jackson.serialization.write-dates-as-timestamps=false

```

#### src/test/java/es/mecd/demo/miproyecto/MiProyectoApplicationTest.java

```

package es.mecd.demo.miproyecto;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

```

```
import org.springframework.test.context.ActiveProfiles;
@SpringBootTest @ActiveProfiles("test")
class MiProyectoApplicationTest { @Test void contextoCarga() { } }
```

#### src/test/java/es/mecd/demo/miproyecto/alumno/AlumnoServiceTest.java

```
package es.mecd.demo.miproyecto.alumno;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;
import java.time.LocalDate;
import java.util.Optional;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
@ExtendWith(MockitoExtension.class)
class AlumnoServiceTest {
 @Mock AlumnoRepository alumnos; @Mock CursoRepository cursos; AlumnoService service;
 @BeforeEach void setUp() { service = new AlumnoService(alumnos, cursos); }
 @Test void crearDebeCrearCursoSiNoExiste() {
 AlumnoRequestDTO dto = new AlumnoRequestDTO(); dto.setNombre("Ana"); dto.setApellidos("García");
 dto.setDni("12345678A"); dto.setFechaNacimiento(LocalDate.of(2010,5,12)); dto.setCurso("5º
Primaria");
 Curso curso = new Curso("5º Primaria"); when(cursos.findByNombre("5º
Primaria")).thenReturn(Optional.of(curso));
 when(alumnos.save(org.mockito.ArgumentMatchers.any())).thenAnswer(i -> i.getArgument(0));
 assertEquals("Ana", service.crear(dto).getNombre());
 }
}
```

#### src/test/java/es/mecd/demo/miproyecto/auth/AuthServiceTest.java

```
package es.mecd.demo.miproyecto.auth;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import java.util.Optional;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.crypto.password.PasswordEncoder;
@ExtendWith(MockitoExtension.class)
class AuthServiceTest {
 @Mock AuthenticationManager manager; @Mock JwtService jwt; @Mock UsuarioRepository usuarios;
 @Mock RolRepository roles; @Mock PasswordEncoder encoder; AuthService service;
 @BeforeEach void setUp() { service = new AuthService(manager, jwt, usuarios, roles, encoder); }
 @Test void registrarDebeAsignarCiudadanoCuandoExiste() {
 Rol ciudadano = new Rol("CIUDADANO", "Ciudadano"); when(roles.findByNombre("CIUDADANO"))
 .thenReturn(Optional.of(ciudadano));
 when(encoder.encode("nuevo12345")).thenReturn("hash");
 when(usuarios.save(any())).thenAnswer(i -> { Usuario u=i.getArgument(0); try {
 var f=Usuario.class.getDeclaredField("id"); f.setAccessible(true); f.set(u,1L); }
 catch(Exception e){ throw new RuntimeException(e); } return u; });
 RegistroRequestDTO r=new RegistroRequestDTO(); r.setUsername("nuevo");
 r.setPassword("nuevo12345");
 r.setEmail("nuevo@educacion.gob.es");
 assertEquals("CIUDADANO", service.registrar(r).getRoles().get(0));
 }
}
```